



Python Junior-разработчик в обучающем мире Grow A Garden

Модуль 1 - Модуль 1 «Семена знаний»

Урок 1 - Посадка первого семени — print() и первая программа

Шаг 01:

Миссия 1

Герой Лисёнок нашёл первую лопатку и хочет посадить семя. Он услышал, что в Python можно «сказать слово» компьютеру. Какой командой это сделать?

Подсказка: Эта команда выводит текст на экран.

Ответ:

- `print()`
- `say()`
- `output()`
- `echo()`

Шаг 02:

Миссия 2

Друзья собрались написать первую программу. Расставь шаги в правильном порядке.

Подсказка: Сначала нужна идея, потом команда, затем запуск.

Ответ:

- Придумать, что напечатать

- Написать команду `print()` с текстом
- Запустить программу
- Увидеть результат на экране

🔮 Шаг 03:

Миссия 3

Герой Сэм просыпается в саду Grow a Garden и хочет показать всем надпись: "Добро пожаловать в сад!". Какой код напечатает это на экране?

Подсказка: Нужно использовать функцию `print` и строку в кавычках.

Ответ:

- **`print("Добро пожаловать в сад!")`**
- `print(Добро пожаловать в сад!)`
- `echo "Добро пожаловать в сад!"`

🔮 Шаг 04:

Миссия 4

Кира находит новое семечко и хочет поздравить: напечатать «Ура, семечко прорастает!». Какой вариант кода правильный?

Подсказка: Строка должна начинаться и заканчиваться кавычками; кавычки двойные или одинарные — оба варианта допустимы.

Ответ:

- **`print('Ура, семечко прорастает!')`**
- `print(Ура, семечко прорастает!)`
- `print("Ура, семечко прорастает!")`

🔮 Шаг 05:

Миссия 5

Минк хочет вывести два сообщения: сначала «Утро в саду.» потом «Пение птиц.» — каждое на новой строке. Что нужно использовать?

Подсказка: Либо два вызова `print`, либо символ новой строки `\n` внутри одной строки.

Ответ:

- **`print("Утро в саду.") print("Пение птиц.")`**
- `print("Утро в саду. Пение птиц.")`
- `print("Утро в саду\nПение птиц.")`

🔮 Шаг 06:

Миссия 6

Хоуп пишет: `print("Рассвет на горизонте)` . Почему программа выдаёт ошибку?

Подсказка: Обратите внимание на кавычки, начальные и конечные.

Ответ:

- **Отсутствует закрывающая кавычка**
- `print` должна быть с маленькой буквы
- Нельзя использовать двойные кавычки

🧙 Шаг 07:

Миссия 7

Задание: Сопоставьте код команды `print` с тем, что будет напечатано на экране.

Ответ:

- `print("Сад полон росы")`
- `print('Рассвет озарился')`
- `print("Ветер шепчет")`

Подсказка:

Вывод точно такой же, как содержимое строки, включая пробелы и заглавные буквы.

- **`print("Сад полон росы")`** — Сад полон росы
- **`print('Рассвет озарился')`** — Рассвет озарился
- **`print("Ветер шепчет")`** — Ветер шепчет

🧙 Шаг 08:

Миссия 8

Задание: Сопоставьте, какая команда `print` напечатает строку с кавычками внутри.

Ответ:

- `print('Она сказала: "Здравствуйте!"')`
- `print("Он ответил: 'Привет.'")`
- `print("Без кавычек внутри")`

Подсказка:

Чтобы внутри строки были кавычки, используйте разные виды кавычек снаружи и внутри или экранирование.

- **`print('Она сказала: "Здравствуйте!"')`** — Она сказала: "Здравствуйте!"
- **`print("Он ответил: 'Привет.'")`** — Он ответил: 'Привет.'
- **`print("Без кавычек внутри")`** — Без кавычек внутри

🧙 Шаг 09:

Миссия 9

Задание: Сопоставьте ситуации в саду с тем, что нужно напечатать, если герои говорят эти фразы.

Ответ:

- **Утро** — «Свет зари»
- **Полдень** — «Теплое солнце»
- **Вечер** — «Тишина и звёзды»

Подсказка:

Подумайте, как герои хотят разделить день: утро, полдень, вечер — и что они видят.

- **`print("Свет зари")`** — Утро — Свет зари
- **`print("Теплое солнце")`** — Полдень — Теплое солнце
- **`print("Тишина и звёзды")`** — Вечер — Тишина и звёзды

🔗 Шаг 10:

Миссия 10

Задание: Сопоставьте формат строки с тем, как она должна быть записана, чтобы code print работал корректно.

Ответ:

- Текст с апострофом: Его зовут О'Коннор
- **Текст с двойными кавычками внутри: "Роса",** — сказала пчёлка
- Обычный текст: Листья зелёные

Подсказка:

Используйте разные виды кавычек или экранирование внутри строки.

- **print("Его зовут О'Коннор")** — Текст с апострофом: Его зовут О'Коннор
- **print('"Роса", — сказала пчёлка')** — Текст с двойными кавычками внутри: "Роса", — сказала пчёлка
- **print("Листья зелёные")** — Обычный текст: Листья зелёные

🔗 Шаг 11:

Миссия 11

Вопрос: Команда героев хочет, чтобы их программа вывела три строки в таком порядке: 1) «Просыпается птица» 2) «Роса блестит» 3) «Свет зари» Какие три команды print в каком порядке дадут именно такой результат?

1. `print("Просыпается птица")`
2. `print("Свет зари")`
3. `print("Роса блестит")`

Подсказка:

Вывод происходит в том порядке, в каком пишутся команды.

Ответ:

- `print("Просыпается птица")`
- `print("Роса блестит")`
- `print("Свет зари")`

🔗 Шаг 12:

Миссия 12

Вопрос: Хоуп планирует рассказ дня: сначала «Утренний звон птиц», потом «Падение капель дождя», затем «Вечерняя тишина». Как расположить три команды print, чтобы получилось именно так?

1. `print("Падение капель дождя")`
2. `print("Утренний звон птиц")`
3. `print("Вечерняя тишина")`

Подсказка:

Учитывайте порядок: утро → день → вечер.

Ответ:

- `print("Утренний звон птиц")`
- `print("Падение капель дождя")`

- `print("Вечерняя тишина")`

🔮 Шаг 13:

Миссия 13

Вопрос: Когда вечер наступает, герои слышат звуки: «Сова кричит», «Тени падают», «Луна светит». Какую последовательность команд `print` использовать?

1. `print("Тени падают")`
2. `print("Сова кричит")`
3. `print("Луна светит")`

Подсказка:

Порядок должен отражать развитие событий: сначала тени, потом сова, потом луна.

Ответ:

- `print("Тени падают")`
- `print("Сова кричит")`
- `print("Луна светит")`

🔮 Шаг 14:

Миссия 14

Вопрос: Команда хочет рассказать утро в саду: «Рассвет», затем «Пение птиц», затем «Солнце клонилось к полудню». Какой порядок вызовов `print` даст такой рассказ?

1. `print("Солнце клонилось к полудню")`
2. `print("Рассвет")`
3. `print("Пение птиц")`

Подсказка:

Сначала рассвет, потом пение птиц, затем солнце — естественный порядок времени.

Ответ:

- `print("Рассвет")`
- `print("Пение птиц")`
- `print("Солнце клонилось к полудню")`

📖 Урок 2 - Грядка с вопросами — `input()` и пользовательский ввод

🔮 Шаг 01:

Миссия 1

Алиса из команды `Grow a Garden` хочет узнать, сколько семян у неё осталось. Она вводит число семян через `input()`. Какой тип данных получится сразу после `input()` ?

Подсказка: `input()` всегда возвращает, что-то, что нужно преобразовать, если нужен другой тип.

Ответ:

- Строка (`str`)
- Целое число (`int`)
- Вещественное число (`float`)
- Булево значение (`bool`)

🔗 Шаг 02:

Миссия 2

Ботаник Боб хочет получить имя растения от пользователя, чтобы подписать грядку. Он пишет `name = input("Как зовут ваше растение? ")`. Что будет в переменной `name` после ввода?

Подсказка: Пользователь может ввести слова, пробелы, символы.

Ответ:

- Строка (**str**) с тем, что пользователь ввёл
- Число, если пользователь введёт цифры
- Список символов
- Тип данных зависит от текста

🔗 Шаг 03:

Миссия 3

Когда Алиса хочет узнать, сколько воды надо растениям, она запрашивает число литров: `water = input("Сколько литров воды? ")`. Чтобы сложить это значение с другим числом, что надо сделать?

Подсказка: Тип строки нельзя просто так складывать с числом — это приведёт к ошибке.

Ответ:

- Преобразовать строку в число, например с помощью **`int()`** или **`float()`**
- Оставить как есть, Python сам поймёт
- Использовать только `+` и `-`
- Сразу запросить как число, без `input()`

🔗 Шаг 04:

Миссия 4

Формулировка: Сопоставьте фрагмент кода с тем, что он делает.

Фрагменты кода:

Ответ:

- `input()`
- `int(input())`
- `float(input())`
- `input().strip()`

Описание:

1. Запрашивает ввод и убирает пробелы слева и справа
2. Запрашивает ввод, даёт строку
3. Запрашивает ввод, преобразует в вещественное число
4. Запрашивает ввод, преобразует в целое число

🔗 Подсказка:

Подумайте, какой тип данных нужен в каждой ситуации, и что делают методы вроде `strip()`.

- **`input()`** — Запрашивает ввод, даёт строку
- **`int(input())`** — Запрашивает ввод, преобразует в целое число
- **`float(input())`** — Запрашивает ввод, преобразует в вещественное число
- **`input().strip()`** — Запрашивает ввод и убирает пробелы слева и справа

🔮 Шаг 05:

Миссия 5

Формулировка: Сопоставьте, что введёт пользователь, и как к этому относится Python.

Ввод пользователя:

Ответ:

- "42"
- "3.14"
- "яблоко"
- " растение "

Интерпретация:

1. Строка без пробелов в начале и конце
2. Вещественное число
3. Строка как есть
4. Целое число

🔮 **Подсказка:**

Посмотрите, как работает `strip()` , и что означает число с точкой для Python.

- "**42**" — Целое число
- "**3.14**" — Вещественное число
- "**яблоко**" — Строка как есть
- " **растение** " — Строка без пробелов в начале и конце

🔮 Шаг 06:

Миссия 6

Формулировка: Сопоставьте, какая функция или метод работает с вводом, и что она делает.

Функции / методы:

Ответ:

- `upper()`
- `lower()`
- `strip()`
- `split()`

Определения:

1. Разбивает строку на список слов по пробелам
2. Убирает пробелы по краям
3. Делает все буквы заглавными
4. Делает все буквы строчными

🔮 **Подсказка: Попробуйте представить, что если пользователь вводит " РоЗовоЕ ЦветОК ". Что сделает каждая функция?**

- **`upper()`** — Делает все буквы заглавными
- **`lower()`** — Делает все буквы строчными
- **`strip()`** — Убирает пробелы по краям
- **`split()`** — Разбивает строку на список слов по пробелам

🔗 Шаг 07:

Миссия 7

Вопрос: Расположите шаги в правильном порядке, чтобы ботаник Боб собрал данные от пользователя, обработал их и вывел итоговое сообщение о растении:

1. Вывести сообщение с итогами
2. Запросить данные с помощью `input()`
3. Преобразовать ввод в нужный тип (`int` или `float` если нужно)
4. Проверить введенные данные на корректность

🔗 Подсказка:

Подумайте, что должен сделать код, прежде чем что-то использовать, и в какой момент можно проверить, что введено правильно.

Ответ:

- Запросить данные с помощью `input()`
- Проверить введенные данные на корректность
- Преобразовать ввод в нужный тип (`int` или `float` если нужно)
- Вывести сообщение с итогами

🔗 Шаг 08:

Миссия 8

Вопрос: Расположите по порядку, что делает программа, когда пользователи вводят температуру, влажность и свет для растения, а затем программа рассчитывает, можно ли полить:

1. Сравнить значения с порогом
2. Запросить температуру, влажность, свет с помощью `input()`
3. Вывести решение: полить или нет
4. Преобразовать введенные строки в числа

🔗 Подсказка:

Обратите внимание: сравнение чисел возможно только когда данные — числа, а не строки.

Ответ:

- Запросить температуру, влажность, свет с помощью `input()`
- Преобразовать введенные строки в числа
- Сравнить значения с порогом
- Вывести решение: полить или нет

🔗 Шаг 09:

Миссия 9

Вопрос: Выстройте правильную последовательность, как `Grow a Garden` проверяет, достаточно ли семян у игрока, чтобы посадить новый ряд:

1. Сравнить число нужных семян с имеющимися
2. Запросить у пользователя, сколько семян он хочет потратить
3. Преобразовать ввод в целое число
4. Вывести сообщение, что семян недостаточно или посадка успешна

🔗 Подсказка:

Подумайте, без чего последовательность нарушится: когда проверка будет до преобразования или до запроса, что-то может сломаться.

Ответ:

- Запросить у пользователя, сколько семян он хочет потратить
- Преобразовать ввод в целое число
- Сравнить число нужных семян с имеющимися
- Вывести сообщение, что семян недостаточно или посадка успешна

Шаг 10:

Миссия 9

Сопоставьте команды и то, что они делают.

Команды:

Ответ:

- `input("Сколько дней прошло?")`
- `print("Доброе утро, садовник!")`
- `name = input("Как тебя зовут?")`
- `print(age)`

Что они делают:

1. Выводят текст на экран
2. Запрашивают ввод от пользователя
3. Сохраняют введённый текст в переменную
4. Печатают содержимое переменной на экран

Подсказка: Некоторые команды сначала что-то просят, другие — показывают.

- **`input("Сколько дней прошло?")`** — Запрашивают ввод от пользователя
- **`print("Доброе утро, садовник!")`** — Выводят текст на экран
- **`name = input("Как тебя зовут?")`** — Сохраняют введённый текст в переменную
- **`print(age)`** — Печатают содержимое переменной на экран

Шаг 11:

Миссия 10

Сопоставьте, что вернёт функция `input()` и что будет содержаться в переменной:

Выражения:

Ответ:

- `input("Введите число")`, пользователь ввёл 42
- `input("Введите слово")`, пользователь ввёл "сад"
- `input("Сколько цветов?")`, пользователь ввёл 3
- `input("Полить? (да/нет)")`, пользователь ввёл "нет"

Результаты:

1. "42"
2. "сад"
3. "3"
4. "нет"

Подсказка: Всё, что вводится — это строка, даже если кажется числом.

- `input("Введите число")`, пользователь ввёл 42 — "42"
- `input("Введите слово")`, пользователь ввёл "сад" — "сад"
- `input("Сколько цветов?")`, пользователь ввёл 3 — "3"
- `input("Полить? (да/нет)")`, пользователь ввёл "нет" — "нет"

🔮 Шаг 12:

Миссия 11

Сопоставьте переменные и значения, которые они могут содержать после использования `input()`:

Переменные:

Ответ:

- `username`
- `age`
- `water_choice`
- `plant_name`

Возможные значения:

1. "Роза"
2. "17"
3. "да"
4. "Аня"

Подсказка: Подумайте, что могла ввести персонаж, в зависимости от того, какую информацию спрашивают.

- **`username`** — "Аня"
- **`age`** — "17"
- **`water_choice`** — "да"
- **`plant_name`** — "Роза"

🔮 Шаг 13:

Миссия 12

Расположите этапы, которые выполняет программа, когда использует `input()` и `print()`, в правильном порядке:

1. Пользователь вводит текст
2. Программа показывает приглашение к вводу
3. Программа сохраняет ввод в переменную
4. Программа выводит или использует сохранённую информацию

Подсказка: Что должно произойти первым, а что — в конце?

Ответ:

- Программа показывает приглашение к вводу
- Пользователь вводит текст
- Программа сохраняет ввод в переменную
- Программа выводит или использует сохранённую информацию

🔮 Шаг 14:

Миссия 13

Расположите шаги от простого запроса до обработки ввода ``да/нет`` в игре:

1. Спрашиваем: "Хочешь полить растение? (да/нет)"
2. Сохраняем ответ в переменную
3. Выводим приглашение к вводу
4. Сравниваем ответ с "да" и дальше решаем, поливать или нет

Подсказка: Какой логический шаг идёт после того, как вы получили ответ?

Ответ:

- Спрашиваем: "Хочешь полить растение? (да/нет)"
- Выводим приглашение к вводу
- Сохраняем ответ в переменную
- Сравниваем ответ с "да" и дальше решаем, поливать или нет

Шаг 15:

Миссия 14

Расположите этапы, которые Лиза выполняет, когда настраивает опрос: "Введите ваше имя", "Введите ваш возраст", и затем отображает оба значения.

1. Спрашиваем возраст
2. Выводим "Ваше имя: [имя]"
3. Спрашиваем имя
4. Выводим "Ваш возраст: [возраст]"

Подсказка: Сначала соберите всё, что нужно, потом показывайте отдельно имя и возраст.

Ответ:

- Спрашиваем возраст
- Выводим "Ваше имя: [имя]"
- Спрашиваем имя
- Выводим "Ваш возраст: [возраст]"

Шаг 16:

Миссия 15

Расположите части программы, в которой садовник спрашивает у пользователя: имя, растение, потом говорит: "Привет, [имя]! Твоё растение — [растение]."

1. Сохраняем имя
2. Выводим приветствие с именем и растением
3. Сохраняем растение
4. Запрашиваем у пользователя растение
5. Запрашиваем у пользователя имя

Подсказка: Подумайте, в каком порядке программа должна запрашивать, сохранять, а потом выводить всё вместе.

Ответ:

- Сохраняем имя
- Выводим приветствие с именем и растением
- Сохраняем растение
- Запрашиваем у пользователя растение
- Запрашиваем у пользователя имя

Урок 3 - Инвентарь садового — переменные и имена

Шаг 01:

Миссия 1

Задание: Сопоставьте имена переменных с их описаниями.

Ответ:

- `seed_count`
- `plant_name`
- `water_amount`
- `is_growing`

Описание:

1. Название растения
2. Количество семян
3. Количество воды для полива
4. Состояние: растёт или нет (True/False)

Подсказка: Имя переменной часто отражает её смысл и тип данных.

- **`seed_count`** — Количество семян
- **`plant_name`** — Название растения
- **`water_amount`** — Количество воды для полива
- **`is_growing`** — Состояние: растёт или нет (True/False)

Шаг 02:

Миссия 2

Задание: Сопоставьте типы данных с подходящими переменными из Grow a Garden.

Ответ:

- `seeds`
- `plant_name`
- `is_watered`
- `height`

Типы данных:

1. Целое число (int)
2. Строка (str)
3. Булево (bool)
4. Вещественное число (float)

Подсказка: Подумайте, как измеряются разные характеристики растений.

- **`seeds`** — Целое число (int)
- **`plant_name`** — Строка (str)
- **`is_watered`** — Булево (bool)
- **`height`** — Вещественное число (float)

Шаг 03:

Миссия 3

Задание: Сопоставьте этапы работы с переменной в Grow a Garden.

Ответ:

- Записать количество семян в переменную
- Проверить, есть ли достаточно семян для посадки
- Обновить количество семян после посадки
- Вывести сообщение об успехе

Последовательность:

1. Присвоить значение переменной
2. Сравнить значение с нужным количеством
3. Изменить значение переменной (вычесть)
4. Показать результат пользователю

Подсказка: Подумайте, как меняется информация на каждом шаге.

- **Записать количество семян в переменную** — Присвоить значение переменной
- **Проверить, есть ли достаточно семян для посадки** — Сравнить значение с нужным количеством
- **Обновить количество семян после посадки** — Изменить значение переменной (вычесть)
- **Вывести сообщение об успехе** — Показать результат пользователю

 Шаг 04:

Миссия 4

Герой Сэм хочет хранить в переменной число семян, которые он посеял. Какое имя переменной будет наиболее подходящим?

Подсказка: Имя переменной должно быть описательным, без пробелов, начинаться с буквы или подчёркивания.

Ответ:

- **seed_count**
- number of seeds
- 123seeds

 Шаг 05:

Миссия 5

Кира хочет сохранить название сорта растения в переменную. Какой из вариантов кода корректен?

Подсказка: Строки заключаются в кавычки, имя переменной — без кавычек.

Ответ:

- **plant_type = "Роза"**
- "plant_type" = Роза
- plant_type "Роза"

 Шаг 06:

Миссия 6

Минк хочет использовать переменную, которая хранит температуру в саду. Что будет корректным присваиванием?

Подсказка: Числа (целые или с плавающей точкой) записываются без кавычек.

Ответ:

- **temperature = 23.5**
- temperature = "23.5"
- "temperature" = 23.5

Шаг 07:

Миссия 7

Хоуп хочет использовать переменную, чтобы хранить количество воды (в литрах), которую поливает растения. Какое имя лучше использовать?

Подсказка: Имя с подчёркиванием удобно, избегайте специальных символов.

Ответ:

- **water_volume**
- water-volume
- volume water

Шаг 08:

Миссия 8

Задание: Сопоставьте имена переменных с тем, что они хранят в саду Grow a Garden.

Ответ:

- seed_count
- plant_name
- soil_moisture

Подсказка: Посмотрите на смысл слов: count — число, moisture — влажность.

- **seed_count** — число семян
- **plant_name** — название растения
- **soil_moisture** — влажность почвы

Шаг 09:

Миссия 9

Задание: Сопоставьте неправильные имена переменных с причинами, почему они плохо подходят.

Ответ:

- 2seeds
- plant type
- soil-moisture

Подсказка: Переменные не могут начинаться с цифр, не должны содержать пробелов или дефисов.

- **2seeds** — начинается с цифры
- **plant type** — содержит пробел
- **soil-moisture** — содержит дефис

Шаг 10:

Миссия 10

Задание: Сопоставьте переменные с типом данных, который они, скорее всего, будут хранить.

Ответ:

- seed_count
- plant_name
- is_wet

Подсказка: Boolean переменные часто начинаются с is-, правильные имена — подсказка про тип данных.

- **seed_count** — целое число (int)
- **plant_name** — строка (string)
- **is_wet** — булево значение (True или False)

Шаг 11:

Миссия 11

Вопрос: Кира записала три переменные: 1) soil_moisture 2) plant_name 3) seed_count Но хочет вывести их в порядке: название растения, число семян, влажность почвы. Как правильно расположить команды print?

1. print("soil_moisture")
2. print("plant_name")
3. print("seed_count")

Подсказка: Порядок вывода зависит от порядка команд print в коде.

Ответ:

- print(plant_name)
- print(seed_count)
- print(soil_moisture)

Шаг 12:

Миссия 12

Вопрос: Герой Сэм хочет записать три строки в этот порядок: 1) Название растения 2) Температура в саду 3) Влажность почвы Какие три вызова print дадут именно такой вывод?

1. print("humidity")
2. print("plant_name")
3. print("temperature")

Подсказка: Подумайте, какие имена переменных соответствуют каждому элементу.

Ответ:

- print(plant_name)
- print(temperature)
- print(soil_moisture)

Шаг 13:

Миссия 13

Вопрос: Команда хочет вывести сначала информацию о растении, затем о влажности, затем о boolean-флаге, показывающем, полита ли почва (is_wet). Как правильно расположить команды?

1. print("is_wet")

```
2. print("plant_name")
3. print("soil_moisture")
```

Подсказка: Сначала переменная, которая описывает растение, потом та, которая измеряет влажность, затем флаг True/False.

Ответ:

- `print(plant_name)`
- `print(soil_moisture)`
- `print(is_wet)`

Урок 4 - Песчаная почва — типы данных `int`, `float`, `str`, `bool`

Шаг 01:

Миссия 1

🕒 Минк нашёл записку от старого садовника: `seeds = 5` . Какой тип данных у переменной `seeds` ?

Подсказка: Число без кавычек — это один из самых базовых типов.

Ответ:

- **`int`**
- `str`
- `float`
- `bool`

Шаг 02:

Миссия 2

🌀 Хоуп записала температуру воздуха так: `temperature = 18.5` . Какой тип данных она использует?

Подсказка: Это число, но с десятичной точкой!

Ответ:

- **`float`**
- `int`
- `bool`
- `str`

Шаг 03:

Миссия 3

🌸 Кира назвала растение: `plant_name = "Астра"` . Какой у этой переменной тип данных?

Подсказка: Кавычки — важный признак этого типа.

Ответ:

- **`str`**
- `int`
- `float`
- `bool`

Шаг 04:

Миссия 4

📌 Сэм задал переменную `is_raining = True` . Какой у неё тип данных?

Подсказка: Это тип, который может принимать только два значения: истина или ложь.

Ответ:

- **bool**
- str
- int
- float

💡 Шаг 05:

Миссия 5

🌿 У Хоуп есть переменная `watering_can = "10"` . Какой у неё тип данных?

Подсказка: Внешне это похоже на число, но заключено в кавычки.

Ответ:

- **str**
- int
- float
- bool

💡 Шаг 06:

Миссия 6

🍂 Сэм собрал разные значения. Помогите ему сопоставить значения с типами данных.

Подсказка: Числа без точек — это int, с точками — float, логические значения — True/False.

Ответ:

- **"Подсолнух"** — str
- **25** — int
- **3.14** — float
- **False** — bool

💡 Шаг 07:

Миссия 7

Кира исследует почву. Помогите ей соотнести переменные и их типы данных.

Подсказка: `is_...` — часто булевы, `moisture` — дробные значения, `name` — строки.

Ответ:

- **plant_name = "Гвоздика"** — str
- **is_wet = True** — bool
- **soil_ph = 5.6** — float
- **seed_count = 30** — int

💡 Шаг 08:

Миссия 8

🌱 Минк записывает: `plants = 10` . Какой тип данных у переменной `plants` ?

Подсказка: Это целое число без десятичной точки.

Ответ:

- **int**

- float
- str
- bool

🔧 Шаг 09:

Миссия 9

☀ Кира записала влажность почвы: `moisture = 76.5` . Какой тип данных у этой переменной?

Подсказка: Есть дробная часть — не целое число.

Ответ:

- **float**
- int
- str
- bool

🔧 Шаг 10:

Миссия 10

🌸 Когда Хоуп говорит: `flower = "Роза"` , какой тип данных у переменной `flower` ?

Подсказка: Строки берутся в кавычки.

Ответ:

- **str**
- int
- float
- bool

🔧 Шаг 11:

Миссия 11

☁ Сэм устанавливает флаг: `is_wet = False` . Какой тип данных используется?

Подсказка: Значения `True` и `False` — это специальный тип.

Ответ:

- **bool**
- int
- str
- float

🔧 Шаг 12:

Миссия 12

🐞 Сопоставьте переменные с их типами данных.

Ответ:

- `seed_count = 20`
- `pH = 6.7`
- `garden_name = "Зелёный уголок"`
- `has_weed = True`

Подсказка: Числа без точки — `int`; с точкой — `float`; кавычки — строка; `True/False` — `bool`.

- **seed_count = 20** — int
- **pH = 6.7** — float
- **garden_name = "Зелёный уголок"** — str
- **has_weed = True** — bool

🧭 Шаг 13:

Миссия 13

🐛 Минк фиксирует показатели садовых растений. Сопоставьте значения с типами.

Ответ:

- "25"
- 30
- 0.001
- False

Подсказка: Если в кавычках — это строка, даже если там цифры; логические значения без кавычек.

- "25" — str
- 30 — int
- 0.001 — float
- False — bool

🧭 Шаг 14:

Миссия 14

🌸 Кира сравнивает записи: сопоставьте переменные с тем, какой тип данных они должны быть, чтобы код работал правильно.

Ответ:

- light_level = "High"
- rainfall = 0
- soil_temp = 12.3
- is_harvest_ready = False

Подсказка: Обратите внимание: 0 — это целое число; значение внутри кавычек — строка.

- light_level = "High" — str
- rainfall = 0 — int
- soil_temp = 12.3 — float
- is_harvest_ready = False — bool

🧭 Шаг 15:

Миссия 15

Вопрос: Команда героев хочет упорядочить четыре значения по возрастанию сложности типов так: строка (str), целое число (int), дробное число (float), булево (bool). Выберите правильный порядок:

1. "семя"
2. True
3. 5

4. 2.71

Подсказка: Какие типы приходят к каким: строка — текст; bool — True/False; числовые — int или float.

Ответ:

- "семя"
- 5
- 2.71
- True

Шаг 16:

Миссия 16

Вопрос: Хоуп записала эти четыре значения: 1) 0 2) "0" 3) False 4) 0.0 Она хочет, чтобы программа вывела их в порядке: строка, булево, целое число, дробное число. Какой порядок вызовов print даст такой вывод?

1. print(0)
2. print("0")
3. print(False)
4. print(0.0)

Подсказка: Строка — текст в кавычках; bool — True/False; int — число без точки; float — с точкой.

Ответ:

- print("0")
- print(False)
- print(0)
- print(0.0)

Урок 5 - Солнечный день — арифметические операции

Шаг 01:

Миссия 1

Сэм в саду собирает ягоды: у него было 5 ягод, он добавил ещё 3. Какое выражение правильно покажет общее число ягод?

Подсказка: Используйте оператор + для сложения целых чисел.

Ответ:

- 5 + 3
- 5 - 3
- 5 * 3

Шаг 02:

Миссия 2

Минк хочет узнать, сколько семян останется, если из 20 семян разделить по 6, используя целочисленное деление //. Что вернёт 20 // 6 ?

Подсказка: // — это целочисленное деление, округляет вниз.

Ответ:

- 3

- 3.333...
- 4

Шаг 03:

Миссия 3

 Сопоставьте выражения с их результатами.

Ответ:

- 8 - 3
- 4 * 2.5
- 9 / 3
- 7 // 2

Подсказка: Деление / даёт float, // — целое число при целых операндах.

- 8 - 3 — 5
- 4 * 2.5 — 10.0
- 9 / 3 — 3.0
- 7 // 2 — 3

Шаг 04:

Миссия 4

Вопрос: Хоуп хочет, чтобы программа вывела результаты выражений в таком порядке: 1) 8 - 3 2) 9 / 3 3) 4 * 2.5 4) 10 % 3 Выберите правильную последовательность команд print.

1. print("9 / 3")
2. print("10 % 3")
3. print("8 - 3")
4. print("4 * 2.5")

Подсказка: Вывод должен идти в том порядке, в каком вы пишете print.

Ответ:

- print(8 - 3)
- print(9 / 3)
- print(4 * 2.5)
- print(10 % 3)

Шаг 05:

Миссия 5

Вопрос: Сэм хочет вывести сначала результат умножения, затем целочисленного деления, потом остатка, и в конце сложения. Выражения: "4 * 3", "7 // 2", "10 % 4", "5 + 5" Какой порядок вызовов print даст такой вывод?

1. print("4 * 3")
2. print("7 // 2")
3. print("10 % 4")
4. print("5 + 5")

Подсказка: Сначала умножение, потом //, %, + в нужном порядке.

Ответ:

- `print(4 * 3)`
- `print(7 // 2)`
- `print(10 % 4)`
- `print(5 + 5)`

 **Шаг 06:**

Миссия 6

Вопрос: Кира собрала числа: "6 / 3" , "2 * 5" , "9 - 1" , "14 // 4" Она хочет вывести их в порядке: вычитание, деление, умножение, целочисленное деление. Как должны идти `print`-команды?

1. `print("9 - 1")`
2. `print("6 / 3")`
3. `print("2 * 5")`
4. `print("14 // 4")`

Подсказка: Определите, какие выражения соответствуют каждому оператору, и упорядочьте их.

Ответ:

- `print(9 - 1)`
- `print(6 / 3)`
- `print(2 * 5)`
- `print(14 // 4)`

 **Шаг 07:**

Миссия 7

Вопрос: Минк хочет сначала показать результат целочисленного деления, затем сложения, потом умножения и в конце остатка от деления. Значения: "14 // 3" , "5 + 4" , "3 * 2" , "10 % 3" Какой порядок `print`-команд даст такой вывод?

1. `print("14 // 3")`
2. `print("5 + 4")`
3. `print("3 * 2")`
4. `print("10 % 3")`

Подсказка: Определите каждое выражение: //, +, *, % — и выстройте порядок по условию.

Ответ:

- `print(14 // 3)`
- `print(5 + 4)`
- `print(3 * 2)`
- `print(10 % 3)`

 **Шаг 08:**

Миссия 8

Вопрос: Хоуп хочет вывести четыре результата арифметических операций в порядке: деление /, умножение *, вычитание -, сложение +. Выражения: "8 / 2" , "5 * 3" , "10 - 4" , "1 + 1" Какой порядок команд `print` нужно использовать?

1. `print("8 / 2")`

2. `print("5 * 3")`
3. `print("10 - 4")`
4. `print("1 + 1")`

Подсказка: Следите за оператором в каждом выражении и порядок, который сказано в условии.

Ответ:

- `print(8 / 2)`
- `print(5 * 3)`
- `print(10 - 4)`
- `print(1 + 1)`

Шаг 09:

Миссия 9

Сэм собрал в саду 7 ягод и нашёл ещё 5. Какое выражение правильно покажет общее число ягод?

Подсказка: Чтобы сложить числа, используйте оператор `+`.

Ответ:

- **`7 + 5`**
- `7 - 5`
- `7 * 5`

Шаг 10:

Миссия 10

Кира поливает 12 растений и хочет разделить 24 литра воды поровну. Какое выражение покажет количество воды на каждое растение?

Подсказка: Для деления используйте оператор `/`, результат может быть дробным числом.

Ответ:

- **`24 / 12`**
- `24 + 12`
- `24 // 12`

Шаг 11:

Миссия 11

 Сопоставьте выражения с их результатами.

Ответ:

- `10 - 4`
- `3 * 3.5`
- `9 / 3`
- `15 // 4`

Подсказка: Деление `/` даёт дробный результат, а `//` — целочисленное.

- **`10 - 4` — 6**
- **`3 * 3.5` — 10.5**
- **`9 / 3` — 3.0**
- **`15 // 4` — 3**

🔮 Шаг 12:

Миссия 12

💡 Кира изучает остаток от деления (оператор %). Сопоставьте выражения с их остатками.

Ответ:

- 14 % 5
- 20 % 6
- 9 % 3
- 7 % 2

Подсказка: Остаток — это то, что остаётся после деления без учёта целых частей.

- **14 % 5** — 4
- **20 % 6** — 2
- **9 % 3** — 0
- **7 % 2** — 1

🔮 Шаг 13:

Миссия 13

💡 Помогите Сэму сопоставить выражения с их правильными значениями:

Ответ:

- 7 + 3
- 16 // 5
- 18 % 4
- 5 * 4.0

Подсказка: Обратите внимание на целочисленное деление и остаток.

- **7 + 3** — 10
- **16 // 5** — 3
- **18 % 4** — 2
- **5 * 4.0** — 20.0

🔮 Шаг 14:

Миссия 14

Вопрос: Кира хочет вывести результаты вычислений в порядке: сложение, умножение, деление, целочисленное деление.

Выражения: 7 + 3, 5 * 4, 16 / 5, 19 // 4

Подсказка: Вывод будет именно в том порядке, в каком расположены print-команды.

1. print(7 + 3)
2. print(5 * 4)
3. print(16 / 5)
4. print(19 // 4)

Ответ:

- print(7 + 3)
- print(5 * 4)
- print(16 / 5)
- print(19 // 4)

Урок 6 - Чудо-компост — преобразование типов

Шаг 01:

Миссия 1

Фея Листик собрала урожай и получила число "5" как строку. Что нужно сделать, чтобы превратить эту строку в число и посчитать удвоенный урожай?

Подсказка: Подумайте, как в Python преобразовать текст в число.

Ответ:

- **Использовать `int('5')`**
- Просто написать `'5' * 2`
- Использовать `str(5)`
- Использовать `float('5')`

Шаг 02:

Миссия 2

Садовник Роуз измерил длину грядки и получил результат 7.8 в виде числа. Он хочет вывести это число как строку, чтобы отправить другу в сообщении. Как это сделать?

Подсказка: Преобразование числа в текст — обратный процесс.

Ответ:

- **Использовать `str(7.8)`**
- Использовать `int(7.8)`
- Использовать `float('7.8')`

Шаг 03:

Миссия 3

Колдун Садовой получил количество волшебных семян в виде строки "12". Он хочет узнать, будет ли это количество больше 10. Что нужно сделать для правильного сравнения?

Подсказка: Нельзя сравнивать число и строку напрямую — надо привести к одному типу.

Ответ:

- **Преобразовать строку в число с помощью `int("12")`**
- Сравнивать как строки `"12" > "10"`
- Преобразовать число 10 в строку и сравнить
- Преобразовать в float

Шаг 04:

Миссия 4

Волшебник Мультик вызвал магию преобразования типов. Сопоставьте тип данных и пример корректного преобразования.

Ответ:

- 1) Преобразование в целое число
- 2) Преобразование в строку
- 3) Преобразование в число с плавающей точкой

1. `float('3.14')`

2. `str(42)`

3. `int('7')`

Подсказка: Внимательно смотрите, какой тип у результата и что был на входе.

- **Преобразование в целое число** — `int('7')`
- **Преобразование в строку** — `str(42)`
- **Преобразование в число с плавающей точкой** — `float('3.14')`

🔮 Шаг 05:

Миссия 5

Фея Перо находит у себя в инвентаре значения разных типов. Помогите ей сопоставить значения с их типами.

Ответ:

- 1) `"123"`
- 2) `45`
- 3) `3.1415`
- 4) `True`

1. `int`
2. `str`
3. `float`
4. `bool`

Подсказка: Обратите внимание, в кавычках — это строка.

- **`"123"`** — `str`
- **`45`** — `int`
- **`3.1415`** — `float`
- **`True`** — `bool`

🔮 Шаг 06:

Миссия 6

Сопоставьте функции преобразования Python с их результатом при вызове с аргументом `"10"`:

Ответ:

- 1) `int("10")`
- 2) `float("10")`
- 3) `str(10)`

1. `"10"` (строка)
2. `10` (целое число)
3. `10.0` (число с плавающей точкой)

Подсказка: Помните, что преобразование к `str` даёт текст, а `int` и `float` — числа.

- **`int("10")`** — `10` (целое число)
- **`float("10")`** — `10.0` (число с плавающей точкой)
- **`str(10)`** — `"10"` (строка)

🔮 Шаг 07:

Миссия 7

Помогите Садовнику правильно распорядиться преобразованием типов для сбора урожая в правильном порядке:

1. Получить данные о количестве семян в строке "15".
2. Преобразовать строку в число для подсчёта.
3. Посчитать удвоенный урожай.
4. Преобразовать результат обратно в строку для отображения.

Подсказка: Подумайте, в каком порядке нужно применять преобразования, чтобы всё работало правильно.

Ответ:

- Получить данные о количестве семян в строке "15".
- Преобразовать строку в число для подсчёта.
- Посчитать удвоенный урожай.
- Преобразовать результат обратно в строку для отображения.

Шаг 08:

Миссия 8

Сопоставьте правильный порядок действий для корректного суммирования количества волшебных плодов:

1. Получить два значения в виде строк: "8" и "12".
2. Преобразовать строки в числа.
3. Сложить числа.
4. Вывести результат.

Подсказка: Вспомните, почему нельзя просто складывать строки, если нужны числа.

Ответ:

- Получить два значения в виде строк: "8" и "12".
- Преобразовать строки в числа.
- Сложить числа.
- Вывести результат.

Шаг 09:

Миссия 9

Расположите этапы превращения числа 5.5 из строки в целое число:

1. Получить строку "5.5".
2. Преобразовать строку в float.
3. Преобразовать float в int.
4. Использовать результат.

Подсказка: Прямое преобразование строки с точкой в int приведёт к ошибке.

Ответ:

- Получить строку "5.5".
- Преобразовать строку в float.
- Преобразовать float в int.
- Использовать результат.

Шаг 10:

Миссия 10

Помогите Алхимику упорядочить процесс преобразования для вычисления итогового веса урожая:

1. Прочитать вес в виде строки "23.7".
2. Преобразовать строку в число с плавающей точкой.
3. Умножить вес на количество растений (5).
4. Преобразовать результат в строку для отчёта.

Подсказка: Не забывайте о порядке операций с типами данных.

Ответ:

- Прочитать вес в виде строки "23.7".
- Преобразовать строку в число с плавающей точкой.
- Умножить вес на количество растений (5).
- Преобразовать результат в строку для отчёта.

Шаг 11:

Миссия 11

Земляничка нашла семена, но они пришли в виде строки "42". Как преобразовать строку в число, чтобы посчитать, сколько семян посадить?

Подсказка: Подумайте, какой тип данных представляет число в Python.

Ответ:

- **Использовать функцию `int()`**
- Использовать функцию `str()`
- Использовать функцию `float()`

Шаг 12:

Миссия 12

Пчёлка Пыльцек собирает нектар и измеряет количество в виде числа 3.14. Как сохранить это значение в виде строки, чтобы отправить другу?

Подсказка: Для отправки сообщений часто нужны строки.

Ответ:

- **Использовать функцию `str()`**
- Использовать функцию `int()`
- Использовать функцию `list()`

Шаг 13:

Миссия 13

Гном-огородник хочет преобразовать строку "7.5" в число с плавающей точкой. Какую функцию нужно применить?

Подсказка: Подумайте, какой тип данных хранит дробные числа.

Ответ:

- **Использовать функцию `float()`**
- Использовать функцию `int()`
- Использовать функцию `bool()`

Шаг 14:

Миссия 14

Мудрая сова объясняет, как преобразовать данные в Python. Сопоставьте типы данных с соответствующими функциями преобразования:

Подсказка: Подумайте, какие функции обычно используют для смены типов.

Ответ:

- Целое число
- Строка
- Число с плавающей точкой
- Логическое значение

1. `bool()`
2. `float()`
3. `str()`
4. `int()`

- **Целое число** — `int()`
- **Строка** — `str()`
- **Число с плавающей точкой** — `float()`
- **Логическое значение** — `bool()`

Шаг 15:

Миссия 15

Воробей сортировщик хочет понять, какие типы можно преобразовывать друг в друга. Сопоставьте пары:

Подсказка: Не все преобразования возможны без ошибок.

Ответ:

- `str` к `int`
- `int` к `str`
- `float` к `int`
- `str "hello"` к `bool`

1. Возможна, если строка содержит число
2. Всегда возможна
3. Возможна, но дробная часть отбрасывается
4. Всегда `True`, кроме пустой строки

- **`str` к `int`** — Возможна, если строка содержит число
- **`int` к `str`** — Всегда возможна
- **`float` к `int`** — Возможна, но дробная часть отбрасывается
- **`str "hello"` к `bool`** — Всегда `True`, кроме пустой строки

Шаг 16:

Миссия 16

Порядок преобразования типов при обработке данных от гномика Листа:

Подсказка: Вспомните, что сначала нужно сделать с данными, чтобы они стали числами, и что делать с ними после.

Расположите шаги в правильном порядке:

1. Получить строку от пользователя
2. Преобразовать строку в число
3. Выполнить вычисления с числом
4. Преобразовать число обратно в строку для вывода

Ответ:

- Получить строку от пользователя
- Преобразовать строку в число
- Выполнить вычисления с числом
- Преобразовать число обратно в строку для вывода

Урок 7 - Тропинка к урожаю — комментарии и читаемый код

Шаг 01:

Миссия 1

Листик Ли заметил, что в коде его друга Саженца растёт много непонятных строк без комментариев. Что лучше всего добавить, чтобы понять, что происходит?

Подсказка: Комментарии помогают объяснить сложные места для других и для себя.

Ответ:

- **Добавить комментарии, поясняющие логику кода**
- Удалить весь код без объяснений
- Писать только на иностранном языке без комментариев
- Сделать код длиннее без пояснений

Шаг 02:

Миссия 2

Ведунья Вера пишет функцию для полива сада, но назвала её "f()". Почему это может вызвать проблемы у команды?

Подсказка: Имена функций должны быть понятными, чтобы быстро понять их назначение.

Ответ:

- **Имя не отражает, что функция делает, это сбивает с толку**
- **Короткое имя** — это всегда хорошо
- Имя функции не влияет на понимание кода
- Лучше использовать буквы, чтобы экономить время

Шаг 03:

Миссия 3

Когда Саженец Антон добавляет комментарии, что лучше всего писать в них?

Подсказка: Комментарии должны помогать понять смысл кода, а не дублировать его.

Ответ:

- **Объяснять, зачем и что делает код, а не повторять сам код**
- Копировать каждую строку кода в комментарии
- Писать только дату и время
- Писать шутки вместо объяснений

Шаг 04:

Миссия 4

Сопоставьте полезные советы по комментированию кода с их описанием:

Ответ:

- Использовать понятные имена
- Писать полезные комментарии
- Не дублировать код в комментариях
- Обновлять комментарии при изменениях

Опции:

1. Комментируйте, чтобы объяснить, а не повторять код.
2. Имя функции должно сразу показывать её назначение.
3. Комментарии должны отражать актуальное состояние кода.
4. Комментарий должен давать дополнительную информацию, полезную для понимания.

Подсказка: Помните: комментарии — это помощники, а имена — подсказки.

- **Использовать понятные имена** — Имя функции должно сразу показывать её назначение.
- **Писать полезные комментарии** — Комментарий должен давать дополнительную информацию, полезную для понимания.
- **Не дублировать код в комментариях** — Комментируйте, чтобы объяснить, а не повторять код.
- **Обновлять комментарии при изменениях** — Комментарии должны отражать актуальное состояние кода.

Шаг 05:

Миссия 5

Сопоставьте типы комментариев с их назначением:

Ответ:

- Строковый комментарий
- Блочный комментарий
- TODO-комментарий
- Документирующий комментарий

Назначения:

1. Поясняет отдельные строки кода
2. Объясняет части кода или большие блоки
3. Отмечает задачи, которые нужно выполнить позже
4. Определяет описание функции или класса

Подсказка: В комментариях есть свои роли — знайте, какую выбрать!

- **Строковый комментарий** — Поясняет отдельные строки кода
- **Блочный комментарий** — Объясняет части кода или большие блоки
- **TODO-комментарий** — Отмечает задачи, которые нужно выполнить позже
- **Документирующий комментарий** — Определяет описание функции или класса

Шаг 06:

Миссия 6

Сопоставьте ошибки при комментировании с их последствиями:

Ответ:

- Комментарии устарели
- Комментарии дублируют код
- Комментарии отсутствуют
- Комментарии слишком длинные и непонятные

Последствия:

1. Путают и вводят в заблуждение разработчиков
2. Затрудняют понимание кода и работу с ним
3. Код сложнее понять другим и себе в будущем
4. Отвлекают и утомляют при чтении

Подсказка: Хороший комментарий должен быть актуальным, полезным и понятным.

- **Комментарии устарели** — Путают и вводят в заблуждение разработчиков
- **Комментарии дублируют код** — Отвлекают и утомляют при чтении
- **Комментарии отсутствуют** — Код сложнее понять другим и себе в будущем
- **Комментарии слишком длинные и непонятные** — Затрудняют понимание кода и работу с ним

Шаг 07:

Миссия 7

Расставьте этапы создания читаемого кода в правильном порядке:

1. Обдумать имена переменных и функций
2. Добавить комментарии там, где нужна дополнительная информация
3. Писать код, следуя логике и структуре задачи
4. Проверить и обновить комментарии после изменений

Подсказка: Хороший код рождается постепенно, шаг за шагом.

Ответ:

- Обдумать имена переменных и функций
- Писать код, следуя логике и структуре задачи
- Добавить комментарии там, где нужна дополнительная информация
- Проверить и обновить комментарии после изменений

Шаг 08:

Миссия 8

Расставьте этапы исправления устаревших комментариев в правильном порядке:

1. Найти комментарии, не соответствующие текущему коду
2. Понять, что именно изменилось в коде
3. Обновить комментарии так, чтобы они соответствовали коду
4. Проверить работу программы и читаемость комментариев

Подсказка: Комментарии должны всегда идти в ногу с кодом.

Ответ:

- Найти комментарии, не соответствующие текущему коду
- Понять, что именно изменилось в коде
- Обновить комментарии так, чтобы они соответствовали коду
- Проверить работу программы и читаемость комментариев

 Шаг 09:

Миссия 9

Устроим порядок в комментариях для команды садоводов! Расположите шаги по созданию документации для функции:

1. Написать описание, что делает функция и какие у неё параметры
2. Проверить, чтобы комментарии были краткими и понятными
3. Добавить примеры использования функции
4. Обновить описание при изменении функции

Подсказка: Хорошая документация — залог успешной работы команды.

Ответ:

- Написать описание, что делает функция и какие у неё параметры
- Добавить примеры использования функции
- Проверить, чтобы комментарии были краткими и понятными
- Обновить описание при изменении функции

 Шаг 10:

Миссия 10

Садовник Дима хочет привести в порядок свой проект. Расположите шаги по улучшению читаемости кода:

1. Проверить имена переменных и функций на понятность
2. Удалить или исправить устаревшие комментарии
3. Добавить полезные комментарии, поясняющие сложные моменты
4. Протестировать программу, чтобы убедиться, что всё работает

Подсказка: Чистый код — это команда, где каждый шаг важен.

Ответ:

- Проверить имена переменных и функций на понятность
- Удалить или исправить устаревшие комментарии
- Добавить полезные комментарии, поясняющие сложные моменты
- Протестировать программу, чтобы убедиться, что всё работает

 Шаг 11:

Миссия 11

Петя-подсолнух нашёл участок кода без комментариев. Как ему помочь команде быстрее понять код?

Подсказка: Комментарии — это как дорожные знаки для других программистов.

Ответ:

- **Добавить краткие и понятные комментарии к сложным местам**
- Игнорировать и надеяться, что другие разберутся

- Переписать код без комментариев
- Сделать код максимально длинным без объяснений

Шаг 12:

Миссия 12

Какую роль выполняют комментарии в коде команды Grow a Garden?

Подсказка: Комментарии помогают и объяснять, и сохранять знание внутри команды.

Ответ:

- **Объяснять логику и помогать быстро понять код**
- Удлинять код без смысла
- Создавать запутанность и путаницу
- Заменять хороший код

Шаг 13:

Миссия 12

В саду Grow a Garden начинающий садовод Макс написал код, но забыл добавить комментарии. Почему важно писать комментарии в коде?

Подсказка: Представь, что ты объясняешь другу, что делает твой код.

Ответ:

- **Чтобы другие (и ты сам позже) понимали, что делает код**
- Чтобы код работал быстрее
- Чтобы компьютер выполнял код без ошибок
- Чтобы уменьшить размер файла

Шаг 14:

Миссия 13

Лея, персонаж Grow a Garden, хочет улучшить свой код, но не знает, как писать комментарии. Как правильно писать комментарии в Python?

Подсказка: Вспомни, какой символ используют для комментариев в Python.

Ответ:

- **Начинать строку с символа #**
- Писать комментарии между двойными кавычками
- Использовать символ //
- Писать комментарии в фигурных скобках

Шаг 15:

Миссия 14

В саду Grow a Garden Тим заметил, что в его коде нет пробелов и отступов, и его трудно читать. Какую роль играют отступы и пробелы в Python?

Подсказка: Вспомни, что Python использует отступы для определения блоков кода.

Ответ:

- **Отступы помогают организовать код и избежать ошибок**
- Отступы делают код длиннее
- Отступы необязательны и не влияют на работу программы
- Отступы нужны для украшения кода

Шаг 16:

Миссия 15

Алиса нашла функцию в коде Grow a Garden, где названия переменных непонятные и короткие (x, y, z). Что лучше сделать с такими переменными?

Подсказка: Подумай, как сделать код понятнее для других игроков.

Ответ:

- **Переименовать переменные в более осмысленные имена**
- **Оставить как есть** — важна скорость написания кода
- Удалить переменные и писать всё без них
- Использовать только английские сокращения

Урок 8 - Росточек! — мини-проект ввод, обработка и вывод роста растения

Шаг 01:

Миссия 1

Аня посадила в своем саду новый росток и хочет записать его текущий рост. Какой тип данных лучше всего использовать для хранения роста растения в Python?

Подсказка: Подумай, будет ли рост всегда целым числом, или может быть дробным.

Ответ:

- **float** — для хранения чисел с десятичной частью
- **string** — для текста
- **boolean** — для истинных или ложных значений
- **list** — для списка значений

Шаг 02:

Миссия 2

Максим хочет запросить у пользователя рост росточка. Какая функция в Python поможет ему получить данные от пользователя?

Подсказка: Введенные данные всегда приходят в виде текста.

Ответ:

- **input()**
- print()
- len()
- type()

Шаг 03:

Миссия 3

Игорь получил рост росточка как строку через input(). Как он может преобразовать это значение в число с плавающей точкой?

Подсказка: Используйте функцию, которая превращает строку в число с десятичной частью.

Ответ:

- **float()**
- int()
- str()

🔮 Шаг 04:

Миссия 4

Катя хочет вывести на экран сообщение: "Рост растения: 12.5 см". Как правильно использовать `print()` для этого, если рост хранится в переменной `height`?

Подсказка: Можно объединять текст и переменную, используя запятую внутри `print`.

Ответ:

- `print("Рост растения:", height, "см")`
- `print("Рост растения:" + height + "см")`
- `print(height)`

🔮 Шаг 05:

Миссия 5

Росточек из игры `Grow a Garden` развивается по дням. Сопоставьте названия переменных с их ролью:

Ответ:

- `height`
- `day`
- `growth_rate`
- `message`

Роли:

1. Текущий день роста
2. Текст с сообщением о росте
3. Рост растения в сантиметрах
4. Скорость роста в сантиметрах в день

Подсказка: Переменные обычно называют так, чтобы сразу было понятно, что они хранят.

- **`height`** — Рост растения в сантиметрах
- **`day`** — Текущий день роста
- **`growth_rate`** — Скорость роста в сантиметрах в день
- **`message`** — Текст с сообщением о росте

🔮 Шаг 06:

Миссия 6

В игре `Росточек` появляется последовательность действий для вычисления нового роста. Сопоставьте шаг с правильным действием:

Ответ:

- Получить ввод от пользователя
- Преобразовать ввод в число
- Вычислить новый рост с учётом скорости роста
- Вывести сообщение с результатом

Действия:

1. Использовать `input()`
2. Преобразовать в `float()`
3. Выполнить операцию `height + growth_rate`

4. Вызвать `print()`

Подсказка: Логика программы должна идти от запроса данных к их обработке, а затем выводу.

- **Получить ввод от пользователя** — Использовать `input()`
- **Преобразовать ввод в число** — Преобразовать в `float()`
- **Вычислить новый рост с учётом скорости роста** — Выполнить операцию `height + growth_rate`
- **Вывести сообщение с результатом** — Вызвать `print()`

Шаг 07:

Миссия 7

Ребята из Grow a Garden решили использовать комментарии в коде. Сопоставьте комментарий с его назначением:

Ответ:

- # Запросить текущий рост
- # Преобразуем строку в число с плавающей точкой
- # Вычисляем новый рост растения
- # Выводим сообщение пользователю

Назначения:

1. Объяснить, что делает конкретный участок кода
2. Отметить важный шаг для других разработчиков
3. Помочь в понимании структуры программы
4. Улучшить читаемость кода

Подсказка: Комментарии — это подсказки внутри кода для людей.

- **# Запросить текущий рост** — Объяснить, что делает конкретный участок кода
- **# Преобразуем строку в число с плавающей точкой** — Отметить важный шаг для других разработчиков
- **# Вычисляем новый рост растения** — Помочь в понимании структуры программы
- **# Выводим сообщение пользователю** — Улучшить читаемость кода

Шаг 08:

Миссия 8

Сортируйте этапы программы "Росточек" в правильном порядке:

1. Преобразование введённого роста в число
2. Вывод результата роста
3. Запрос роста у пользователя
4. Вычисление нового роста с учётом скорости

Подсказка: Сначала нужно получить данные, потом обработать их, затем показать результат.

Ответ:

- Запрос роста у пользователя
- Преобразование введённого роста в число
- Вычисление нового роста с учётом скорости
- Вывод результата роста

Шаг 09:

Миссия 9

Упорядочите шаги добавления комментариев в код программы:

1. Проверить, какие части кода могут быть непонятны другим
2. Написать короткий и понятный комментарий
3. Добавить комментарий над или рядом с кодом
4. Перепроверить код на читаемость с комментариями

Подсказка: Комментарии должны помогать понимать код, но не загромождать его.

Ответ:

- Проверить, какие части кода могут быть непонятны другим
- Написать короткий и понятный комментарий
- Добавить комментарий над или рядом с кодом
- Перепроверить код на читаемость с комментариями

Шаг 10:

Миссия 10

Упорядочьте действия по созданию мини-программы "Росточек" для вычисления и вывода роста растения:

1. Запросить рост у пользователя
2. Преобразовать введенное значение в float
3. Вычислить новый рост, прибавив скорость роста
4. Создать сообщение для пользователя
5. Вывести сообщение на экран

Подсказка: Логика программы — это цепочка последовательных действий от ввода к выводу.

Ответ:

- Запросить рост у пользователя
- Преобразовать введенное значение в float
- Вычислить новый рост, прибавив скорость роста
- Создать сообщение для пользователя
- Вывести сообщение на экран

Шаг 11:

Миссия 11

Росточек вырос и теперь высота может быть отрицательной из-за ошибки. Как лучше всего защитить программу от ввода отрицательного значения?

Подсказка: Подумайте, что нужно проверить сразу после ввода данных.

Ответ:

- **Добавить проверку, что $\text{рост} \geq 0$, и попросить повторить ввод, если нет**
- Просто принимать любое число без проверки
- Автоматически делать рост положительным (abs), не предупреждая пользователя

Шаг 12:

Миссия 12

Чтобы сделать вывод программы красивым и понятным, какой способ форматирования строки использовать?

Подсказка: Форматирование с f"..." позволяет вставлять переменные прямо внутрь строки.

Ответ:

- **Использовать f-строки (f"Рост: {height} см")**
- Соединять строки с помощью + без преобразования типов
- Выводить переменные отдельно без текста
- Использовать print без аргументов

Шаг 13:

Миссия 13

Аня только что посадила семечко в свой виртуальный сад. Она хочет узнать, как правильно принять ввод от пользователя для измерения роста растения. Какой командой в Python можно получить ввод пользователя?

Подсказка: Вспомните базовую функцию для получения текста от пользователя.

Ответ:

- **input()**
- print()
- int()
- len()

Шаг 14:

Миссия 14

Росточек Ваня измерил высоту своего растения и получил число 12.7. Какой тип данных лучше всего использовать для хранения роста?

Подсказка: Подумайте, можно ли рост быть дробным числом?

Ответ:

- **float**
- int
- str

Шаг 15:

Миссия 15

Аня хочет вывести на экран сообщение: "Рост растения: 15 см". Какой из вариантов кода выведет это правильно?

Подсказка: Помните, как соединять строки и числа в Python.

Ответ:

- **print("Рост растения:", 15, "см")**
- print("Рост растения:" + 15 + "см")
- print("Рост растения: 15см")
- print(Рост растения: 15 см)

Шаг 16:

Миссия 16

Росточек Коля учится вычислять рост растения. Сопоставьте команды Python с их назначением в проекте:

Ответ:

- `input()`
- `float()`
- `print()`
- `int()`

Сопоставьте их с функциями:

1. Вывод на экран результатов измерения
2. Преобразование текста в число с плавающей точкой
3. Преобразование текста в целое число
4. Получение данных от пользователя

Подсказка: Подумайте, какие действия выполняет каждая функция.

- **`input()`** — Получение данных от пользователя
- **`float()`** — Преобразование текста в число с плавающей точкой
- **`print()`** — Вывод на экран результатов измерения
- **`int()`** — Преобразование текста в целое число

Модуль 2 - **Модуль 2 «Первые всход**

Урок 1 - Небо хмурится — условные конструкции if и else

Шаг 01:

Миссия 1

Задача: Помогите Джеку сопоставить ситуацию с правильным действием в коде.

Ситуации:

Ответ:

- Небо хмурится
- Солнце светит
- Идёт дождь
- Пасмурно, но дождя нет

Действия:

1. Полить цветы
2. Укрыть рассаду
3. Отдохнуть и не работать в саду
4. Проверить прогноз погоды

Подсказка: Помните, что действия должны подходить под условия, не перепутайте их.

- **Небо хмурится** — Полить цветы
- **Солнце светит** — Отдохнуть и не работать в саду
- **Идёт дождь** — Укрыть рассаду
- **Пасмурно, но дождя нет** — Проверить прогноз погоды

Шаг 02:

Миссия 2

Задача: Сопоставьте части условного оператора с их назначением в Python.

Ответ:

- if
- else
- elif

1. Обработывает альтернативный случай, если if не сработал
2. Проверяет дополнительное условие, если if не выполнено
3. Начинает проверку условия

Подсказка: Вспомните, что if — это основное условие, а else — запасной вариант.

- **if** — Начинает проверку условия
- **else** — Обработывает альтернативный случай, если if не сработал
- **elif** — Проверяет дополнительное условие, если if не выполнено

Шаг 03:

Миссия 3

Задача: Сопоставьте действия героя с результатом условия в игре.

Ответ:

- Проверить, идет ли дождь
- Рассчитать, нужна ли подкормка
- Определить, стоит ли поливать рассаду

1. Если да, то укрыть растения
2. Если нет, добавить удобрения
3. Если да, полить растения

Подсказка: Условия в игре помогают герою принимать правильные решения для сада.

- **Проверить, идет ли дождь** — Если да, то укрыть растения
- **Рассчитать, нужна ли подкормка** — Если нет, добавить удобрения
- **Определить, стоит ли поливать рассаду** — Если да, полить растения

Шаг 04:

Миссия 4

Вопрос: Расположите шаги, которые делает садовник, чтобы проверить погоду и решить, поливать ли цветы:

1. Посмотреть на небо
2. **Если небо хмурится** — приготовить лейку
3. **Если не хмурится** — отложить полив
4. Полить цветы, если подготовился

Подсказка: Начните с наблюдения за погодой и принимайте решение шаг за шагом.

Ответ:

- Посмотреть на небо
- **Если небо хмурится** — приготовить лейку

- **Если не хмурится** — отложить полив
- Полить цветы, если подготовился

Шаг 05:

Миссия 5

Вопрос: Упорядочите этапы принятия решения в программе для укрытия растений:

1. Проверить, идёт ли дождь
2. **Если дождь** — закрыть растения
3. **Если дождя нет** — продолжить работу в саду
4. Отслеживать изменения погоды

Подсказка: Решение строится на последовательной проверке условий.

Ответ:

- Проверить, идёт ли дождь
- **Если дождь** — закрыть растения
- **Если дождя нет** — продолжить работу в саду
- Отслеживать изменения погоды

Шаг 06:

Миссия 6

Вопрос: Расположите этапы работы программы, которая решает, когда подкормить растения:

1. Проверить состояние почвы
2. **Если почва сухая** — добавить подкормку
3. **Если почва влажная** — пропустить подкормку
4. Записать результат в журнал

Подсказка: Логика программы — проверить условие и выполнить соответствующее действие.

Ответ:

- Проверить состояние почвы
- **Если почва сухая** — добавить подкормку
- **Если почва влажная** — пропустить подкормку
- Записать результат в журнал

Шаг 07:

Миссия 7

Вопрос: Отсортируйте этапы алгоритма, который помогает садовнику определить, нужна ли поливка и укрытие:

1. Посмотреть на небо
2. **Если идёт дождь** — укрыть растения
3. Если не идёт дождь, проверить влажность почвы
4. **Если почва сухая** — полить растения
5. **Если почва влажная** — не поливать

Подсказка: Последовательность действий зависит от проверки нескольких условий.

Ответ:

- Посмотреть на небо

- **Если идёт дождь** — укрыть растения
- Если не идёт дождь, проверить влажность почвы
- **Если почва сухая** — полить растения
- **Если почва влажная** — не поливать

🔗 Шаг 08:

Миссия 8

В саду героя Петра сегодня тучи сгущаются. Он хочет узнать, взять ли с собой зонтик. Какой код поможет Петру проверить, нужно ли взять зонтик, если дождь идёт?

Подсказка: Используйте if, чтобы проверить условие дождя.

Ответ:

- **if дождь == True: взять_зонтик()**
- while дождь == True: взять_зонтик()
- if дождь != True: взять_зонтик()

🔗 Шаг 09:

Миссия 9

Мария наблюдает за погодой в своём саду и хочет решить: если светит солнце, она поливает растения меньше, а если пасмурно — больше. Какой условный оператор поможет ей это сделать?

Подсказка: Используйте if-else для двух вариантов действий.

Ответ:

- **if солнце == True: полить(меньше) else: полить(больше)**
- if солнце == False: полить(меньше)
- if солнце == True: полить(больше)
- while солнце: полить(меньше)

🔗 Шаг 10:

Миссия 10

В саду Кати температура воздуха важна: если температура ниже 10, ей нужно укрыть рассаду, иначе — поливать как обычно. Какой вариант кода правильный?

Подсказка: Сравните температуру с числом.

Ответ:

- **if температура < 10: укрыть_рассаду() else: полить_обычно()**
- if температура > 10: укрыть_рассаду()
- if температура == 10: полить_обычно()
- while температура < 10: укрыть_рассаду()

🔗 Шаг 11:

Миссия 11

Дружная команда героев Grow a Garden хочет проверить, если ли у них семена подсолнуха. Если есть, посадить их, иначе посадить морковь. Что нужно написать в условии?

Подсказка: Проверяйте наличие семян с помощью if-else.

Ответ:

- **if подсолнухи_в_наличии: посадить('подсолнух') else: посадить('морковь')**

- `if морковь_в_наличии: посадить('подсолнух')`
- `while подсолнухи_в_наличии: посадить('морковь')`

🧙 Шаг 12:

Миссия 12

Сопоставьте термины из мира Python с их описаниями, связанными с условными операторами.

Термины:

Ответ:

- `if`
- `else`
- `elif`
- условие

Описание:

1. Блок кода, выполняющийся, если условие истинно
2. Альтернативный блок, выполняющийся при ложном условии
3. Дополнительная проверка в цепочке условий
4. Выражение, которое проверяется как `True` или `False`

Подсказка: Вспомните, как работает проверка условий в Python.

- **`if`** — Блок кода, выполняющийся, если условие истинно
- **`else`** — Альтернативный блок, выполняющийся при ложном условии
- **`elif`** — Дополнительная проверка в цепочке условий
- **условие** — Выражение, которое проверяется как `True` или `False`

🧙 Шаг 13:

Миссия 13

Герой Алекс проверяет погоду для выращивания рассады. Сопоставьте условие с соответствующим действием.

Условия:

Ответ:

- дождь идет
- солнце светит
- температура ниже 15
- температура выше 25

Действия:

1. укрыть рассаду
2. полить меньше
3. не брать зонтик
4. проветрить теплицу

Подсказка: Подумайте, как погодные условия влияют на действия садовода.

- **дождь идет** — не брать зонтик
- **солнце светит** — полить меньше
- **температура ниже 15** — укрыть рассаду
- **температура выше 25** — проветрить теплицу

Шаг 14:

Миссия 14

Сопоставьте этапы проверки погоды перед посадкой семян.

Этапы:

Ответ:

- Проверить, идёт ли дождь
- Определить температуру
- Решить, брать ли зонтик
- Выбрать растение для посадки

Описание действий:

1. Использовать if для условий дождя
2. Сравнить температуру с порогом
3. **Если дождь, взять зонтик, иначе** — нет
4. Подобрать растение, подходящее к погоде

Подсказка: Продумайте порядок логики в садоводстве.

- **Проверить, идёт ли дождь** — Использовать if для условий дождя
- **Определить температуру** — Сравнить температуру с порогом
- **Решить, брать ли зонтик** — Если дождь, взять зонтик, иначе — нет
- **Выбрать растение для посадки** — Подобрать растение, подходящее к погоде

Шаг 15:

Миссия 15

Расположите действия героя Олега при проверке погодных условий в правильном порядке:

1. Посмотреть прогноз дождя
2. Решить, брать ли зонтик
3. Проверить температуру
4. Принять решение о посадке растений

Подсказка: Подумайте, что важнее проверить сначала — дождь или температуру.

Ответ:

- Посмотреть прогноз дождя
- Проверить температуру
- Решить, брать ли зонтик
- Принять решение о посадке растений

Шаг 16:

Миссия 16

Упорядочьте шаги обработки погодных условий в коде для посадки растений:

1. Если идёт дождь, не поливать
2. Если нет дождя, проверить температуру
3. Если температура ниже 10, укрыть рассаду
4. Иначе полить растения

Подсказка: Вспомните логику вложенных условий if-else.

Ответ:

- Если идёт дождь, не поливать
- Если нет дождя, проверить температуру
- Если температура ниже 10, укрыть рассаду
- Иначе полить растения

Урок 2 - Проверка погоды — логические операторы (and, or, not)

Шаг 01:

Миссия 1

Сопоставьте логические операторы с их описанием.

Ответ:

- and
 - or
 - not
1. Возвращает True, если оба условия истинны.
 2. Возвращает True, если хотя бы одно условие истинно.
 3. Меняет значение условия на противоположное.

Подсказка: Вспомните правила работы операторов в логике.

- **and** — Возвращает True, если оба условия истинны.
- **or** — Возвращает True, если хотя бы одно условие истинно.
- **not** — Меняет значение условия на противоположное.

Шаг 02:

Миссия 2

Сопоставьте условие и описание из сада Grow a Garden:

Ответ:

- not дождь
 - тепло and солнце
 - дождь or холодно
1. Погода позволяет сажать, если тепло и светит солнце.
 2. **Нет дождя** — можно полить растения.
 3. Плохая погода, либо дождь, либо холодно.

Подсказка: Подумайте, когда условия дают True.

- **not дождь** — Нет дождя — можно полить растения.
- **тепло and солнце** — Погода позволяет сажать, если тепло и светит солнце.
- **дождь or холодно** — Плохая погода, либо дождь, либо холодно.

Шаг 03:

Миссия 3

Соотнесите описание с правильным логическим оператором:

Ответ:

- Объединяет условия, когда нужно оба выполнить

- Выбирает одно из условий, достаточно, чтобы хоть одно было истинным
- Отрицание условия

1. and
2. or
3. not

Подсказка: Ключевые слова - "оба", "хотя бы одно", "противоположное".

- **Объединяет условия, когда нужно оба выполнить** — and
- **Выбирает одно из условий, достаточно, чтобы хоть одно было истинным** — or
- **Отрицание условия** — not

🔮 Шаг 04:

Миссия 4

Сопоставьте тип погоды и правильное логическое выражение для условий сада:

Ответ:

- Идет дождь и холодно
- Солнечно или тепло
- Не идет дождь

1. дождь and холодно
2. солнечно or тепло
3. not дождь

Подсказка: Вспомните, какие операторы соответствуют "и", "или", "не".

- **Идет дождь и холодно** — дождь and холодно
- **Солнечно или тепло** — солнечно or тепло
- **Не идет дождь** — not дождь

🔮 Шаг 05:

Миссия 5

Сопоставьте результат выражения с описанием из жизни героев:

Ответ:

- True, если тепло и солнце
- False, если идет дождь
- True, если не холодно

1. тепло and солнце
2. дождь
3. not холодно

Подсказка: Как логические значения влияют на выполнение условий?

- **True, если тепло и солнце** — тепло and солнце
- **False, если идет дождь** — дождь
- **True, если не холодно** — not холодно

🔮 Шаг 06:

Миссия 6

Расположите этапы проверки погоды для полива сада:

1. Проверить, что не идет дождь и не холодно
2. Принять решение о поливе
3. Собрать данные о погоде

Подсказка: Что делается сначала, а что — после?

Ответ:

- Собрать данные о погоде
- Проверить, что не идет дождь и не холодно
- Принять решение о поливе

Шаг 07:

Миссия 7

Расставьте шаги проверки погоды перед посадкой саженцев:

1. Проверить, что тепло или солнечно
2. Если условия подходят, начать посадку
3. Узнать текущую температуру и осадки

Подсказка: Логично сначала узнать данные, потом проверить, затем действовать.

Ответ:

- Узнать текущую температуру и осадки
- Проверить, что тепло или солнечно
- Если условия подходят, начать посадку

Шаг 08:

Миссия 8

Упорядочите действия героя, который решает, когда открывать теплицу:

1. Проверить, что не холодно и нет дождя
2. Открыть теплицу, если условия подходящие
3. Получить данные о погоде

Подсказка: Сначала получаем данные, потом проверяем, потом действуем.

Ответ:

- Получить данные о погоде
- Проверить, что не холодно и нет дождя
- Открыть теплицу, если условия подходящие

Шаг 09:

Миссия 9

Ботаник Борис заметил, что растение растёт хорошо, если на улице тепло или если есть искусственный свет.

Как правильно написать условие, используя логический оператор "or"?

Подсказка: "or" означает, что достаточно, чтобы было верно хотя бы одно условие.

Ответ:

- `warm or artificial_light`

- warm and artificial_light
- not warm and not artificial_light

🔮 Шаг 10:

Миссия 10

Маша проверяет погоду перед посадкой. Она не хочет сажать семена, если сегодня дождь или слишком холодно.

Какое условие выражает запрет на посадку?

Подсказка: Используйте "or" для объединения нежелательных условий.

Ответ:

- **raining or cold**
- not raining and not cold
- raining and cold

🔮 Шаг 11:

Миссия 11

Даниил решил проверить, стоит ли включать отопление в теплице. Он включает его, если на улице холодно и при этом нет солнечного света.

Какое логическое выражение соответствует этому правилу?

Подсказка: "and" объединяет два условия, которые должны быть истинны одновременно.

Ответ:

- **cold and not sunny**
- cold or sunny
- not cold and sunny

🔮 Шаг 12:

Миссия 12

Кирилл изучает условия для полива растений и сопоставляет логические выражения с их значениями.

Сопоставьте условие с его объяснением:

Ответ:

- not raining and warm
- cold or raining
- not (cold or raining)
- warm and sunny

Объяснения:

1. Полив возможен, если не холодно и не идёт дождь
2. **Погода плохая** — холодно или дождь
3. **Погода хорошая** — не холодно и нет дождя
4. Тёплый и солнечный день

Подсказка: Обратите внимание на отрицания и соединения условий.

- **not raining and warm** — Полив возможен, если не холодно и не идёт дождь
- **cold or raining** — Погода плохая — холодно или дождь
- **not (cold or raining)** — Погода хорошая — не холодно и нет дождя

- **warm and sunny** — Тёплый и солнечный день

Шаг 13:

Миссия 13

Вера хочет понять, какое из условий описывает ситуацию, когда нельзя поливать растения.

Сопоставьте условия и их смыслы:

Ответ:

- raining or cold
- not raining and warm
- not (raining and warm)
- cold and sunny

Значения:

1. **Нельзя поливать** — дождь или холод
2. **Можно поливать** — тепло и нет дождя
3. Полив возможен, если не одновременно дождь и тепло
4. Холодный, но солнечный день

Подсказка: Обратите внимание, где используется отрицание и как меняются условия.

- **raining or cold** — Нельзя поливать — дождь или холод
- **not raining and warm** — Можно поливать — тепло и нет дождя
- **not (raining and warm)** — Полив возможен, если не одновременно дождь и тепло
- **cold and sunny** — Холодный, но солнечный день

Шаг 14:

Миссия 14

Команда Grow a Garden сравнивает разные условия для автоматического включения полива.

Сопоставьте условия с результатом включения:

Ответ:

- not raining and warm
- raining or cold
- not (cold or raining)
- warm and not sunny

Результаты:

1. Полив включается
2. Полив выключается
3. Погода идеальна для сада
4. Тёплый, но пасмурный день

Подсказка: Подумайте, когда условия благоприятны для полива.

- **not raining and warm** — Полив включается
- **raining or cold** — Полив выключается
- **not (cold or raining)** — Погода идеальна для сада
- **warm and not sunny** — Тёплый, но пасмурный день

Шаг 15:

Миссия 15

Лена описывает процесс проверки условий для полива:

1. Проверить, идёт ли дождь
2. Проверить, холодно ли на улице
3. **Если не дождь и не холодно** — включить полив
4. **Иначе** — не включать полив

Подсказка: Логическая проверка начинается с простых условий, потом объединяется результат.

Отсортируйте шаги в правильном порядке:

Ответ:

- Проверить, идёт ли дождь
- Проверить, холодно ли на улице
- **Если не дождь и не холодно** — включить полив
- **Иначе** — не включать полив

Шаг 16:

Миссия 16

Максим описывает логику включения отопления в теплице:

1. Проверить, холодно ли на улице
2. Проверить, есть ли солнечный свет
3. **Если холодно и нет солнца** — включить отопление
4. **Иначе** — не включать отопление

Подсказка: Логика должна учитывать оба условия одновременно.

Отсортируйте шаги в правильном порядке:

Ответ:

- Проверить, холодно ли на улице
- Проверить, есть ли солнечный свет
- **Если холодно и нет солнца** — включить отопление
- **Иначе** — не включать отопление

Урок 3 - Нужно ли поливать — вложенные условия

Шаг 01:

Миссия 1

Алиса смотрит на свою розу в саду. Сегодня солнечно и земля влажная. Нужно ли ей поливать розу?

Подсказка: Полив нужен, если земля сухая и нет дождя.

Ответ:

- Да, нужно поливать
- **Нет, не нужно поливать**
- Зависит от времени суток

Шаг 02:

Миссия 2

Макс заметил, что у его тюльпана сухая земля, но сегодня идет дождь. Поливать ли тюльпан?

Подсказка: Если идет дождь, дополнительный полив не нужен.

Ответ:

- Да, обязательно поливать
- **Нет, дождь поливает за нас**
- Поливать только вечером

Шаг 03:

Миссия 3

Света проверяет свой огород. Земля сухая, и сегодня пасмурно, дождя не ожидается. Что делать?

Подсказка: Сухая земля без дождя — сигнал для полива.

Ответ:

- **Поливать растения**
- Не поливать
- Ждать дождя

Шаг 04:

Миссия 4

Помоги Лизе правильно проверить, нужно ли поливать её комнатные растения. Расположи шаги в правильном порядке:

Подсказка: Логика вложенных условий — проверять сначала самые важные факторы.

1. Проверить, идет ли дождь
2. Посмотреть, сухая ли земля
3. Решить, поливать или нет
4. Если дождя нет, проверить влажность

Ответ:

- Проверить, идет ли дождь
- Если дождя нет, проверить влажность
- Посмотреть, сухая ли земля
- Решить, поливать или нет

Шаг 05:

Миссия 5

Расположи этапы вложенного условия для решения, нужно ли поливать:

Подсказка: Сначала проверяем условия, которые отменяют необходимость полива.

1. Проверить, идет ли дождь
2. Если дождя нет, проверить влажность почвы
3. Если земля сухая, проверить возраст растения
4. Если растение молодое и земля сухая, полить
5. Иначе не поливать

Ответ:

- Проверить, идет ли дождь
- Если дождя нет, проверить влажность почвы
- Если земля сухая, проверить возраст растения
- Если растение молодое и земля сухая, полить
- Иначе не поливать

 Шаг 06:

Миссия 6

Команда Grow a Garden разрабатывает алгоритм полива. Помогите им правильно упорядочить шаги проверки:

Подсказка: Вложенные условия строятся от общего к частному.

1. Проверить, есть ли прогноз дождя
2. Если дождя нет, проверить уровень влаги почвы
3. Если уровень влаги низкий, проверить время последнего полива
4. Если полив был давно, полить растение
5. Иначе не поливать

Ответ:

- Проверить, есть ли прогноз дождя
- Если дождя нет, проверить уровень влаги почвы
- Если уровень влаги низкий, проверить время последнего полива
- Если полив был давно, полить растение
- Иначе не поливать

 Шаг 07:

Миссия 7

Даша смотрит на свои ростки и думает: «Если земля сухая, нужно полить». Какое условие в Python отражает эту логику?

Подсказка: Проверяем, что земля действительно сухая, прежде чем поливать.

Ответ:

- **if soil == 'dry': water()**
- if water(): soil = 'dry'
- if soil != 'dry': water()
- if soil == 'wet': water()

 Шаг 08:

Миссия 8

Макс заметил, что если на улице дождь, то поливать не нужно, даже если земля сухая. Какое условие правильно описывает эту ситуацию?

Подсказка: Дождь отменяет необходимость полива.

Ответ:

- **if soil == 'dry' and not raining: water()**
- if soil == 'dry' or raining: water()
- if soil == 'wet' and raining: water()
- if not soil == 'dry' and raining: water()

Шаг 09:

Миссия 9

Лео хочет полить сад только если земля сухая, дождь не идет и температура выше 10 градусов. Какой код подойдет?

Подсказка: Все три условия должны выполняться одновременно.

Ответ:

- **if soil == 'dry' and not raining and temperature > 10: water()**
- if soil == 'dry' or not raining or temperature > 10: water()
- if soil != 'dry' and raining and temperature < 10: water()
- if soil == 'wet' and raining and temperature > 10: water()

Шаг 10:

Миссия 10

Сопоставьте ситуации с их правильными условиями на Python:

Ответ:

- Поливать, если земля сухая и нет дождя
- Не поливать, если идет дождь
- Поливать, если температура выше 15 и земля сухая
- Не поливать, если земля влажная

Подсказка: Вложенные условия помогут учесть сразу несколько факторов.

- **Поливать, если земля сухая и нет дождя** — if soil == 'dry' and not raining: water()
- **Не поливать, если идет дождь** — if raining: pass
- **Поливать, если температура выше 15 и земля сухая** — if temperature > 15 and soil == 'dry': water()
- **Не поливать, если земля влажная** — if soil == 'wet': pass

Шаг 11:

Миссия 11

Герои Grow a Garden обсуждают, когда нужно поливать. Сопоставьте условия с правильным объяснением:

Ответ:

- soil == 'dry' and temperature > 5
- not raining and soil == 'dry'
- soil == 'wet' or raining
- temperature < 5

Подсказка: Логика «и», «или» и отрицание помогут выбрать правильное действие.

- **soil == 'dry' and temperature > 5** — Поливать, если сухо и тепло
- **not raining and soil == 'dry'** — Поливать, если нет дождя и земля сухая
- **soil == 'wet' or raining** — Не поливать, если земля влажная или идет дождь
- **temperature < 5** — Не поливать, если холодно

Шаг 12:

Миссия 12

Кира хочет написать условие, что если земля сухая и температура в пределах от 10 до 25 градусов, нужно полить. Какое условие правильное?

Подсказка: Используйте сравнения с помощью and и объединение условий.

Ответ:

- **if soil == 'dry' and 10 <= temperature <= 25: water()**
- if soil == 'dry' and temperature > 10 or temperature < 25: water()
- if soil == 'dry' or 10 <= temperature <= 25: water()
- if soil != 'dry' and 10 <= temperature <= 25: water()

Шаг 13:

Миссия 13

Расположите действия по проверке необходимости полива в правильном порядке:

1. Проверить, идет ли дождь
2. Если дождя нет, проверить влажность почвы
3. Если почва сухая, проверить температуру
4. Если температура подходящая, полить

Подсказка: Логика вложенных условий строится сверху вниз.

Ответ:

- Проверить, идет ли дождь
- Если дождя нет, проверить влажность почвы
- Если почва сухая, проверить температуру
- Если температура подходящая, полить

Шаг 14:

Миссия 14

Расположите строки кода по вложенной проверке необходимости полива:

1. if temperature > 10:
2. if soil == 'dry':
3. if not raining:
4. water()

Подсказка: Помните, что вложенность означает, что одна проверка находится внутри другой.

Ответ:

- if not raining:
- if soil == 'dry':
- if temperature > 10:
- water()

Шаг 15:

Миссия 15

Расположите правильно вложенные условия, которые учитывают все факторы: дождь, влажность и температура для полива:

1. if temperature > 15:
2. if not raining:
3. if soil == 'dry':

4. water()

Подсказка: Вложенность отражает порядок проверок в коде.

Ответ:

- if temperature > 15:
- if not raining:
- if soil == 'dry':
- water()

Шаг 16:

Миссия 16

Расположите этапы принятия решения о поливе с использованием вложенных условий:

1. Проверить температуру
2. Проверить дождь
3. Проверить влажность почвы
4. Полить растения, если все условия выполнены

Подсказка: Сначала проверяются внешние условия, потом внутренние.

Ответ:

- Проверить температуру
- Проверить дождь
- Проверить влажность почвы
- Полить растения, если все условия выполнены

Урок 4 - Температурный режим — сравнение значений (==, , , !=)

Шаг 01:

Миссия 1

Сопоставьте условие Python с его значением для проверки температуры:

Ответ:

- temperature == 20
- temperature != 0
- temperature > 25
- temperature < 10

Подсказка: Подумайте, что проверяет каждый оператор сравнения.

- **temperature == 20** — Температура равна 20°C
- **temperature != 0** — Температура не равна 0°C
- **temperature > 25** — Температура больше 25°C
- **temperature < 10** — Температура меньше 10°C

Шаг 02:

Миссия 2

Сопоставьте ситуацию из игры Grow a Garden с правильным условием Python:

Ответ:

- Проверка, что температура не опасна (не равна 0)

- Проверка, что температура идеальна (ровно 22)
- Проверка, что температура слишком высокая (больше 30)
- Проверка, что температура прохладная (меньше 15)

Подсказка: Вспомните, как работают операторы сравнения.

- **Проверка, что температура не опасна (не равна 0)** — `temperature != 0`
- **Проверка, что температура идеальна (ровно 22)** — `temperature == 22`
- **Проверка, что температура слишком высокая (больше 30)** — `temperature > 30`
- **Проверка, что температура прохладная (меньше 15)** — `temperature < 15`

🔮 Шаг 03:

Миссия 3

Сопоставьте оператор сравнения с его описанием:

Ответ:

- `==`
- `!=`
- `>`
- `<`

Подсказка: Обратите внимание на символы и их смысл в Python.

- `==` — Равенство
- `!=` — Неравенство
- `>` — Больше
- `<` — Меньше

🔮 Шаг 04:

Миссия 4

Задача: Расположите шаги проверки температуры для посадки растений в правильном порядке:

1. Проверить, что температура выше 15°C
2. Убедиться, что температура не равна 0°C
3. Проверить, что температура не слишком высокая (меньше 30°C)
4. **Если все условия выполнены** — начать посадку

Подсказка: Сначала исключите опасные условия, потом проверьте нормальные и начните действие.

Ответ:

- Проверить, что температура не равна 0°C
- Проверить, что температура выше 15°C
- Проверить, что температура не слишком высокая (меньше 30°C)
- **Если все условия выполнены** — начать посадку

🔮 Шаг 05:

Миссия 5

Задача: Расположите проверки температурного режима в порядке приоритетности для сохранения здоровья растений:

1. Проверить, что температура не равна 0°C

2. Убедиться, что температура не ниже 10°C
3. Проверить, что температура не выше 35°C
4. Проверить, что температура равна оптимальной (22°C)

Подсказка: Сначала исключаем опасные значения, потом проверяем комфортные.

Ответ:

- Проверить, что температура не равна 0°C
- Убедиться, что температура не ниже 10°C
- Проверить, что температура не выше 35°C
- Проверить, что температура равна оптимальной (22°C)

Шаг 06:

Миссия 6

Задача: Расположите правильную логику условных операторов для проверки температурного режима в порядке выполнения:

1. Проверить, что температура не равна 0°C
2. Если температура равна 22°C, вывести "Идеальная температура"
3. Проверить, что температура больше 15°C
4. Если температура меньше 15°C, вывести "Слишком холодно"

Подсказка: Подумайте о порядке приоритетов и корректных условиях в коде.

Ответ:

- Проверить, что температура не равна 0°C
- Проверить, что температура больше 15°C
- Если температура равна 22°C, вывести "Идеальная температура"
- Если температура меньше 15°C, вывести "Слишком холодно"

Шаг 07:

Миссия 7

Лена наблюдает за своим садом. Она хочет проверить, температура воздуха выше или ниже 20 градусов. Помогите ей написать условие.

Подсказка: Нужно использовать операторы сравнения для проверки температуры.

Ответ:

- `temperature > 20`
- `temperature < 20`
- `temperature == 20`
- `temperature != 20`

Шаг 08:

Миссия 8

Сегодня температура в саду 15 градусов. Сравните её с идеальной температурой для растений, которая равна 18. Какая из следующих команд корректно проверяет, что температура меньше 18?

Подсказка: Используйте оператор сравнения для проверки, меньше ли температура идеальной.

Ответ:

- `temperature < 18`

- temperature > 18
- temperature == 18
- temperature != 18

Шаг 09:

Миссия 9

Тимур наблюдает за своим растением. Он заметил, что температура в комнате опустилась до 10 градусов. Он должен ли он включить обогреватель?

Подсказка: Растения нуждаются в тепле, но не все температуры подходят. Вспомни, какая температура подходит для роста.

Ответ:

- **Да, 10 градусов слишком холодно для растений**
- Нет, растения могут расти при 10 градусах
- Да, но растения могут расти и при 5 градусах
- Нет, температура подходящая для растений

Шаг 10:

Миссия 10

Задание: Расположите температуры по порядку от самой низкой к самой высокой:

1. 30 градусов
2. 15 градусов
3. 22 градуса
4. 8 градусов

 **Подсказка:**

Помни, что температура 8 градусов — самая холодная, а 30 градусов — самая жаркая.

Ответ:

- 8 градусов
- 15 градусов
- 22 градуса
- 30 градусов

Урок 5 - Свет или тьма — работа с булевыми значениями

Шаг 01:

Миссия 1

Тимур проверяет, включен ли свет в теплице. Помогите ему сделать проверку с помощью булевого значения.

Подсказка: Используй условие, которое проверяет, включен ли свет (True или False).

Ответ:

- **light_on == True**
- light_on != False
- light_on == False
- light_on > 0

Шаг 02:

Миссия 2

Лена наблюдает за светом в своей теплице. Свет выключен, и она хочет узнать, пусто ли в помещении.

Подсказка: Помни, что проверка на пустоту будет делать свет выключенным равным False.

Ответ:

- **light_on == False**
- light_on == True
- light_on != True
- light_on != False

Шаг 03:

Миссия 3

Задача: Сопоставьте булево значение с его правильным описанием.

Булевы значения:

Ответ:

- True
- False

Описание:

1. Свет включен
2. Свет выключен

 **Подсказка:**

True — это когда свет включен, а False — это когда он выключен.

- **True** — Свет включен
- **False** — Свет выключен

Шаг 04:

Миссия 4

Тимур хочет проверить, включен ли свет в теплице, и если он включен, начать полив. Для этого он использует условие. Какой код правильно проверит, что свет включен?

Подсказка: Свет включен, когда булево значение равно True.

Ответ:

- **if light_on == True:**
- if light_on == False:
- if light_on != True:
- if light_on:

Шаг 05:

Миссия 5

Задание: Расположите булевы значения по порядку от ложного к истинному:

1. True
2. False

💡 Подсказка:

Помните, что False — это ложь, а True — это истина.

Ответ:

- False
- True

💡 Шаг 06:

Миссия 6

Задача: Сопоставьте действие с булевым значением, которое оно вызывает.

Действия:

Ответ:

- Включить свет
- Выключить свет

Булевы значения:

1. True
2. False

💡 Подсказка:

Если свет включен, то булево значение будет True, а если выключен — False.

- **Включить свет** — True
- **Выключить свет** — False

💡 Шаг 07:

Миссия 7

Лена проверяет состояние света в теплице. Если свет включен, она продолжит полив. Помогите ей написать условие с использованием булевых значений.

Подсказка: В случае, когда свет включен, переменная light_on должна быть True.

Ответ:

- **if light_on == True:**
- if light_on != True:
- if light_on == False:
- if light_on:

💡 Шаг 08:

Миссия 8

Тимур проверяет, включен ли свет в теплице. Помогите ему сделать проверку с помощью булевого значения.

Подсказка: Используйте условие, которое проверяет, включен ли свет (True или False).

Ответ:

- **light_on == True**
- light_on != False
- light_on == False
- light_on > 0

🔮 Шаг 09:

Миссия 9

Лена наблюдает за светом в своей теплице. Свет выключен, и она хочет узнать, пусто ли в помещении.

Подсказка: Помни, что проверка на пустоту будет делать свет выключенным равным False.

Ответ:

- **light_on == False**
- light_on == True
- light_on != True
- light_on != False

🔮 Шаг 10:

Миссия 10

Задача: Сопоставьте булево значение с его правильным описанием.

Булевы значения:

Ответ:

- True
- False

Описание:

1. Свет включен
2. Свет выключен

🔮 **Подсказка:**

True — это когда свет включен, а False — это когда он выключен.

- **True** — Свет включен
- **False** — Свет выключен

🔮 Шаг 11:

Миссия 11

Тимур хочет проверить, включен ли свет в теплице, и если он включен, начать полив. Для этого он использует условие. Какой код правильно проверит, что свет включен?

Подсказка: Свет включен, когда булево значение равно True.

Ответ:

- **if light_on == True:**
- if light_on == False:
- if light_on != True:
- if light_on:

🔮 Шаг 12:

Миссия 12

Задание: Расположите булевы значения по порядку от ложного к истинному:

1. True
2. False

Подсказка:

Помните, что False — это ложь, а True — это истина.

Ответ:

- False
- True

Шаг 13:

Миссия 13

Задача: Сопоставьте действие с булевым значением, которое оно вызывает.

Действия:

Ответ:

- Включить свет
- Выключить свет

Булевы значения:

1. True
2. False

Подсказка:

Если свет включен, то булево значение будет True, а если выключен — False.

- **Включить свет** — True
- **Выключить свет** — False

Шаг 14:

Миссия 14

Лена проверяет состояние света в теплице. Если свет включен, она продолжит полив. Помогите ей написать условие с использованием булевых значений.

Подсказка: В случае, когда свет включен, переменная light_on должна быть True.

Ответ:

- **if light_on == True:**
- if light_on != True:
- if light_on == False:
- if light_on:

Урок 6 - Реакция растения — структура if elif else

Шаг 01:

Миссия 1

Задание: Расположите состояния света по порядку: сначала выключен, потом включен.

1. True
2. False

Подсказка:

True — это включенный свет, а False — это выключенный свет.

Ответ:

- False
- True

Шаг 02:

Миссия 2

Тимур хочет узнать, включен ли свет в теплице, и продолжить работу, только если свет включен. Какой код правильно проверяет включен ли свет?

Подсказка: Условие должно проверять, что переменная `light_on` равна `True`.

Ответ:

- **if `light_on` == `True`:**
- if `light_on` != `False`:
- if `light_on` == `False`:
- if `light_on`:

Шаг 03:

Миссия 3

Задача: Сопоставьте булево значение с действием, которое оно вызывает в теплице.

Булевы значения:

Ответ:

- True
- False

Действия:

1. Включить свет
2. Выключить свет

Подсказка:

`True` означает включенный свет, а `False` — выключенный.

- **True** — Включить свет
- **False** — Выключить свет

Шаг 04:

Миссия 4

Вы только что посадили семена в свой сад. Как вы думаете, что нужно сделать, чтобы растение начало расти?

Подсказка: Посмотрите на условия, при которых растение растет.

Ответ:

- **Обеспечить растение водой и светом.**
- Ничего не делать, просто ждать.
- Постоянно менять место для растения.
- Поливать растение только вечером.

Шаг 05:

Миссия 5

Ваше растение не растет. Как вы проверите, что причина в недостатке света или воды?

Подсказка: Подумайте, как можно проверить состояние воды и света для растения.

Ответ:

- **Проверить уровень воды и наличие света в месте, где растение растет.**
- Просто пересадить растение в другой горшок.
- Постоянно поливать растение, не проверяя его состояние.
- Положить растение на более темное место, чтобы оно привыкло к тени.

Шаг 06:

Миссия 6

Задача: Сопоставьте действия с соответствующими условиями, когда они могут быть выполнены в коде.

Действия:

Ответ:

- Полить растение
- Поставить растение на солнце
- Проверить количество воды
- Пересадить растение

Условия:

1. Растение слишком сухое.
2. Растение не получает достаточно света.
3. Вы не уверены, сколько воды в горшке.
4. Корни растения стали слишком большими для горшка.

Подсказка:

Не все действия подходят для всех ситуаций, подумайте, что именно поможет растению.

- **Полить растение** — Растение слишком сухое.
- **Поставить растение на солнце** — Растение не получает достаточно света.
- **Проверить количество воды** — Вы не уверены, сколько воды в горшке.
- **Пересадить растение** — Корни растения стали слишком большими для горшка.

Шаг 07:

Миссия 7

Растение выглядит вялыми и плохо растет. Какое из этих условий может быть причиной?

Подсказка: Проверьте, насколько растение получает достаточно воды и света.

Ответ:

- **Недостаток воды или света.**
- Пересадка в новый горшок.
- Проблемы с температурой в помещении.
- Недостаток удобрений.

Шаг 08:

Миссия 8

Задача: Сопоставьте ситуации с их возможными решениями.

Ситуации:

Ответ:

- Растение не растёт
- Растение желтеет
- Растение погибает

Решения:

1. Проверить условия полива и света.
2. Обеспечить растение достаточным количеством воды.
3. Пересадить в больший горшок и улучшить условия для роста.

 Подсказка:

Каждая ситуация требует внимательного подхода и анализа состояния растения.

- **Растение не растёт** — Проверить условия полива и света.
- **Растение желтеет** — Обеспечить растение достаточным количеством воды.
- **Растение погибает** — Пересадить в больший горшок и улучшить условия для роста.

 Шаг 09:

Миссия 9

В саду появилась туча. Если идёт дождь, растения радуются. Какую конструкцию Python выбрать, чтобы проверить условие?

Подсказка: Вспомни, что условие в Python начинается со слова, означающего «если».

Ответ:

- **if**
- else
- print
- while

 Шаг 10:

Миссия 10

Герой заметил: если солнце светит, растение растёт. Какой блок использовать для альтернативы, если солнца нет?

Подсказка: Подумай, какое слово используется в Python для ветки «иначе».

Ответ:

- elif
- **else**
- if
- for

 Шаг 11:

Миссия 11

Если земля сухая, нужно полить растение. Если земля влажная, поливать не нужно. Что будет в коде во второй проверке?

Подсказка: Вторая часть условия описывает противоположную ситуацию.

Ответ:

- **else**
- elif

- if
- print

🔮 Шаг 12:

Миссия 12

В саду три состояния погоды: солнце, дождь или туман. Какую конструкцию использовать, чтобы проверить все варианты?

Подсказка: Вспомни про слово, которое означает «иначе если».

Ответ:

- elif
- else
- for
- and

🔮 Шаг 13:

Миссия 13

Герой проверяет: если росток меньше 10 см, он маленький. Как записать условие?

Подсказка: Вспомни знак «меньше» в Python.

Ответ:

- if height < 10:
- if height > 10:
- if height = 10:
- if height == 10:

🔮 Шаг 14:

Миссия 14

Сопоставь ситуацию в саду с правильным блоком кода.

Подсказка: Вспомни, что «if» — это «если», «elif» — «иначе если», «else» — «иначе».

Ответ:

- Если идёт дождь — if
- Если туман, но не дождь — elif
- Во всех остальных случаях — else

🔮 Шаг 15:

Миссия 15

Сопоставь рост растения и действие героя.

Подсказка: Подумай, что нужно делать, если растение ещё маленькое или уже выросло.

Ответ:

- Росток меньше 10 см — Полить водой
- Росток выше 20 см — Подвязать к опоре
- Иначе — Оставить расти спокойно

🔮 Шаг 16:

Миссия 16

Сопоставь ключевое слово с его назначением.

Подсказка: Эти три слова помогают обрабатывать разные ситуации.

Ответ:

- **if** — Проверяет первое условие
- **elif** — Проверяет следующее условие
- **else** — Выполняется, если предыдущие условия не подошли

Урок 7 - Садовник-советник — простая логика решений в коде

Шаг 01:

Миссия 1

Садовник заметил, что цветок грустит. Он думает: если идёт дождь — растение будет радостным, иначе оно будет вянуть. Какую конструкцию он должен использовать?

Подсказка: Подумай о том, что проверяет условие и что выполняется только в одном из случаев.

Ответ:

- **if / else**
- for / while
- print / input
- def / return

Шаг 02:

Миссия 2

Герой нашёл сухое растение. Если полить его — оно оживёт, иначе останется сухим. Какая строка кода это отразит?

Подсказка: Подумай: условие проверяет, есть ли вода.

Ответ:

- **if water: print('Растение оживает')**
- print('Растение оживает')
- else: print('Растение оживает')
- for water in plant: print('Растение оживает')

Шаг 03:

Миссия 3

Садовник проверяет: если температура ниже 0, растение замёрзнет. Если выше 30 — перегреется. Иначе будет расти нормально. Какой оператор использовать для трёх вариантов?

Подсказка: Нужно больше, чем просто if/else.

Ответ:

- **if / elif / else**
- for / break
- try / except
- input / output

Шаг 04:

Миссия 4

Герой хочет проверить: если земля влажная — он не поливает, если сухая — он поливает. Какая проверка подойдёт?

Подсказка: Подумай, что проверяет True и False.

Ответ:

- **if soil_dry: print('Полить растение')**
- if soil_dry: print('Оставить без воды')
- print('Полить растение')
- else: print('Полить растение')

Шаг 05:

Миссия 5

Команда садовников спорит: если светло — растения фотосинтезируют, если темно — отдыхают. Какое условие описывает эту логику?

Подсказка: Проверка связана с переменной light.

Ответ:

- **if light: print('Фотосинтез') else: print('Отдых')**
- for light in garden: print('Фотосинтез')
- print('Фотосинтез')
- def light(): print('Фотосинтез')

Шаг 06:

Миссия 6

Сопоставь условия с их результатом для растений:

Ответ:

- if rain
 - if sun
 - if frost
1. Растение получает свет
 2. Растение получает влагу
 3. Растение замерзает

Подсказка: Вспомни, как природа реагирует на разные явления.

- **if rain** — Растение получает влагу
- **if sun** — Растение получает свет
- **if frost** — Растение замерзает

Шаг 07:

Миссия 7

Садовник проверяет почву. Сопоставь условие и действие:

Ответ:

- if soil_dry
 - if soil_wet
 - if soil_frozen
1. Полить растение
 2. Оставить как есть

3. Подождать потепления

Подсказка: Подумай, что делает садовник в каждой ситуации.

- **if soil_dry** — Полить растение
- **if soil_wet** — Оставить как есть
- **if soil_frozen** — Подождать потепления

🧙 Шаг 08:

Миссия 8

Сопоставь условие и реакцию растения:

Ответ:

- if light and water
 - if not light
 - if too_hot
1. Растение отдыхает
 2. Растение вянет
 3. Растение растёт

Подсказка: Обрати внимание на комбинацию условий.

- **if light and water** — Растение растёт
- **if not light** — Растение отдыхает
- **if too_hot** — Растение вянет

🧙 Шаг 09:

Миссия 9

Садовник делится опытом. Сопоставь условие Python с его смыслом:

Ответ:

- if plant_grows
 - if not soil_dry
 - if temperature < 0
1. Земля не сухая
 2. Растение развивается
 3. Холодно, мороз

Подсказка: Переведи условия с английского на обычные слова.

- **if plant_grows** — Растение развивается
- **if not soil_dry** — Земля не сухая
- **if temperature < 0** — Холодно, мороз

🧙 Шаг 10:

Миссия 10

Сопоставь условие и действие команды героев:

Ответ:

- if weeds
- if harvest_ready

- if storm

1. Собрать урожай
2. Укрыть растения
3. Убрать сорняки

Подсказка: Подумай, что делают садовники при каждой ситуации.

- **if weeds** — Убрать сорняки
- **if harvest_ready** — Собрать урожай
- **if storm** — Укрыть растения

Шаг 11:

Миссия 11

Расположи шаги ухода за растением в правильном порядке:

1. Проверить сухая ли почва
2. Полить растение
3. Посмотреть на солнце
4. Оставить отдыхать в тени

Подсказка: Сначала идёт проверка условий, затем действие.

Ответ:

- Проверить сухая ли почва
- Полить растение
- Посмотреть на солнце
- Оставить отдыхать в тени

Шаг 12:

Миссия 12

Упорядочи действия садовника:

1. **Если есть буря** — накрыть растения
2. **Если солнце** — вынести горшки
3. **Если дождь** — собрать дождевую воду
4. **Если мороз** — занести в дом

Подсказка: Думай, что проверяется в первую очередь для безопасности растений.

Ответ:

- **Если есть буря** — накрыть растения
- **Если мороз** — занести в дом
- **Если дождь** — собрать дождевую воду
- **Если солнце** — вынести горшки

Шаг 13:

Миссия 13

Расположи проверки кода по важности:

1. if temperature < 0
2. if soil_dry
3. if weeds

4. if harvest_ready

Подсказка: Сначала жизненно важные условия, потом остальные.

Ответ:

- if temperature < 0
- if soil_dry
- if weeds
- if harvest_ready

Урок 8 - Жив ли росток — мини-проект симулятор условий роста

Шаг 01:

Миссия 1

Герой посадил семена. Нужно проверить: если есть солнце, росток вырастет. Какое условие верное?

Подсказка: Подумай: росток любит солнце, без него он не растет.

Ответ:

- **if sun == True:**
- if seed = True:
- if water:
- if plant > 0

Шаг 02:

Миссия 2

Садовник проверяет: если полил растение — оно растет, иначе грустит. Какая конструкция подходит?

Подсказка: Используй if...else: два варианта поведения.

Ответ:

- **if watered: grow() else: sad()**
- if watered: grow()
- else watered: sad()
- if else water = True:

Шаг 03:

Миссия 3

Лепесток проверяет: если тепло — цветок раскрывается, если холодно — закрывается. Как написать?

Подсказка: Подумай о сравнении температур и выборе действий.

Ответ:

- **if temperature > 15: open() else: close()**
- if temperature: open()
- if temperature < 15 then close()
- if else temp > 15:

Шаг 04:

Миссия 4

Фея проверяет: если есть солнце и вода — растение растёт. Какое условие правильное?

Подсказка: Два условия можно соединить через and.

Ответ:

- **if sun and water:**
- if sun or water:
- if sun + water:
- if sun = water:

🧙 Шаг 05:

Миссия 5

Сопоставь условия и действия садовника.

Подсказка: Вспомни: полив = рост, холод = замедление, солнце = радость.

Ответ:

- **if watered** — plant grows
- **if cold** — plant slows down
- **if sunny** — plant is happy

🧙 Шаг 06:

Миссия 6

Сопоставь Python-операторы с их смыслом в саду.

Подсказка: and — это оба условия, or — хотя бы одно.

Ответ:

- **and** — нужно солнце и вода
- **or** — достаточно или солнца, или лампы
- **else** — если условия не выполнены
- **if** — проверка условия

🧙 Шаг 07:

Миссия 7

Сопоставь действия героя с кодом.

Подсказка: Подумай, что значит дождь, солнце и тень.

Ответ:

- **if rain:** — растение пьёт воду
- **if sun:** — растение тянется вверх
- **else:** — растение отдыхает

🧙 Шаг 08:

Миссия 8

Расположи шаги симуляции роста растения в правильном порядке.

Подсказка: Сначала проверка условий, затем выбор действия, потом результат.

Ответ:

- Проверить солнце и воду
- Выбрать действие (расти или отдыхать)
- Вывести результат роста

Шаг 09:

Миссия 9

Упорядочи действия садовника при холодной погоде.

Подсказка: Сначала проверка, потом реакция, затем вывод результата.

Ответ:

- Проверить температуру
- Закрыть лепестки
- Сообщить: растение отдыхает

Шаг 10:

Миссия 10

Расположи шаги проекта «Жив ли росток?»

Подсказка: Сначала проверка условий (солнце, вода), потом действия, потом вывод.

Ответ:

- Проверить наличие солнца и воды
- Применить условия if/else
- Показать результат: росток жив или нет

Модуль 3 - Модуль 3 «Огородная рутина»

Урок 1 - Полив по графику — цикл for и range()

Шаг 01:

Миссия 1

Лена хочет, чтобы её огурцы поливались 5 дней подряд. Как правильно написать цикл?

Подсказка: range(n) даёт числа от 0 до n-1.

Ответ:

- **for day in range(5): print('День полива:', day)**
- for day in range(1,5): print('День полива:', day)
- for day in range(6): print('День полива:', day)
- for day in range(0): print('День полива:', day)

Шаг 02:

Миссия 2

Миша хочет поливать клубнику с 1 по 4 день. Какой цикл правильный?

Подсказка: В range(a,b) число b не включается.

Ответ:

- **for day in range(1,5): print('Полив клубники день', day)**
- for day in range(1,4): print('Полив клубники день', day)
- for day in range(5): print('Полив клубники день', day)
- for day in 1..5: print('Полив клубники день', day)

Шаг 03:

Миссия 3

Аня считает, что `range(2,7)` даст числа от 2 до 7. Верно ли это?

Подсказка: Второе число в `range` не включается.

Ответ:

- Нет, будет от 2 до 6
- Да, будет от 2 до 7

🔗 Шаг 04:

Миссия 4

Игорь хочет поливать морковь через день: 0, 2, 4, 6. Какой код подойдёт?

Подсказка: В `range()` можно задать шаг третьим параметром.

Ответ:

- **for day in range(0,7,2): print('Полив моркови день', day)**
- `for day in range(7,2): print('Полив моркови день', day)`
- `for day in range(2,7): print('Полив моркови день', day)`
- `for day in range(4): print('Полив моркови день', day)`

🔗 Шаг 05:

Миссия 5

Сопоставь выражения `range()` с тем, какие числа они дают.

Подсказка: Помни: первое число — начало, второе — конец (не включается), третье — шаг.

Ответ:

- **`range(3)` — 0, 1, 2**
- **`range(2,5)` — 2, 3, 4**
- **`range(1,7,2)` — 1, 3, 5**
- **`range(5,1,-1)` — 5, 4, 3, 2**

🔗 Шаг 06:

Миссия 6

Сопоставь героев и их цикл полива.

Подсказка: Смотри на диапазон и шаг.

Ответ:

- **Тима** — `for i in range(3):` → 0,1,2
- **Лена** — `for d in range(1,4):` → 1,2,3
- **Миша** — `for d in range(0,7,2):` → 0,2,4,6
- **Аня** — `for d in range(5,0,-1):` → 5,4,3,2,1

🔗 Шаг 07:

Миссия 7

Соедини код и его описание.

Подсказка: Читай внимательно, где цикл начинается, где заканчивается, и какой шаг.

Ответ:

- **for i in range(4)** — Повторить 4 раза: 0,1,2,3
- **for d in range(2,6)** — Начать с 2 и до 5 включительно
- **for j in range(10,5,-2)** — Начать с 10 и уменьшать до 6

- **for k in range(1,10,3)** — Начать с 1 и увеличивать по 3

💡 Шаг 08:

Миссия 8

Расположи шаги: Тима хочет сделать цикл для полива 5 дней.

Подсказка: Подумай: сначала цикл, потом range, потом print.

Ответ:

- Написать print('Полив!')
- Написать for day in range(5):
- Внутри цикла добавить действие

💡 Шаг 09:

Миссия 9

Расположи шаги, чтобы сделать цикл с шагом 2 для 0..6.

Подсказка: Сначала указываем начало, конец и шаг, потом действие.

Ответ:

- Написать print('Полив моркови')
- Написать for day in range(0,7,2):
- Внутри цикла добавить действие

📖 Урок 2 - Засуха и контроль — цикл while и условия выхода

💡 Шаг 01:

Миссия 1

В саду началась засуха, и Лея поливает грядку водой. Как правильно записать цикл while , чтобы он работал, пока в бочке есть вода?

Подсказка: В условии цикла пишется проверка, когда нужно продолжать действия.

Ответ:

- **while water > 0:**
- while: water > 0
- if water > 0:
- repeat water > 0:

💡 Шаг 02:

Миссия 2

Тим заметил, что семена сохнут. Он написал цикл: while dry_soil: . Что это условие означает?

Подсказка: Переменная в условии цикла сама может быть True или False .

Ответ:

- **Цикл будет выполняться, пока почва сухая**
- Цикл будет выполняться всегда
- Цикл не будет выполняться
- Цикл выполнится только один раз

Шаг 03:

Миссия 3

Чтобы растения не погибли, София хочет завершить цикл, если воды больше нет. Какая команда используется для выхода из цикла?

Подсказка: Эта команда прерывает выполнение цикла в любой момент.

Ответ:

- **break**
- stop
- exit
- quit

Шаг 04:

Миссия 4

Крис написал: `while True:` . Что произойдет с таким циклом, если внутри нет `break` ?

Подсказка: Цикл всегда проверяет условие, и `True` не меняется.

Ответ:

- **Цикл будет бесконечным**
- Цикл выполнится один раз
- Цикл никогда не начнётся
- Цикл остановится автоматически

Шаг 05:

Миссия 5

В саду у Нины осталось 5 капель воды. Она написала: `while drops > 0:` . Что произойдёт, когда `drops` станет 0?

Подсказка: В условии цикла должно выполняться сравнение.

Ответ:

- **Цикл завершится**
- Цикл продолжит выполняться
- Цикл выдаст ошибку
- Цикл выполнится ещё один раз

Шаг 06:

Миссия 6

Лея и Тим обсуждают команды для управления циклами. Сопоставьте команду с её действием.

Подсказка: Вспомни, какая команда завершает цикл, а какая пропускает шаг.

Ответ:

- **break** — Полностью завершает цикл
- **continue** — Пропускает текущую итерацию
- **while** — Запускает цикл с условием

Шаг 07:

Миссия 7

У Софии есть список растений. Она использует цикл `while`, чтобы поливать их одно за другим. Сопоставьте действие с его результатом.

Подсказка: Подумай: условие цикла — проверка, команда внутри — действие.

Ответ:

- **while plants:** — Продолжать, пока список не пуст
- **plants.pop()** — Удаляет и возвращает растение
- **print('Полив')** — Выводит сообщение на экран

🔗 Шаг 08:

Миссия 8

Тим проверяет влажность почвы с помощью датчика. Сопоставьте условие цикла и поведение программы.

Подсказка: Если условие ложно, цикл не выполняется.

Ответ:

- **while moisture > 50:** — Поливать пока почва достаточно влажная
- **while moisture <= 20:** — Поливать пока почва слишком сухая
- **while True:** — Цикл без условий, бесконечный

🔗 Шаг 09:

Миссия 9

Крис забыл поставить условие выхода из цикла. Сопоставьте его ошибки с последствиями.

Подсказка: Ошибки в условии цикла могут приводить к зависанию программы.

Ответ:

- **while True без break** — Программа зависает
- **while water > 0 (но water не уменьшается)** — Цикл выполняется бесконечно
- **while False** — Цикл не выполняется ни разу

🔗 Шаг 10:

Миссия 10

Нина использует цикл для подсчета, сколько раз она полила грядку. Сопоставьте переменные и их значение.

Подсказка: Счётчик нужно увеличивать в теле цикла.

Ответ:

- **count = 0** — Начальное значение счётчика
- **count += 1** — Увеличивает счётчик на 1
- **while water > 0** — Цикл продолжается, пока есть вода

🔗 Шаг 11:

Миссия 11

Тим написал программу для полива. Расставь шаги в логическом порядке:

1. Запустить цикл
2. Вывести сообщение
3. Проверить уровень воды
4. Уменьшить уровень воды

Подсказка: Логика цикла всегда повторяет проверку и действия.

Ответ:

- Запустить цикл
- Проверить уровень воды
- Вывести сообщение
- Уменьшить уровень воды

Шаг 12:

Миссия 12

Крис хочет написать цикл, который останавливается при сухой почве. Расставь шаги:

1. Проверить влажность почвы
2. Написать условие while
3. Вывести сообщение о поливе
4. Использовать break при слишком сухой почве

Подсказка: Break используется внутри цикла, когда нужно срочно выйти.

Ответ:

- Написать условие while
- Проверить влажность почвы
- Вывести сообщение о поливе
- Использовать break при слишком сухой почве

Шаг 13:

Миссия 13

Нина хочет подсчитать, сколько раз она полила растения. Расставь шаги:

1. Установить счётчик
2. Написать цикл while
3. Увеличить счётчик
4. Вывести результат

Подсказка: Счётчик заводят до цикла, увеличивают внутри, выводят после.

Ответ:

- Установить счётчик
- Написать цикл while
- Увеличить счётчик
- Вывести результат

Шаг 14:

Миссия 14

Лея использует цикл для автоматического полива. Расставь шаги:

1. Считать данные датчика
2. Проверить условие
3. Полить растения
4. Уменьшить уровень воды

Подсказка: В цикле всегда: считывание → проверка → действие → обновление.

Ответ:

- Считать данные датчика
- Проверить условие
- Полить растения
- Уменьшить уровень воды

 Шаг 15:

Миссия 15

Тим написал цикл, но забыл уменьшать воду. Расставь шаги так, чтобы программа не зациклилась:

1. Задать начальный уровень воды
2. Написать цикл while
3. Уменьшить воду в цикле
4. Вывести сообщение о поливе

Подсказка: Если не уменьшать ресурс, цикл станет бесконечным.

Ответ:

- Задать начальный уровень воды
- Написать цикл while
- Вывести сообщение о поливе
- Уменьшить воду в цикле

 Урок 3 - Считаем грядки — счётчики и итерации

 Шаг 01:

Миссия 1

Герой Гроу и его друзья пришли в сад. На первой грядке они посадили 5 морковок. Какое значение нужно хранить в счётчике, чтобы отразить количество морковок?

Подсказка: Счётчик фиксирует количество элементов. Здесь важно посчитать морковки, а не грядки.

Ответ:

- 5
- 1
- 10
- 0

 Шаг 02:

Миссия 2

На второй грядке друг Лили посадил 3 огурца. Каждый раз, когда он сажает огурец, он увеличивает счётчик. Сколько станет на счётчике после посадки?

Подсказка: Начало всегда с нуля. Потом прибавляем по одному за каждый огурец.

Ответ:

- 3
- 2
- 5
- 1

Шаг 03:

Миссия 3

На третьей грядке герой Том поливал растения 4 раза. Какой цикл в Python подойдёт для повторения действия несколько раз?

Подсказка: Когда мы знаем точное количество повторов, используем цикл с диапазоном.

Ответ:

- **for**
- if
- while
- def

Шаг 04:

Миссия 4

На четвёртой грядке Лили считает ростки. Она использует цикл `for i in range(5):`. Сколько раз выполнится цикл?

Подсказка: Диапазон `range(n)` всегда идёт от 0 до `n-1`.

Ответ:

- **5**
- 4
- 6
- 0

Шаг 05:

Миссия 5

В саду разные действия: полив, посадка, сбор урожая. Соотнеси действие с его описанием.

Подсказка: Думай о порядке: сначала посадка, потом уход, потом сбор.

Ответ:

- **Посадка** — Мы кладем семена в землю.
- **Полив** — Мы наливаем воду на грядку.
- **Сбор урожая** — Мы собираем плоды после роста.

Шаг 06:

Миссия 6

Герои посадили разные культуры. Соотнеси растение с количеством семян.

Подсказка: Просто вспомни, сколько каждый герой посадил.

Ответ:

- **Морковка** — 5
- **Огурец** — 3
- **Кукуруза** — 2

Шаг 07:

Миссия 7

Каждое действие в Python связано с понятием. Соотнеси команды и их назначение.

Подсказка: Вспомни базовые конструкции: цикл, сложение, печать.

Ответ:

- **for** — Повторяет действия несколько раз.
- **+** — Складывает числа.
- **print** — Выводит текст или число.

 Шаг 08:

Миссия 8

На грядках подсчитали растения. Соотнеси грядку и количество растений.

Подсказка: Это как пара ключ-значение: каждая грядка имеет своё число.

Ответ:

- **Грядка 1** — 5 морковок
- **Грядка 2** — 3 огурца
- **Грядка 3** — 1 помидор

 Шаг 09:

Миссия 9

В саду герои используют разные счётчики. Соотнеси героя и его счётчик.

Подсказка: Запомни, что счётчик хранит число действий героя.

Ответ:

- **Гроу** — Посадил 2 кукурузы
- **Лили** — Посчитала 5 морковок
- **Том** — Сделал 3 полива

 Шаг 10:

Миссия 10

Каждое слово в саду связано с действием. Соотнеси слово и действие в коде.

Подсказка: Привяжи понятия из жизни к Python-командам.

Ответ:

- **Семя** — Начальное значение
- **Росток** — Результат выполнения цикла
- **Вода** — Прибавление к счётчику

 Шаг 11:

Миссия 11

Расположи шаги работы в саду по порядку.

Подсказка: Всегда начинаем с посадки, потом уход, и только потом сбор.

Ответ:

- Посадить семена
- Полить растения
- Подождать роста
- Собрать урожай

 Шаг 12:

Миссия 12

Расположи команды Python в порядке выполнения счётчика.

Подсказка: Сначала мы задаём начальное значение, потом запускаем цикл, а внутри увеличиваем и выводим.

Ответ:

- counter = 0
- for i in range(3):
- counter += 1
- print(counter)

Шаг 13:

Миссия 13

Расположи действия Тома в правильном порядке: он ухаживает за грядкой.

Подсказка: Сначала полив, потом подсчёт, потом вывод результата.

Ответ:

- Сделать полив
- Увеличить счётчик
- Напечатать результат

Шаг 14:

Миссия 14

Гроу решил написать программу. Расположи шаги написания цикла правильно.

Подсказка: Сначала нужно создать счётчик, потом определить цикл, и только потом выводить.

Ответ:

- Создать переменную-счётчик
- Написать цикл for
- Внутри цикла увеличить счётчик
- Вывести счётчик

Шаг 15:

Миссия 15

Лили захотела собрать все плоды. Расположи шаги её программы.

Подсказка: Сначала список плодов, потом цикл, внутри — вывод каждого элемента.

Ответ:

- Создать список плодов
- Запустить цикл for
- Вывести каждый плод

Урок 4 - Вредители! — цикл с проверкой (флаги и условия выхода)

Шаг 01:

Миссия 1

В саду завелись жуки-листоеды. Команда решила проверять грядки по очереди. Какое условие выхода из цикла нужно, чтобы остановиться, когда найден первый жук?

Подсказка: Цикл проверяет все грядки, но как только нашли жука — можно выйти.

Ответ:

- **Использовать флаг и оператор break**
- Продолжать до конца всегда
- Сразу завершить программу
- Проверять без условия выхода

🔗 **Шаг 02:**

Миссия 2

Оля проверяет клубничные грядки. Она использует переменную `found`, чтобы хранить, нашла ли она жука. Какое значение у переменной в начале?

Подсказка: До проверки Оля не уверена, есть ли жуки.

Ответ:

- **False**
- True
- None
- 0

🔗 **Шаг 03:**

Миссия 3

Иван проверяет грядки с морковью. Он пишет цикл `for`. Где лучше всего поставить проверку `if found`: ?

Подсказка: Проверка нужна после каждого шага цикла, чтобы вовремя остановиться.

Ответ:

- **Внутри цикла, сразу после проверки грядки**
- Вне цикла
- До начала цикла
- После завершения цикла

🔗 **Шаг 04:**

Миссия 4

Катя нашла жука на второй грядке. Какой оператор Python сразу завершает цикл?

Подсказка: Вспомни команду, которая «ломает» цикл.

Ответ:

- **break**
- stop
- exit
- continue

🔗 **Шаг 05:**

Миссия 5

Если на грядке нет жуков, что должна сделать программа?

Подсказка: Важно не завершать цикл раньше времени.

Ответ:

- Завершить программу

- **Перейти к следующей грядке**
- Удалить переменную found
- Пропустить все проверки

🔗 Шаг 06:

Миссия 6

Дима проверяет грядки и использует while not found: . Когда цикл остановится?

Подсказка: Подумай, когда условие not found станет ложным.

Ответ:

- **Когда found станет True**
- Когда found станет False
- Когда программа завершится
- Никогда

🔗 Шаг 07:

Миссия 7

Сопоставьте героев и их действия при встрече с жуками.

Подсказка: Вспомни, что каждый герой делает своё.

Ответ:

- **Оля** — Поднимает флаг found = True
- **Иван** — Использует break для выхода
- **Катя** — Проверяет каждую грядку до конца
- **Дима** — Работает с циклом while

🔗 Шаг 08:

Миссия 8

Сопоставьте переменные и их назначение в программе поиска жуков.

Подсказка: Подумай, зачем нужны счетчик, флаг и список.

Ответ:

- **found** — Флаг: найден ли жук
- **i** — Номер текущей грядки
- **beds** — Список всех грядок
- **count** — Количество найденных жуков

🔗 Шаг 09:

Миссия 9

Сопоставьте циклы и ситуации.

Подсказка: Разные типы циклов подходят под разные задачи.

Ответ:

- **for** — Проверяем все грядки подряд
- **while** — Проверяем, пока жук не найден
- **break** — Прерываем цикл при первом жуке
- **continue** — Пропускаем текущую грядку

🔗 Шаг 10:

Миссия 10

Сопоставьте ошибки и способы их исправления.

Подсказка: Важно помнить правильный синтаксис Python.

Ответ:

- **break вне цикла** — Ошибка — break можно только внутри цикла
- **continue вне цикла** — Ошибка — continue работает только в цикле
- **if found = True:** — Ошибка — нужно использовать == или просто if found
- **while found = False:** — Ошибка — нужно использовать == или not

🔗 Шаг 11:

Миссия 11

Расположите шаги проверки грядок в правильном порядке.

Подсказка: Сначала начинаем проверку, потом идём по грядкам, останавливаемся, если нашли жука.

Ответ:

- Установить found = False
- Начать обход грядок
- Проверить грядку
- **Если нашли жука** — found = True и break

🔗 Шаг 12:

Миссия 12

Расположите шаги написания программы с циклом while и флагом.

Подсказка: Сначала задаём условие, потом проверяем, потом меняем флаг.

Ответ:

- found = False
- while not found:
- Проверка текущей грядки
- **Если жук найден** — found = True

🔗 Шаг 13:

Миссия 13

Расположите действия героев в порядке.

Подсказка: Каждый делает шаг за шагом: проверка — действие — выход.

Ответ:

- Оля ставит флаг found = False
- Иван начинает проверку грядок
- Катя находит жука
- Дима завершает цикл

🔗 Шаг 14:

Миссия 14

Расположите шаги от простого к сложному при обучении флагам.

Подсказка: Сначала теория, потом практика.

Ответ:

- Объяснить, что такое флаг
- Написать простой цикл for
- Добавить проверку с if
- Использовать break при нахождении жука

💡 **Шаг 15:**

Миссия 16

Расположите шаги работы программы с continue.

Подсказка: continue пропускает текущую грядку.

Ответ:

- Начало цикла
- Проверка грядки
- **Если грядка пустая** — continue
- Проверить следующую грядку

💡 **Шаг 16:**

Миссия 17

Расположите шаги финальной программы поиска жуков.

Подсказка: Вспомни весь алгоритм целиком от начала до конца.

Ответ:

- Объявить список beds
- found = False
- Начать цикл по грядкам
- **Если нашли жука** — found = True и break
- Вывести результат

📖 **Урок 5 - Много растений — вложенные циклы**

💡 **Шаг 01:**

Миссия 1

Команда героев решила посадить грядку с морковкой и луком. Они хотят использовать два цикла: внешний — по грядкам, внутренний — по растениям. Что будет делать внутренний цикл?

Подсказка: Вспомни: внутренний цикл повторяет действия для каждого элемента внутри внешнего.

Ответ:

- **Будет сажать растения в каждой грядке**
- Будет перескакивать на следующую грядку
- Будет завершать внешний цикл
- Будет очищать грядку

💡 **Шаг 02:**

Миссия 2

Игрок Вася написал код с двумя циклами: внешний — по рядам, внутренний — по растениям. Что произойдет, если убрать внутренний цикл?

Подсказка: Подумай: внешний цикл работает по рядам, но что будет с растениями?

Ответ:

- Растения будут посажены в каждом ряду
- **В рядах ничего не посадится**
- Посадка выполнится только в первой грядке
- Всё будет работать как раньше

Шаг 03:

Миссия 3

Герой Пчёлка использует вложенные циклы: внешний по дням недели, внутренний по цветам на клумбе. Что будет на выходе?

Подсказка: Подумай: на каждый день будут проверяться все цветы.

Ответ:

- **Каждый день будут обходиться все цветы**
- Все цветы проверятся только один раз
- Только один цветок на день
- Будет ошибка

Шаг 04:

Миссия 4

На огороде есть 3 грядки, и на каждой должно быть по 2 растения. Какой код с вложенными циклами подойдет?

Подсказка: Внешний цикл — грядки, внутренний — растения.

Ответ:

- **for bed in range(3): for plant in range(2): print('растение')**
- for bed in range(2): for plant in range(3): print('растение')
- for plant in range(2): print('растение')
- for bed in range(3): print('растение')

Шаг 05:

Миссия 5

Фермер Котя хочет напечатать все пары (ряд, растение) для грядки из 2 рядов и 3 растений в каждом. Сколько всего пар получится?

Подсказка: Подумай: внешний цикл × внутренний цикл = общее количество.

Ответ:

- **6**
- 5
- 3
- 2

Шаг 06:

Миссия 6

Сопоставь: что делает внешний цикл, а что — внутренний, когда герои сажают растения.

Подсказка: Внешний цикл отвечает за большие шаги, внутренний — за мелкие.

Ответ:

- **Внешний цикл** — Обходит грядки
- **Внутренний цикл** — Сажает растения в грядке

🔮 Шаг 07:

Миссия 7

Сопоставь действие с правильным циклом.

Подсказка: Один цикл работает по дням, другой — по растениям.

Ответ:

- **Внешний цикл** — Дни недели
- **Внутренний цикл** — Растения в ряду

🔮 Шаг 08:

Миссия 8

Герои проверяют урожай: внешний цикл — по грядкам, внутренний — по растениям. Сопоставь результат.

Подсказка: Вспомни: внешний цикл задаёт «рамку», внутренний — детали.

Ответ:

- **Внешний цикл** — Перебирает грядки
- **Внутренний цикл** — Перебирает растения внутри грядки

🔮 Шаг 09:

Миссия 9

Сопоставь описание с частью вложенного цикла.

Подсказка: Внутренний цикл выполняется чаще.

Ответ:

- **Внешний цикл** — Повторяется реже
- **Внутренний цикл** — Повторяется чаще

🔮 Шаг 10:

Миссия 10

Сопоставь результат работы программы.

Подсказка: Считай количество повторений: внешний × внутренний.

Ответ:

- **Внешний цикл = 3, внутренний = 2** — 6 действий
- **Внешний цикл = 2, внутренний = 4** — 8 действий

🔮 Шаг 11:

Миссия 10

Нужно сопоставить команды и действия, которые выполняют вложенные циклы при работе с садом.

Команды:

Ответ:

- Вложенный цикл обхода всех растений
- Вложенный цикл полива
- Вложенный цикл сбора урожая
- Вложенный цикл подсчёта плодов

Действия:

1. Проход по каждой грядке и каждому растению на ней
2. Полив каждого растения на каждой грядке несколько раз
3. Сбор ягод с каждого куста в каждой грядке
4. Подсчёт количества плодов у каждого растения в каждой грядке

Подсказка: Вложенные циклы часто имеют внешний цикл по грядкам и внутренний — по растениям.

- **Вложенный цикл обхода всех растений** — Проход по каждой грядке и каждому растению на ней
- **Вложенный цикл полива** — Полив каждого растения на каждой грядке несколько раз
- **Вложенный цикл сбора урожая** — Сбор ягод с каждого куста в каждой грядке
- **Вложенный цикл подсчёта плодов** — Подсчёт количества плодов у каждого растения в каждой грядке

 Шаг 12:

Миссия 11

Помогите команде Grow a Garden сопоставить элементы вложенных циклов с их задачами.

Элементы вложенных циклов:

Ответ:

- Внешний цикл
- Внутренний цикл
- Вложенный цикл с условием
- Цикл с подсчетом

Задачи:

1. Перебор всех грядок по очереди
2. Перебор всех растений в грядке
3. Обработка только тех растений, которые готовы к сбору
4. Подсчет общего урожая

Подсказка: Внешний цикл — внешний уровень перебора, внутренний — вложенный уровень, а условия помогают фильтровать.

- **Внешний цикл** — Перебор всех грядок по очереди
- **Внутренний цикл** — Перебор всех растений в грядке
- **Вложенный цикл с условием** — Обработка только тех растений, которые готовы к сбору
- **Цикл с подсчетом** — Подсчет общего урожая

 Шаг 13:

Миссия 12

В саду есть разные грядки и на каждой растут разные овощи. Сопоставьте овощ с примером вложенного цикла для обработки этих грядок.

Овощи:

Ответ:

- Помидоры
- Огурцы
- Морковь
- Капуста

Примеры вложенных циклов:

1. Для каждой грядки и каждого куста помидоров полить растение
2. Для каждой грядки и каждого огурца собрать урожай
3. Для каждой грядки и каждой морковки прополоть сорняки
4. Для каждой грядки и каждой кочана капусты проверить листья

Подсказка: Вспомните, что вложенный цикл помогает выполнять действия для каждой группы внутри другой группы.

- **Помидоры** — Для каждой грядки и каждого куста помидоров полить растение
- **Огурцы** — Для каждой грядки и каждого огурца собрать урожай
- **Морковь** — Для каждой грядки и каждой морковки прополоть сорняки
- **Капуста** — Для каждой грядки и каждой кочана капусты проверить листья

Урок 6 - Табличка роста — табличный вывод и форматирование

Шаг 01:

Миссия 1

В саду герои решили составить табличку роста растений. Какой символ лучше всего использовать для разделения колонок в простой текстовой таблице?

Подсказка: Подумай, как обычно разделяют значения в таблицах без оформления.

Ответ:

- **Пробел или табуляция**
- Точка с запятой
- Знак вопроса
- Хэштег (#)

Шаг 02:

Миссия 2

Фермер Лёня напечатал таблицу, но числа '5' и '50' оказались смещены. Какое решение лучше подойдёт, чтобы выровнять их по ширине?

Подсказка: Подумай о форматировании строк и выравнивании по ширине.

Ответ:

- Писать пробелы вручную
- **Использовать метод `format()` или f-строки с выравниванием**
- Ставить табы в случайных местах
- Добавить нули спереди

Шаг 03:

Миссия 3

Сопоставь элемент таблицы и то, как его можно отформатировать.

Подсказка: Вспомни разные способы работы с текстом и числами.

Ответ:

- **Заголовок таблицы** — Можно выравнивать по центру
- **Имя растения** — Можно выравнивать по левому краю
- **Высота в см** — Можно выравнивать по правому краю
- **Дробное число** — Можно округлить до 2 знаков

Шаг 04:

Миссия 4

Герои сделали таблицу роста. Соедини, какой символ чаще всего используется и где.

Подсказка: Вспомни про пробелы, табы и переносы строк.

Ответ:

- **Между колонками** — Пробел или табуляция
- **В конце строки таблицы** — Символ новой строки ()
- **Для выравнивания чисел** — Ширина поля в форматировании
- **Для разделения заголовков** — Пробел

Шаг 05:

Миссия 5

Садоводы спорят, как оформить таблицу. Соедини способ и его описание.

Подсказка: Форматирование строк в Python очень гибкое.

Ответ:

- **print с табами** — Быстро, но не всегда аккуратно
- **print с пробелами** — Просто, но требует ручной работы
- **f-строки** — Удобное и современное решение
- **format()** — Гибкий метод, подходит для старых версий Python

Шаг 06:

Миссия 6

Расположи шаги создания таблички роста в правильном порядке.

Подсказка: Сначала выводят заголовки, затем данные, а потом форматируют.

Ответ:

- Написать print для заголовков
- Добавить строки с данными
- Применить форматирование для выравнивания
- Вывести таблицу на экран

Шаг 07:

Миссия 7

Герои делают аккуратную таблицу. Расставь шаги форматирования чисел и текста.

Подсказка: Сначала реши, что печатать, затем задай ширину и выравнивание.

Ответ:

- Определить содержимое колонок

- Задать ширину для каждого столбца
- Установить выравнивание (лево/право/центр)
- Применить вывод с форматированием

🔗 Шаг 08:

Миссия 7

Зоя из Grow a Garden хочет показать на табличке рост моркови в сантиметрах. Какой из вариантов форматирования строки в Python правильно выведет: "Морковь: 12 см"?

Подсказка: Обратите внимание на формат спецификаторов строк.

Ответ:

- **`print(f"Морковь: {рост} см")`**
- `print("Морковь: %рост% см")`
- `print("Морковь: {} см".filter(рост))`
- `print("Морковь: " + рост + " см")`

🔗 Шаг 09:

Миссия 8

Когда Зоя хочет, чтобы таблица роста выглядела ровно, она хочет выровнять названия овощей по левому краю и рост по правому. Какой метод форматирования удобнее всего для этого?

Подсказка: Исследуйте форматные строки и спецификаторы выравнивания.

Ответ:

- **`print(f"{овощ:<10}{рост:>5} см")`**
- `print(f"{овощ:>10}{рост:<5} см")`
- `print(f"{овощ}{рост} см")`

🔗 Шаг 10:

Миссия 9

Сопоставьте: формат спецификатора → его действие.

Спецификаторы:

Ответ:

- `"{:>5}"`
- `"{:<8}"`
- `"{: ^10}"`
- `"{: .2f}"`

Действие:

1. Ограничивает число до двух десятичных
2. Выровнять по центру в поле ширины 10
3. Выровнять по левому краю в поле ширины 8
4. Выровнять по правому краю в поле ширины 5

Подсказка: Сравните порядок и ширину поля, а также вид выравнивания.

- **`{:>5}`** — Выровнять по правому краю в поле ширины 5
- **`{:<8}`** — Выровнять по левому краю в поле ширины 8
- **`{: ^10}`** — Выровнять по центру в поле ширины 10
- **`{: .2f}`** — Ограничивает число до двух десятичных

🔗 Шаг 11:

Миссия 10

Сопоставьте: описание задачи → формат вывода.

Описание:

Ответ:

- Показывать рост в метрах с двумя десятичными цифрами
- Выравнивать название растения в поле шириной 12 по центру
- Рост целым числом, справа в поле ширины 4

Форматы:

1. "{:.2f}"
2. "{:^12}"
3. "{:>4d}"

Подсказка: Подумайте, какой спецификатор отвечает за целочисленность, за число после точки, и за выравнивание.

- Показывать рост в метрах с двумя десятичными цифрами — {:.2f}
- Выравнивать название растения в поле шириной 12 по центру — {:^12}
- Рост целым числом, справа в поле ширины 4 — {:>4d}

🔗 Шаг 12:

Миссия 11

Сопоставьте: что выводят эти выражения — какой результат строки.

Выражения:

Ответ:

- f{"Яблоко":<10}{5:>3} см"
- f{"Капуста":>8}{10:^4} см"
- f{"Помидор":^9}{7:.2f} см"

Результат:

1. Яблоко 5 см
2. Капуста10 см
3. Помидор 7.00 см

Подсказка: Обратите внимание на ширину полей и позицию строки и числа.

- f{"Яблоко":<10}{5:>3} см" — Яблоко 5 см
- f{"Капуста":>8}{10:^4} см" — Капуста10 см
- f{"Помидор":^9}{7:.2f} см" — Помидор 7.00 см

🔗 Шаг 13:

Миссия 12

Упорядочьте шаги, которые делает Алекс, чтобы сформировать красивую таблицу роста в консоли:

1. Определить максимальную длину названий растений
2. Вывести заголовок таблицы с выравниванием
3. Составить список кортежей (название, рост)

4. Форматировать каждую строку с учётом нужной ширины поля

Подсказка: Сначала собираем данные, потом измеряем, затем печатаем.

Ответ:

- Составить список кортежей (название, рост)
- Определить максимальную длину названий растений
- Вывести заголовок таблицы с выравниванием
- Форматировать каждую строку с учётом нужной ширины поля

Шаг 14:

Миссия 13

Упорядочьте этапы, когда команда Grow a Garden добавляет новые растения и хочет обновить табличку:

1. Добавить новый элемент в список растений
2. Пересчитать максимальную длину названия и ростов для форматирования
3. Переформатировать все строки таблицы
4. Вывести обновлённую таблицу на экран

Подсказка: Добавление данных — до перерасчёта — до вывода.

Ответ:

- Добавить новый элемент в список растений
- Пересчитать максимальную длину названия и ростов для форматирования
- Переформатировать все строки таблицы
- Вывести обновлённую таблицу на экран

Шаг 15:

Миссия 14

Упорядочьте логический процесс, который Вика проходит, чтобы преобразовать рост, указанный в сантиметрах, в таблице, в метры с форматированием:

1. Разделить сантиметры на 100 для перевода в метры
2. Собрать все данные растений в список
3. Отформатировать число с двумя десятичными цифрами и добавить суффикс "м"
4. Вывести таблицу, где все значения в метрах

Подсказка: Сначала собираем, потом переводим, потом форматируем, потом печатаем.

Ответ:

- Собрать все данные растений в список
- Разделить сантиметры на 100 для перевода в метры
- Отформатировать число с двумя десятичными цифрами и добавить суффикс "м"
- Вывести таблицу, где все значения в метрах

Шаг 16:

Миссия 15

Упорядочьте шаги, необходимые Серёже, чтобы учесть, что названия растений и росты могут иметь разную длину, и табличка всё равно была ровной:

1. Найти самую длинную строку среди названий растений
2. Определить максимальную длину числа роста (в символах)

3. Составить шаблон форматирования с учётом обеих ширин
4. Вывести каждую строку через шаблон: название + рост

Подсказка: Учтите оба параметра — название и число — для выравнивания.

Ответ:

- Найти самую длинную строку среди названий растений
- Определить максимальную длину числа роста (в символах)
- Составить шаблон форматирования с учётом обеих ширин
- Вывести каждую строку через шаблон: название + рост

Урок 7 - Циклический прогноз — input внутри цикла

Шаг 01:

Миссия 1

Герой сажает семена и хочет спросить у игрока: сколько семян он посадит. Какой оператор позволяет вводить данные от пользователя в Python?

Подсказка: Представь, что Python задаёт вопрос игроку и ждёт ответ.

Ответ:

- **input()**
- print()
- for
- if

Шаг 02:

Миссия 2

Фермер Том хочет каждый день спрашивать у игрока погоду. Какой цикл подойдёт, если мы заранее знаем, что нужно 7 раз спросить?

Подсказка: Если известно количество повторений, используем цикл с диапазоном.

Ответ:

- **for**
- while
- if
- break

Шаг 03:

Миссия 3

Герой Алиса хочет, чтобы игрок вводил рост растения до тех пор, пока он не достигнет 10 см. Какой цикл лучше выбрать?

Подсказка: Когда не знаем количество повторений заранее, используем условный цикл.

Ответ:

- **while**
- for
- if
- range()

Шаг 04:

Миссия 4

Фермер Боб записывает каждый день количество поливов в цикле. Что нужно сделать с вводом `input()`, чтобы превратить его в число?

Подсказка: По умолчанию `input()` возвращает строку.

Ответ:

- `int(input())`
- `str(input())`
- `print(input())`
- `float(print())`

Шаг 05:

Миссия 5

Команда героев хочет 3 раза спросить у игрока любимый фрукт. Какой код будет правильным?

Подсказка: Используй цикл `for` с `range(3)`.

Ответ:

- `for i in range(3): fruit = input()`
- `while i in range(3): input()`
- `for fruit in 3: input()`
- `if range(3): input()`

Шаг 06:

Миссия 6

Сопоставь цикл и ситуацию в саду.

Подсказка: Подумай: `for` — когда известно количество повторов, `while` — когда ждём определённое условие.

Ответ:

- **for** — Игрок поливает грядку ровно 5 раз
- **while** — Игрок вводит рост растения, пока оно не достигнет 15 см

Шаг 07:

Миссия 7

Сопоставь функцию с её назначением в игре.

Подсказка: `input()` задаёт вопрос, `print()` отвечает игроку.

Ответ:

- **input()** — Игрок вводит число поливов
- **print()** — Программа выводит результат поливов

Шаг 08:

Миссия 8

Сопоставь элементы кода и их роль.

Подсказка: `range()` создаёт список чисел, `for` перебирает их.

Ответ:

- **for i in range(3)** — Цикл выполняется 3 раза
- **i** — Счётчик повторений

🔗 **Шаг 09:**

Миссия 9

Сопоставь описание и часть кода.

Подсказка: while продолжает работать, пока условие истинно.

Ответ:

- **while height < 10** — Цикл работает, пока рост меньше 10
- **height = int(input())** — Игрок вводит рост растения

🔗 **Шаг 10:**

Миссия 10

Сопоставь код и результат.

Подсказка: input() всегда возвращает строку, поэтому для чисел нужен int().

Ответ:

- **x = input()** — Игрок вводит строку
- **x = int(input())** — Игрок вводит число

🔗 **Шаг 11:**

Миссия 11

Расположи шаги программы: фермер хочет 3 раза спросить у игрока любимый фрукт.

Подсказка: Сначала цикл, потом ввод, потом вывод.

Ответ:

- **for i in range(3):**
- **fruit = input()**
- **print(fruit)**

🔗 **Шаг 12:**

Миссия 12

Расположи шаги: команда героев вводит количество семян каждый день (5 дней).

Подсказка: Используй цикл for с range(5).

Ответ:

- **for i in range(5):**
- **seeds = int(input())**
- **print(seeds)**

🔗 **Шаг 13:**

Миссия 13

Расположи шаги: Алиса проверяет погоду 7 дней подряд.

Подсказка: Используем цикл for и ввод внутри него.

Ответ:

- `for i in range(7):`
- `weather = input()`
- `print(weather)`

💡 **Шаг 14:**

Миссия 14

Расположи шаги: команда героев вводит количество овощей для каждого дня недели.

Подсказка: `range(7)` перебирает числа от 0 до 6.

Ответ:

- `for i in range(7):`
- `vegetables = int(input())`
- `print(vegetables)`

📖 **Урок 8 - Бот-поливальщик — мини-проект автоматизация полива**

💡 **Шаг 01:**

Миссия 1

Герой-садовник хочет создать бота, который спрашивает, сколько литров воды использовать для полива. Какой оператор в Python позволяет получать данные от игрока?

Подсказка: Подумайте, как бот может задать вопрос и получить ответ от игрока.

Ответ:

- `input()`
- `print()`
- `for`
- `while`

💡 **Шаг 02:**

Миссия 2

Бот должен полить каждую грядку по очереди 5 дней подряд. Какой цикл лучше использовать?

Подсказка: Если количество повторов заранее известно, выбираем цикл с диапазоном.

Ответ:

- `for`
- `while`
- `if`
- `break`

💡 **Шаг 03:**

Миссия 3

Бот должен спрашивать у игрока, полита ли грядка, пока игрок не введёт 'да'. Какой цикл лучше всего использовать?

Подсказка: Когда неизвестно заранее количество повторений, выбираем условный цикл.

Ответ:

- `while`

- for
- if
- range()

🔗 Шаг 04:

Миссия 4

Бот спрашивает количество воды, но `input()` возвращает строку. Как преобразовать её в число?

Подсказка: Нужно число, чтобы бот мог правильно поливать.

Ответ:

- `int(input())`
- `str(input())`
- `float(input())`
- `print(input())`

🔗 Шаг 05:

Миссия 5

Бот поливает 3 грядки. Какой код корректно выполняет три полива подряд?

Подсказка: Используй цикл `for` с `range(3)`.

Ответ:

- `for i in range(3): water = int(input())`
- `while i in range(3): input()`
- `for water in 3: input()`
- `if range(3): input()`

🔗 Шаг 06:

Миссия 6

Сопоставь цикл и ситуацию с ботом-поливальщиком.

Подсказка: `for` — заранее известное количество повторов, `while` — до выполнения условия.

Ответ:

- **for** — Бот поливает 5 грядок
- **while** — Бот спрашивает, полита ли грядка, пока игрок не введёт 'да'

🔗 Шаг 07:

Миссия 7

Сопоставь функцию и её роль в проекте бота.

Подсказка: `input()` спрашивает у игрока, `print()` отвечает.

Ответ:

- **input()** — Игрок вводит количество воды
- **print()** — Бот сообщает результат полива

🔗 Шаг 08:

Миссия 8

Сопоставь элементы кода и их значение в боте.

Подсказка: `range()` создаёт список чисел для цикла, `for` перебирает их.

Ответ:

- **for i in range(3)** — Цикл выполняется 3 раза
- **i** — Счётчик повторений

🔗 **Шаг 09:**

Миссия 9

Сопоставь код и его действие в проекте бота.

Подсказка: while продолжает выполняться, пока условие верно.

Ответ:

- **while watered != 'да'** — Бот спрашивает, полита ли грядка, пока игрок не введёт 'да'
- **watered = input()** — Игрок вводит ответ

🔗 **Шаг 10:**

Миссия 10

Расположи шаги бота: полив 3 грядок.

Подсказка: Сначала цикл, затем ввод, затем вывод.

Ответ:

- **for i in range(3):**
- **water = int(input())**
- **print(water)**

🔗 **Шаг 11:**

Миссия 11

Расположи шаги: бот поливает 5 грядок, собирает данные о поливе.

Подсказка: Используем for и ввод внутри него, затем вывод.

Ответ:

- **for i in range(5):**
- **water = int(input())**
- **print(water)**

🔗 **Шаг 12:**

Миссия 12

Расположи шаги: бот проверяет погоду 7 дней подряд.

Подсказка: Используем цикл for и input внутри него.

Ответ:

- **for i in range(7):**
- **weather = input()**
- **print(weather)**

Модуль 4 - Модуль 4 «Плодородные структуры»

Урок 1 - Ящик с семенами — списки (list) и индексация

Шаг 01:

Миссия 1

Герой нашёл ящик с семенами: ['морковь', 'помидор', 'огурец']. Как обратиться к первому элементу списка?

Подсказка: В Python индексация начинается с нуля.

Ответ:

- **seeds[0]**
- seeds[1]
- seeds.first()
- seeds.get(0)

Шаг 02:

Миссия 2

Герой хочет узнать, сколько семян в ящике ['морковь', 'помидор', 'огурец']?

Подсказка: Для подсчёта элементов списка есть встроенная функция.

Ответ:

- **len(seeds)**
- seeds.count()
- count(seeds)
- seeds.length

Шаг 03:

Миссия 3

Герой хочет заменить второй элемент ящика с семенами на 'баклажан'. Как это сделать?

Подсказка: Используйте индекс для изменения элемента списка.

Ответ:

- **seeds[1] = 'баклажан'**
- seeds.replace(1, 'баклажан')
- seeds.set(1, 'баклажан')
- seeds.insert(1, 'баклажан')

Шаг 04:

Миссия 4

Герой хочет добавить новое семя 'редис' в конец ящика. Какой метод списка использовать?

Подсказка: Для добавления элемента в конец есть специальный метод.

Ответ:

- **seeds.append('редис')**
- seeds.add('редис')
- seeds.insert('редис')
- seeds.push('редис')

Шаг 05:

Миссия 5

Герой хочет узнать, на какой позиции находится 'огурец' в ящике. Как это сделать?

Подсказка: Для поиска элемента в списке есть специальный метод.

Ответ:

- `seeds.index('огурец')`
- `seeds.find('огурец')`
- `index(seeds, 'огурец')`
- `seeds.loc('огурец')`

Шаг 06:

Миссия 6

Сопоставьте действие с правильным методом списка.

Подсказка: Каждый метод списка выполняет конкретную операцию.

Ответ:

- **Добавить элемент в конец** — `append()`
- **Найти позицию элемента** — `index()`
- **Посчитать количество элементов** — `len()`
- **Изменить элемент по индексу** — assignment через [индекс]

Шаг 07:

Миссия 7

Сопоставьте индекс и элемент в ящике ['морковь', 'баклажан', 'огурец'].

Подсказка: Вспомните нумерацию элементов списка.

Ответ:

- **0** — морковь
- **1** — баклажан
- **2** — огурец

Шаг 08:

Миссия 8

Сопоставьте метод списка с его действием.

Подсказка: Различные методы изменяют или возвращают данные по-разному.

Ответ:

- **`append()`** — Добавляет элемент в конец списка
- **`len()`** — Возвращает количество элементов в списке
- **`index()`** — Возвращает индекс первого вхождения элемента

Шаг 09:

Миссия 9

Расположи действия героя по работе с ящиком: добавление 'редис', замена второго элемента на 'баклажан', узнать индекс 'огурец'.

Подсказка: Сначала добавляем, затем заменяем, затем ищем индекс.

Ответ:

- `seeds.append('редис')`
- `seeds[1] = 'баклажан'`
- `seeds.index('огурец')`

 **Шаг 10:**

Миссия 10

Расположи элементы ящика ['морковь', 'помидор', 'огурец'] по порядку: вывести первый, заменить второй, добавить 'редис'.

Подсказка: Сначала обращение по индексу, затем присвоение, затем добавление.

Ответ:

- `seeds[0]`
- `seeds[1] = 'баклажан'`
- `seeds.append('редис')`

Урок 2 - Урожай по дням — перебор списков в цикле

 **Шаг 01:**

Миссия 1

Герой ведёт учёт урожая за неделю: ['морковь', 'помидор', 'огурец']. Как перебрать каждый элемент с помощью цикла `for`?

Подсказка: Используй конструкцию `for item in список`.

Ответ:

- **`for vegetable in harvest:`**
- `for i = 0; i < len(harvest); i++:`
- `foreach vegetable in harvest:`
- `while vegetable in harvest:`

 **Шаг 02:**

Миссия 2

Герой хочет вывести каждый день урожай из списка ['морковь', 'помидор', 'огурец'] на экран. Какой код подойдёт?

Подсказка: Используй цикл и функцию `print` внутри него.

Ответ:

- **`for vegetable in harvest: print(vegetable)`**
- `print(harvest)`
- `for i in range(harvest): print(i)`
- `while harvest: print(harvest)`

 **Шаг 03:**

Миссия 3

Герой хочет узнать длину каждого названия овоща в списке ['морковь', 'помидор', 'огурец'] и вывести её. Какой код использовать?

Подсказка: Используй функцию `len()` внутри цикла.

Ответ:

- **for vegetable in harvest: print(len(vegetable))**
- `print(len(harvest))`
- `for i in harvest: print(harvest[i])`
- `for vegetable in harvest: print(vegetable[i])`

 **Шаг 04:**

Миссия 4

Герой хочет создать суммарный список урожая за неделю: [3, 5, 2]. Как пройти по нему и подсчитать общее количество овощей?

Подсказка: Используй цикл `for` и переменную-счётчик.

Ответ:

- **total = 0 for count in week_harvest: total += count**
- `total = sum(week_harvest)`
- `for count in week_harvest: total = count`
- `total = 0 while week_harvest: total += week_harvest`

 **Шаг 05:**

Миссия 5

Герой хочет вывести каждый овощ и его количество из списка [('морковь', 3), ('помидор', 5), ('огурец', 2)]. Какой код правильный?

Подсказка: Используй распаковку кортежа в цикле.

Ответ:

- **for vegetable, count in harvest: print(vegetable, count)**
- `for i in harvest: print(i)`
- `for vegetable in harvest: print(harvest[vegetable])`
- `print(harvest[0], harvest[1], harvest[2])`

 **Шаг 06:**

Миссия 6

Сопоставьте термин и его описание в контексте перебора списка:

Подсказка: Вспомните, что делает цикл `for` с элементами списка.

Ответ:

- **Элемент списка** — Один объект, находящийся в списке
- **Цикл for** — Проходит по каждому элементу списка
- **len()** — Возвращает количество элементов в списке

 **Шаг 07:**

Миссия 7

Сопоставьте переменную и её роль при подсчёте урожая:

Подсказка: Одна переменная хранит сумму, другая — текущий элемент.

Ответ:

- **total** — Сумма всех овощей
- **count** — Количество текущего овоща в цикле

🔗 Шаг 08:

Миссия 8

Сопоставьте пример кода с его действием:

Подсказка: Обратите внимание, что внутри цикла происходит действие с каждым элементом.

Ответ:

- **for vegetable in harvest: print(vegetable)** — Вывод каждого элемента списка на экран
- **for vegetable, count in harvest: print(vegetable, count)** — Вывод элемента и количества

🔗 Шаг 09:

Миссия 9

Расположите действия по перебору списка урожая [('морковь', 3), ('помидор', 5), ('огурец', 2)] для вывода имени и количества овощей:

Подсказка: Сначала начинаем цикл, потом распаковываем кортеж, затем выводим данные.

Ответ:

- for vegetable, count in harvest:
- print(vegetable, count)

🔗 Шаг 10:

Миссия 10

Сегодня герои Grow a Garden собрали урожай моркови. Какой способ перебора списка поможет узнать, сколько морковок собрано за каждый день?

Подсказка: Подумайте о том, как последовательно пройти по элементам списка.

Ответ:

- **Цикл for, перебирающий элементы списка**
- Случайный выбор элемента списка
- Прямой доступ только к первому элементу

🔗 Шаг 11:

Миссия 11

Герои заметили, что некоторые грядки не политы. Какой подход поможет проверить состояние каждой грядки в списке?

Подсказка: Нужно проверить все элементы, а не только первый.

Ответ:

- **Использовать цикл for для перебора всех грядок**
- Проверить случайную грядку
- Пропустить проверку и полить все грядки одинаково

🔗 Шаг 12:

Миссия 12

Для учета урожая нужно суммировать количество яблок за каждый день. Какой метод лучше использовать?

Подсказка: Суммировать удобно, если проходить по каждому элементу списка.

Ответ:

- **Использовать цикл for для сложения всех элементов**
- Взять первый элемент списка и умножить на количество дней
- Случайным образом выбрать день и считать только его урожай

🔮 **Шаг 13:**

Миссия 13

Герои решили узнать, какие растения требуют полива чаще всего. Какой способ перебора списка подходит для этого?

Подсказка: Нужно проверить каждый элемент и подсчитать, сколько раз он встречается.

Ответ:

- **Цикл for с подсчетом количества поливов**
- Выбрать случайное растение и проверить только его
- Пропустить подсчет и поливать все растения одинаково

🔮 **Шаг 14:**

Миссия 14

Чтобы узнать, какие дни были самыми урожайными, герои хотят просмотреть все значения списка с урожаем. Что лучше использовать?

Подсказка: Думайте о том, как получить доступ к каждому дню по очереди.

Ответ:

- **Перебрать список циклом for**
- Взять только первый и последний день
- Переписать список вручную

🔮 **Шаг 15:**

Миссия 15

Формулировка: Сопоставьте день недели с количеством собранного урожая.

Дни:

Ответ:

- Понедельник
- Среда
- Пятница
- Суббота

Урожай:

1. 10 морковок
2. 7 помидоров
3. 15 яблок
4. 12 клубник

🔮 **Подсказка:**

Подумайте, что герои собирали в разные дни и какие овощи/фрукты были в приоритете.

- **Понедельник** — 10 морковок
- **Среда** — 7 помидоров

- **Пятница** — 15 яблок
- **Суббота** — 12 клубник

Шаг 16:

Миссия 16

Формулировка: Сопоставьте тип растения с количеством поливов за неделю.

Растения:

Ответ:

- Морковь
- Помидоры
- Яблоки
- Клубника

Поливы:

1. 3
2. 5
3. 4
4. 2

Подсказка:

Учтите, какие растения нуждаются в более частом поливе по сюжету игры.

- **Морковь** — 3
- **Помидоры** — 5
- **Яблоки** — 4
- **Клубника** — 2

Урок 3 - Инвентарь урожая — методы списков (append, remove, len)

Шаг 01:

Миссия 1

Герои Grow a Garden собрали первые яблоки и хотят сохранить их в списке inventory . Какой метод они должны использовать?

Подсказка: Вспомни, что чтобы добавить предмет в список, используют особый метод.

Ответ:

- **inventory.append('яблоко')**
- inventory.remove('яблоко')
- inventory.len()
- inventory.add('яблоко')

Шаг 02:

Миссия 2

Лиза случайно положила гнилую грушу в инвентарь и хочет её убрать. Какой метод списка ей поможет?

Подсказка: Удалить предмет можно с помощью метода, противоположного append() .

Ответ:

- inventory.remove('груша')

- **inventory.remove('груша')**
- `inventory.append('груша')`
- `inventory.delete('груша')`

🔮 Шаг 03:

Миссия 3

Команда хочет узнать, сколько всего овощей уже в корзине. Какой встроенный метод поможет?

Подсказка: Это не метод списка, а встроенная функция, которая возвращает число элементов.

Ответ:

- **len(inventory)**
- `inventory.size()`
- `count(inventory)`
- `inventory.length()`

🔮 Шаг 04:

Миссия 4

Андрей добавил морковь в инвентарь, а затем ещё одну. Что произойдёт?

Подсказка: Подумай: список позволяет хранить одинаковые элементы или нет?

Ответ:

- Список выдаст ошибку
- Старая морковь заменится новой
- **В списке будет две морковки**
- Элемент удалится автоматически

🔮 Шаг 05:

Миссия 5

У Даши в корзине список ['яблоко', 'груша', 'слива']. Какой код удалит 'груша' ?

Подсказка: Вспомни метод для удаления конкретного значения.

Ответ:

- `inventory.pop(1)`
- **inventory.remove('груша')**
- `del inventory`
- `inventory.clear()`

🔮 Шаг 06:

Миссия 6

Сопоставь методы списка с их действиями, чтобы помочь героям правильно сортировать урожай.

Подсказка: Вспомни: одни методы добавляют, другие удаляют, а есть ещё встроенная функция для подсчёта.

Ответ:

- **append** — Добавляет элемент в конец списка
- **remove** — Удаляет элемент по значению
- **len** — Считает количество элементов в списке
- **clear** — Очищает весь список

🔗 Шаг 07:

Миссия 7

Соедини предметы инвентаря с правильными действиями.

Подсказка: Обрати внимание, что `append()` и `remove()` требуют аргумента — элемента списка.

Ответ:

- **`inventory.append('тыква')`** — Добавить тыкву в список
- **`inventory.remove('яблоко')`** — Удалить яблоко из списка
- **`len(inventory)`** — Посчитать количество элементов

🔗 Шаг 08:

Миссия 8

В команде спор: что делают разные команды с инвентарём? Соедини код и результат.

Подсказка: Представь, что у тебя список `['морковь', 'яблоко']`.

Ответ:

- **`inventory.append('груша')`** — `['морковь', 'яблоко', 'груша']`
- **`inventory.remove('морковь')`** — `['яблоко']`
- **`len(inventory)`** — `2`

🔗 Шаг 09:

Миссия 9

Подбери коды методов к их логическим описаниям.

Подсказка: Вспомни: `append()` всегда добавляет, `remove()` — удаляет, а `len()` — считает.

Ответ:

- **Добавить элемент** — `append()`
- **Удалить элемент по значению** — `remove()`
- **Посчитать элементы** — `len()`

🔗 Шаг 10:

Миссия 10

У команды три команды кода. Соедини их с тем, что произойдёт.

Подсказка: Обрати внимание, что `clear()` действует на весь список.

Ответ:

- **`inventory.clear()`** — Инвентарь станет пустым
- **`inventory.append('арбуз')`** — Добавится арбуз
- **`len(inventory)`** — Покажет число элементов

🔗 Шаг 11:

Миссия 11

Расположи действия с инвентарём в правильной последовательности, чтобы герой успешно собрал урожай.

Подсказка: Сначала список создают, затем добавляют, потом убирают, и наконец считают.

Ответ:

- Создать пустой список

- `append('яблоко')`
- `remove('яблоко')`
- `len(inventory)`

🔮 Шаг 12:

Миссия 12

Даша собрала урожай и записала действия. В каком порядке они должны выполняться?

Подсказка: Сначала добавляем, потом можно считать, и только потом очищать.

Ответ:

- `inventory.append('слива')`
- `len(inventory)`
- `inventory.clear()`

🔮 Шаг 13:

Миссия 13

Расположи шаги по работе с корзиной героя.

Подсказка: Подумай, что делают `append()`, `remove()` и `len()` в логике.

Ответ:

- `append('тыква')`
- `append('яблоко')`
- `remove('яблоко')`
- `len(inventory)`

🔮 Шаг 14:

Миссия 14

Герои решили провести уборку в инвентаре. В каком порядке идут действия?

Подсказка: Сначала добавляем, потом удаляем, и только после этого очищаем.

Ответ:

- `append('арбуз')`
- `remove('арбуз')`
- `clear()`

🔮 Шаг 15:

Миссия 15

Последовательность действий для сбора и подсчёта овощей.

Подсказка: Сначала нужно наполнить корзину, а в конце — посчитать.

Ответ:

- `append('морковь')`
- `append('картофель')`
- `len(inventory)`

🔮 Шаг 16:

Миссия 16

В команде решили сначала собрать, потом убрать лишнее и подсчитать остаток. Расположи шаги правильно.

Подсказка: Убираем после того, как добавили.

Ответ:

- `append('яблоко')`
- `append('груша')`
- `remove('груша')`
- `len(inventory)`

Урок 4 - Ярлыки на ящиках — словари (dict) и пары ключ-значение

Шаг 01:

Миссия 1

Герои Grow a Garden подписали первый ящик с морковью. Как правильно создать словарь с ключом 'морковь' и значением 10?

Подсказка: В словарях ключи и значения разделяются двоеточием, а пары заключаются в фигурные скобки.

Ответ:

- **`inventory = {'морковь': 10}`**
- `inventory = ['морковь': 10]`
- `inventory = ('морковь', 10)`
- `inventory = {'морковь', 10}`

Шаг 02:

Миссия 2

Лиза хочет добавить новый ящик с яблоками в словарь. Какой синтаксис верный?

Подсказка: Чтобы добавить новую пару ключ-значение, укажи имя словаря и в квадратных скобках ключ.

Ответ:

- **`inventory['яблоки'] = 15`**
- `inventory.add('яблоки', 15)`
- `inventory.append('яблоки':15)`
- `inventory.push('яблоки', 15)`

Шаг 03:

Миссия 3

Андрей проверяет, сколько морковок в ящике. Как правильно узнать значение по ключу?

Подсказка: Чтобы получить значение из словаря, используй имя словаря и ключ в квадратных скобках.

Ответ:

- **`inventory['морковь']`**
- `inventory.getvalue('морковь')`
- `inventory.value('морковь')`
- `inventory.morковь`

🔮 Шаг 04:

Миссия 4

Команда хочет обновить количество яблок до 20. Как это сделать?

Подсказка: Используй квадратные скобки и присвоение нового значения.

Ответ:

- **inventory['яблоки'] = 20**
- `inventory.update('яблоки', 20)`
- `inventory.append('яблоки', 20)`
- `inventory['яблоки'].add(20)`

🔮 Шаг 05:

Миссия 5

Сопоставь методы словаря с их действием.

Подсказка: Один метод добавляет, другой удаляет, третий позволяет узнать значение.

Ответ:

- **inventory['ключ'] = значение** — Добавляет или обновляет пару ключ-значение
- **inventory.pop('ключ')** — Удаляет пару по ключу
- **inventory['ключ']** — Получает значение по ключу

🔮 Шаг 06:

Миссия 6

Соедини ключи с их значениями в словаре инвентаря.

Подсказка: Подумай, что хранит каждый ключ.

Ответ:

- **морковь** — 10
- **яблоки** — 20
- **тыква** — 5

🔮 Шаг 07:

Миссия 7

Соедини команды с их логическим результатом в словаре инвентаря.

Подсказка: Обновление, удаление, доступ по ключу — три основных действия.

Ответ:

- **inventory['груша'] = 12** — Добавление или изменение значения ключа 'груша'
- **inventory.pop('тыква')** — Удаление пары ключ-значение с ключом 'тыква'
- **inventory['яблоки']** — Получение значения ключа 'яблоки'

🔮 Шаг 08:

Миссия 8

Соедини ключи с верным действием в словаре инвентаря.

Подсказка: Ключ всегда уникален; для обновления просто присвоить новое значение.

Ответ:

- **яблоки** — Изменение количества или добавление ключа

- **морковь** — Удаление ключа с помощью pop
- **тыква** — Получение текущего количества

🔮 Шаг 09:

Миссия 8

В саду героя Клэр появилась задача: найти количество каждого типа растений в ящике. Как лучше всего хранить такую информацию в Python?

Подсказка: Подумайте, какой тип данных позволяет связать название растения с его количеством.

Ответ:

- **Использовать словарь с ключами** — названия растений, значениями — количество
- Использовать список только с названиями растений
- Использовать строку с названиями растений через запятую
- Использовать множество (set) с названиями растений

🔮 Шаг 10:

Миссия 9

Макс хочет добавить в словарь новые растения и их количество. Какой способ подходит для добавления пары ключ-значение в словарь?

Подсказка: Подумайте, как в Python добавляют элементы в словарь по ключу.

Ответ:

- **dict_name[ключ] = значение**
- dict_name.append(ключ, значение)
- dict_name.add(ключ, значение)

🔮 Шаг 11:

Миссия 10

Даша хочет узнать, есть ли в её словаре растение "петуния". Как проверить наличие ключа в словаре?

Подсказка: Для проверки ключа подходит специальный оператор в Python.

Ответ:

- **ключ in dict_name**
- ключ.exists(dict_name)
- dict_name.has_key(ключ)

🔮 Шаг 12:

Миссия 11

Команда садоводов запуталась в терминах о словарях. Помогите сопоставить понятия со значениями:

Ответ:

- Ключ
- Значение
- Пара ключ-значение
- Словарь (dict)

Подсказка: Вспомните, что ключ — это уникальный идентификатор, а значение — данные, которые он хранит.

- **Ключ** — Уникальный идентификатор элемента в словаре
- **Значение** — Данные, связанные с ключом
- **Пара ключ-значение** — Связка ключа и значения в словаре
- **Словарь (dict)** — Структура данных, хранящая пары ключ-значение

🔮 Шаг 13:

Миссия 12

Садовник Артур сравнивает разные словари. Помогите ему сопоставить операции с результатами:

Ответ:

- `dict.keys()`
- `dict.values()`
- `dict.items()`
- `dict.get(key)`

Подсказка: Подумайте, что возвращает каждый метод.

- **`dict.keys()`** — Список всех ключей словаря
- **`dict.values()`** — Список всех значений словаря
- **`dict.items()`** — Список всех пар ключ-значение
- **`dict.get(key)`** — Значение по ключу, если ключ есть в словаре

🔮 Шаг 14:

Миссия 13

В словаре Кати количество моркови хранится под ключом "carrot". Как получить это значение?

Подсказка: Вспомните синтаксис обращения к значению по ключу.

Ответ:

- **`dict_name['carrot']`**
- `dict_name.carrot`
- `dict_name.get('carrot', None)`

🔮 Шаг 15:

Миссия 14

Команда решает, какие ключи и значения могут быть в словаре для ящика с растениями. Сопоставьте ключи и подходящие типы значений:

Ответ:

- "name"
- "quantity"
- "harvested"
- "price_per_unit"

Подсказка: Подумайте, какой тип данных лучше всего подходит под каждое описание.

- **name** — строка (str)
- **quantity** — целое число (int)
- **harvested** — булево значение (bool)

- **price_per_unit** — число с плавающей точкой (float)

Шаг 16:

Миссия 15

Расположите действия по созданию словаря с информацией о растении в правильном порядке:

1. Добавить ключ "name" с названием растения
2. Создать пустой словарь
3. Добавить ключ "quantity" с количеством
4. Добавить ключ "harvested" с состоянием урожая

Подсказка: Помните, что сначала нужно инициализировать словарь, а потом добавлять пары ключ-значение.

Ответ:

- Создать пустой словарь
- Добавить ключ "name" с названием растения
- Добавить ключ "quantity" с количеством
- Добавить ключ "harvested" с состоянием урожая

Урок 5 - Проверка хранилища — поиск по ключам и значениям

Шаг 01:

Миссия 1

Задача: Сопоставьте методы словаря с их описанием.

Методы:

Ответ:

- keys()
- values()
- items()
- pop()

Описание:

1. Возвращает пары (ключ, значение)
2. Удаляет элемент по ключу и возвращает его значение
3. Возвращает список всех ключей
4. Возвращает список всех значений

Подсказка: Обратите внимание, что items() возвращает пары, а pop() удаляет.

- **keys()** — Возвращает список всех ключей
- **values()** — Возвращает список всех значений
- **items()** — Возвращает пары (ключ, значение)
- **pop()** — Удаляет элемент по ключу и возвращает его значение

Шаг 02:

Миссия 2

Задача: Сопоставьте сценарий с правильным методом для работы со словарём.

Ответ:

- Проверить наличие ключа
- Получить количество элементов
- Удалить элемент
- Добавить или обновить значение по ключу

1. pop()
2. len()
3. in
4. assignment (присвоение через [])

Подсказка: Сравните операции с методами и оператором.

- **Проверить наличие ключа** — in
- **Получить количество элементов** — len()
- **Удалить элемент** — pop()
- **Добавить или обновить значение по ключу** — assignment (присвоение через [])

 Шаг 03:

Миссия 3

Задача: Сопоставьте ключи словаря с типами их значений в нашем саду.

Ответ:

- "яблоки"
 - "цветы"
 - "грядки"
 - "растения"
1. число (int)
 2. список (list)
 3. словарь (dict)
 4. строка (str)

Подсказка: Подумайте, какие данные хранятся под каждым ключом.

- **яблоки** — число (int)
- **цветы** — список (list)
- **грядки** — словарь (dict)
- **растения** — строка (str)

 Шаг 04:

Миссия 4

Задача: Сопоставьте команды для поиска в словаре с их результатами при словаре {'помидор': 5, 'огурец': 8, 'тыква': 2}.

Ответ:

- "помидор" in garden
- 10 in garden.values()
- garden.get('тыква')
- garden.get('баклажан', 0)

1. True

2. False
3. 2
4. 0

Подсказка: Обратите внимание на работу in и get.

- **"помидор" in garden** — True
- **10 in garden.values()** — False
- **garden.get('тыква')** — 2
- **garden.get('баклажан', 0)** — 0

Шаг 05:

Миссия 5

Задача: Сопоставьте методы словаря с их результатом при работе со словарём:

Ответ:

- items()
 - keys()
 - values()
 - pop('огурец')
1. Удаляет и возвращает пару ('огурец', 8)
 2. Возвращает пары (ключ, значение)
 3. Возвращает список ключей
 4. Возвращает список значений

Подсказка: Внимательно посмотрите, что возвращают методы и что делает pop.

- **pop('огурец')** — Удаляет и возвращает пару ('огурец', 8)
- **items()** — Возвращает пары (ключ, значение)
- **keys()** — Возвращает список ключей
- **values()** — Возвращает список значений

Шаг 06:

Миссия 6

Вопрос: Расположите шаги проверки, есть ли ключ в словаре, в правильном порядке:

1. Использовать оператор "in" для проверки
2. Обратиться к значению по ключу, если ключ найден
3. Обработать ситуацию, если ключа нет

Подсказка:

Сначала проверяем ключ, потом используем его, и всегда готовимся к отсутствию.

Ответ:

- Использовать оператор "in" для проверки
- Обратиться к значению по ключу, если ключ найден
- Обработать ситуацию, если ключа нет

Шаг 07:

Миссия 7

Вопрос: Расположите этапы безопасного удаления элемента из словаря по ключу:

1. Вызвать метод pop с ключом и значением по умолчанию
2. Проверить, существует ли ключ (необязательно при pop с дефолтом)
3. Обработать возвращаемое значение или None

Подсказка:

Используйте pop с дефолтом, чтобы избежать ошибки, если ключа нет.

Ответ:

- Проверить, существует ли ключ (необязательно при pop с дефолтом)
- Вызвать метод pop с ключом и значением по умолчанию
- Обработать возвращаемое значение или None

 Шаг 08:

Миссия 8

Вопрос: Расположите действия для обновления значения по ключу в словаре:

1. Проверить, существует ли ключ
2. Если ключ есть, изменить значение
3. Если ключа нет, добавить новую пару ключ-значение

Подсказка:

Подумайте, как избежать ошибки при обновлении.

Ответ:

- Проверить, существует ли ключ
- Если ключ есть, изменить значение
- Если ключа нет, добавить новую пару ключ-значение

 Шаг 09:

Миссия 8

Герой Grow a Garden, Том, хранит данные о растениях в словаре. Как проверить, есть ли ключ "морковь" в его словаре?

Подсказка: В Python для проверки ключа в словаре можно использовать оператор in.

Ответ:

- `"морковь" in plants_dict`
- `plants_dict.contains("морковь")`
- `plants_dict.has_key("морковь")`

 Шаг 10:

Миссия 9

Аня хранит урожай в словаре. Как узнать, сколько различных ключей (растений) в словаре?

Подсказка: Используйте встроенную функцию для получения количества элементов в словаре.

Ответ:

- `len(harvest_dict)`
- `harvest_dict.count()`
- `harvest_dict.size()`
- `harvest_dict.length`

🔗 Шаг 11:

Миссия 10

Иван ищет, есть ли значение "спелый" среди значений словаря с плодами. Как проверить это правильно?

Подсказка: Для поиска по значениям словаря можно использовать метод `values()` и оператор `in`.

Ответ:

- `"спелый" in fruits_dict.values()`
- `fruits_dict.has_value("спелый")`
- `fruits_dict.contains_value("спелый")`
- `"спелый" in fruits_dict.keys()`

🔗 Шаг 12:

Миссия 11

Старший садовник Лена хочет добавить новый плод в словарь только если ключа ещё нет. Как лучше проверить это?

Подсказка: Используйте оператор `in` для проверки наличия ключа до добавления.

Ответ:

- `if "яблоко" not in garden_dict: garden_dict["яблоко"] = "зрелый"`
- `if garden_dict["яблоко"] == None: garden_dict["яблоко"] = "зрелый"`
- `if garden_dict.has_key("яблоко"): garden_dict["яблоко"] = "зрелый"`

🔗 Шаг 13:

Миссия 12

Сопоставьте методы и операции словаря с их описаниями.

Методы и операции:

Ответ:

- `in`
- `values()`
- `keys()`
- `len()`

Описание:

1. Возвращает все ключи словаря
2. Проверяет наличие ключа в словаре
3. Возвращает количество элементов в словаре
4. Возвращает все значения словаря

Подсказка: Помните, что ключи и значения — разные части словаря.

- `in` — Проверяет наличие ключа в словаре
- `values()` — Возвращает все значения словаря
- `keys()` — Возвращает все ключи словаря
- `len()` — Возвращает количество элементов в словаре

🔗 Шаг 14:

Миссия 13

Сопоставьте ситуацию с правильным способом проверки данных в словаре.

Ситуации:

Ответ:

- Проверить, есть ли ключ "груша"
- Узнать все значения словаря
- Проверить, есть ли значение "зрелый"
- Получить количество пар ключ-значение

Способы:

1. `len(dict)`
2. `"груша" in dict`
3. `dict.values()`
4. `"зрелый" in dict.values()`

Подсказка: Ключи проверяются оператором `in`, значения через метод `values()`.

- **Проверить, есть ли ключ "груша"** — `"груша" in dict`
- **Узнать все значения словаря** — `dict.values()`
- **Проверить, есть ли значение "зрелый"** — `"зрелый" in dict.values()`
- **Получить количество пар ключ-значение** — `len(dict)`

Шаг 15:

Миссия 14

Сопоставьте ошибки с причинами, почему они возникают при проверке в словаре.

Ошибки:

Ответ:

- `KeyError`
- Неверное использование оператора `in` для значений
- Использование несуществующего метода

Причины:

1. Попытка доступа к ключу, которого нет в словаре
2. Проверка значения через `in` без метода `values()`
3. Метода не существует у объекта словаря

Подсказка: Проверьте, какой объект и операция применяются в вашем коде.

- **KeyError** — Попытка доступа к ключу, которого нет в словаре
- **Неверное использование оператора `in` для значений** — Проверка значения через `in` без метода `values()`
- **Использование несуществующего метода** — Метода не существует у объекта словаря

Шаг 16:

Миссия 15

Расположите этапы проверки, есть ли ключ в словаре и получение значения, если он есть:

1. Использовать оператор `in` для проверки ключа
2. Если ключ есть, получить значение через `dict[key]`
3. **Если ключа нет** — обработать ситуацию

Подсказка: Сначала проверяем, потом обращаемся, чтобы избежать ошибок.

Ответ:

- Использовать оператор `in` для проверки ключа
- Если ключ есть, получить значение через `dict[key]`
- **Если ключа нет** — обработать ситуацию

Урок 6 - Сортировка семян — функции `min`, `max`, `sum`, `sorted`

Шаг 01:

Миссия 1

Сопоставьте функции Python с их описанием, чтобы помочь Лизе понять, какую функцию использовать для своих семян.

Ответ:

- `min`
 - `max`
 - `sum`
 - `sorted`
1. Возвращает минимальное значение из списка
 2. Возвращает максимальное значение из списка
 3. Подсчитывает сумму всех элементов списка
 4. Возвращает отсортированный список

Подсказка: Подумайте, какая функция подходит для поиска минимального, максимального значения, суммы и упорядочивания.

- **`min`** — Возвращает минимальное значение из списка
- **`max`** — Возвращает максимальное значение из списка
- **`sum`** — Подсчитывает сумму всех элементов списка
- **`sorted`** — Возвращает отсортированный список

Шаг 02:

Миссия 2

Команда Grow a Garden хочет понять, какую функцию использовать для сортировки своих семян от самого лёгкого к самому тяжёлому. Помогите сопоставить действие и функцию.

Ответ:

- Подсчитать общую массу семян
 - Найти самое лёгкое семя
 - Отсортировать семена по весу
 - Найти самое тяжёлое семя
1. `sum`
 2. `min`
 3. `sorted`
 4. `max`

Подсказка: Подумайте, какая функция помогает подсчитать сумму, какая — найти минимальное/максимальное, а какая — отсортировать.

- **Подсчитать общую массу семян** — `sum`

- **Найти самое лёгкое семя** — min
- **Отсортировать семена по весу** — sorted
- **Найти самое тяжёлое семя** — max

🧭 Шаг 03:

Миссия 3

Сопоставьте результаты функций Python с примерами списка семян весом в граммах:

Список семян: [2, 5, 1, 8, 3]

Ответ:

- min
- max
- sum
- sorted

1. 1
2. 8
3. 19
4. [1, 2, 3, 5, 8]

Подсказка: Вспомните, что возвращает каждая функция, используя числа из списка.

- **min** — 1
- **max** — 8
- **sum** — 19
- **sorted** — [1, 2, 3, 5, 8]

🧭 Шаг 04:

Миссия 4

Помогите Тиме расположить шаги применения функции sorted() к списку семян от лёгких к тяжёлым в правильном порядке:

1. Передать список семян в функцию sorted()
2. Получить отсортированный список от лёгких к тяжёлым
3. Определить список семян, который нужно отсортировать
4. Использовать результат для дальнейшей работы в саду

Подсказка: Подумайте, что сначала выбирается, потом обрабатывается и результат используется.

Ответ:

- Определить список семян, который нужно отсортировать
- Передать список семян в функцию sorted()
- Получить отсортированный список от лёгких к тяжёлым
- Использовать результат для дальнейшей работы в саду

🧭 Шаг 05:

Миссия 5

Команда обнаружила список с весами семян: [15, 3, 9, 22, 7]. Расположите операции по нахождению максимального веса семян в правильном порядке:

1. Передать список в функцию max()

2. Определить список весов семян
3. Получить максимальный вес
4. Использовать максимальный вес для выбора самого крупного семени

Подсказка: Подумайте, с чего начинается работа с данными, и как использовать функцию для поиска максимума.

Ответ:

- Определить список весов семян
- Передать список в функцию `max()`
- Получить максимальный вес
- Использовать максимальный вес для выбора самого крупного семени

Шаг 06:

Миссия 6

Ваша задача — помочь Миле упорядочить её список семян по возрастанию веса: [6, 12, 4, 9, 1]. Расположите операции в правильном порядке, чтобы получить отсортированный список.

1. Использовать функцию `sorted()` для сортировки списка
2. Определить список семян
3. Получить отсортированный список
4. Использовать отсортированный список для посадки

Подсказка: Сначала выбираем, потом сортируем, затем применяем результат.

Ответ:

- Определить список семян
- Использовать функцию `sorted()` для сортировки списка
- Получить отсортированный список
- Использовать отсортированный список для посадки

Шаг 07:

Миссия 7

Садовник Никита хочет понять, как правильно использовать функцию `sum()` для подсчёта общего веса семян [2, 8, 5, 3, 7]. Помогите расположить шаги в логическом порядке.

1. Определить список весов семян
2. Передать список в функцию `sum()`
3. Получить сумму весов
4. Использовать сумму для планирования удобрений

Подсказка: Сначала указываем данные, потом считаем, после — используем результат.

Ответ:

- Определить список весов семян
- Передать список в функцию `sum()`
- Получить сумму весов
- Использовать сумму для планирования удобрений

Шаг 08:

Миссия 7

Помоги фермеру Лизе выбрать наименьшее количество семян для посадки, чтобы не тратить лишние ресурсы. У неё есть список семян по количеству: [5, 3, 9, 1, 6]. Какую функцию Python лучше использовать, чтобы быстро найти минимальное число?

Подсказка: Ищем функцию, которая возвращает самое маленькое значение из списка.

Ответ:

- **min()**
- max()
- sum()
- sorted()

Шаг 09:

Миссия 8

Робот-садовник Макс должен определить, сколько всего семян у него есть в запасе: [4, 7, 2, 8]. Какая функция поможет ему быстро посчитать сумму?

Подсказка: Функция, которая складывает все элементы списка.

Ответ:

- **sum()**
- min()
- sorted()
- max()

Шаг 10:

Миссия 9

Аня сортирует свои семена по размеру, чтобы быстрее найти самые большие. Какую функцию Python ей следует использовать?

Подсказка: Функция, которая упорядочивает элементы списка по возрастанию.

Ответ:

- **sorted()**
- sum()
- max()
- min()

Шаг 11:

Миссия 10

Фермер Вова хочет найти самый большой мешок с семенами из списка: [12, 15, 8, 20, 5]. Какую функцию использовать?

Подсказка: Функция, которая возвращает максимальное значение.

Ответ:

- **max()**
- min()
- sum()
- sorted()

Шаг 12:

Миссия 11

Команда Grow a Garden решила разобраться с функциями, которые помогают работать с семенами. Сопоставьте функцию с её описанием.

Функции:

Ответ:

- `min()`
- `max()`
- `sum()`
- `sorted()`

Описание:

1. Возвращает сумму всех элементов
2. Возвращает минимальный элемент
3. Сортирует список по возрастанию
4. Возвращает максимальный элемент

Подсказка: Подумайте, какая функция что делает, чтобы помочь садоводам.

- **`min()`** — Возвращает минимальный элемент
- **`max()`** — Возвращает максимальный элемент
- **`sum()`** — Возвращает сумму всех элементов
- **`sorted()`** — Сортирует список по возрастанию

Шаг 13:

Миссия 12

Клэр собирает урожай и хочет сопоставить функции с примерами их использования:

Функции:

Ответ:

- `min()`
- `max()`
- `sum()`
- `sorted()`

Примеры:

1. `sum([10, 20, 30]) = 60`
2. `max([3, 6, 9]) = 9`
3. `sorted([5, 1, 7]) = [1, 5, 7]`
4. `min([8, 4, 2]) = 2`

Подсказка: Сопоставьте функции с правильными результатами.

- **`min()`** — `min([8, 4, 2]) = 2`
- **`max()`** — `max([3, 6, 9]) = 9`
- **`sum()`** — `sum([10, 20, 30]) = 60`
- **`sorted()`** — `sorted([5, 1, 7]) = [1, 5, 7]`

Шаг 14:

Миссия 13

Фермерша Мира хочет организовать свои семена по возрастанию количества в мешках: [9, 3, 12, 6]. Помогите ей расположить действия в правильном порядке для сортировки семян.

1. Вызвать функцию `sorted()` с списком семян
2. Получить новый отсортированный список
3. Использовать отсортированный список для планирования посева
4. Вывести результат на экран

Подсказка: Подумайте, что происходит сначала, а что потом при сортировке.

Ответ:

- Вызвать функцию `sorted()` с списком семян
- Получить новый отсортированный список
- Использовать отсортированный список для планирования посева
- Вывести результат на экран

Шаг 15:

Миссия 14

Робот-садовник Бенджи сортирует мешки с семенами по размеру и хочет найти самый маленький и самый большой мешок. Расположите шаги в правильном порядке:

1. Вызвать `min()` и `max()` для списка мешков
2. Получить минимальное и максимальное значение
3. Использовать эти значения для сортировки и планирования
4. Вывести результаты на экран

Подсказка: Найти значения лучше до их использования в планировании.

Ответ:

- Вызвать `min()` и `max()` для списка мешков
- Получить минимальное и максимальное значение
- Использовать эти значения для сортировки и планирования
- Вывести результаты на экран

Шаг 16:

Миссия 15

Команда Grow a Garden готовит отчет по количеству семян разных видов: [12, 7, 9, 15]. Расположите шаги для вычисления суммы, сортировки и выбора максимального значения в правильном порядке:

1. Вычислить сумму семян с помощью `sum()`
2. Отсортировать список семян с помощью `sorted()`
3. Найти максимальное значение с помощью `max()`
4. Использовать результаты для планирования и отчета

Подсказка: Сначала получаем нужные данные, потом используем их.

Ответ:

- Вычислить сумму семян с помощью `sum()`
- Отсортировать список семян с помощью `sorted()`
- Найти максимальное значение с помощью `max()`
- Использовать результаты для планирования и отчета

Урок 7 - Помощник-садовод — создание функций (def, параметры)

Шаг 01:

Миссия 1

Помоги команде Grow a Garden сопоставить части функции с их объяснениями.

Части функции:

Ответ:

- def
- имя функции
- параметры
- return

Объяснения:

1. Задаёт имя для вызова функции
2. Возвращает значение из функции
3. Ключевое слово для объявления функции
4. Входные данные, которые функция принимает

Подсказка: Каждый элемент выполняет свою роль: объявление, имя, вход и выход.

- **def** — Ключевое слово для объявления функции
- **имя функции** — Задаёт имя для вызова функции
- **параметры** — Входные данные, которые функция принимает
- **return** — Возвращает значение из функции

Шаг 02:

Миссия 2

В саду Grow a Garden нужно сопоставить примеры параметров функции с их описанием.

Параметры:

Ответ:

- вода
- температура
- время_полива
- тип_растения

Описания:

1. Количество жидкости для полива
2. Время, сколько длится полив
3. Вид растения в саду
4. Значение температуры для оптимального роста

Подсказка: Подумайте, какой параметр отвечает за что в жизни сада.

- **вода** — Количество жидкости для полива
- **температура** — Значение температуры для оптимального роста
- **время_полива** — Время, сколько длится полив
- **тип_растения** — Вид растения в саду

🔧 Шаг 03:

Миссия 3

Расставь в правильном порядке шаги создания функции, которая поможет полить растения в Grow a Garden.

1. Написать тело функции с логикой полива
2. Объявить функцию с помощью def и имени
3. Вызвать функцию с нужными параметрами
4. Вернуть результат (например, количество политых растений)

Подсказка: Сначала объявляем, потом пишем тело, потом возвращаем результат и вызываем функцию.

Ответ:

- Объявить функцию с помощью def и имени
- Написать тело функции с логикой полива
- Вернуть результат (например, количество политых растений)
- Вызвать функцию с нужными параметрами

🔧 Шаг 04:

Миссия 4

Сопоставь ошибки в объявлениях функций с их причинами.

Ошибки:

Ответ:

- `def water(): return вода`
- `def 123function(): pass`
- `def полить(вод@): pass`
- `def полить_растения(вода=): pass`

Причины:

1. Использование недопустимых символов в имени параметра
2. Параметр с пустым значением по умолчанию
3. Имя функции начинается с цифры
4. Использование переменной, которая не определена внутри функции

Подсказка: Внимательно проверь синтаксис и имена.

- **`def water(): return вода`** — Использование переменной, которая не определена внутри функции
- **`def 123function(): pass`** — Имя функции начинается с цифры
- **`def полить(вод@): pass`** — Использование недопустимых символов в имени параметра
- **`def полить_растения(вода=): pass`** — Параметр с пустым значением по умолчанию

🔧 Шаг 05:

Миссия 5

Упорядочи правильные шаги для вызова функции с параметрами в Grow a Garden.

1. Объявить функцию с параметрами
2. Передать аргументы при вызове функции
3. Определить тело функции

4. Получить результат работы функции

Подсказка: Сперва пишем функцию, потом вызываем, а потом работаем с результатом.

Ответ:

- Объявить функцию с параметрами
- Определить тело функции
- Передать аргументы при вызове функции
- Получить результат работы функции

Шаг 06:

Миссия 6

Расставь в правильном порядке этапы создания функции с параметрами, возвращающей информацию о растении.

1. Определить функцию с параметрами
2. Написать тело функции с логикой обработки данных
3. Вернуть нужное значение с помощью return
4. Вызвать функцию с конкретными аргументами

Подсказка: Сначала объявляем, потом реализуем, возвращаем результат и вызываем.

Ответ:

- Определить функцию с параметрами
- Написать тело функции с логикой обработки данных
- Вернуть нужное значение с помощью return
- Вызвать функцию с конкретными аргументами

Шаг 07:

Миссия 6

Садовод Алиса хочет написать функцию, которая приветствует каждого нового растения в её саду. Какая запись создаст функцию без параметров?

Подсказка: Функция начинается с ключевого слова def и имени.

Ответ:

- **def greet_plant():**
- function greet_plant()
- def greet_plant
- def greet_plant[]:

Шаг 08:

Миссия 7

Боб хочет, чтобы функция приветствовала растение по имени. Как правильно добавить параметр "name"?

Подсказка: Параметры указываются в круглых скобках после имени функции.

Ответ:

- **def greet_plant(name):**
- def greet_plant[name]:
- def greet_plant{name}:
- def greet_plant = name:

🔮 Шаг 09:

Миссия 8

В функции Алисы нужно вывести приветствие для растения по имени. Какой код внутри функции правильно это сделает?

Подсказка: Используйте `print()` и параметр `name` для вывода.

Ответ:

- **`print(f"Привет, {name}!")`**
- `print("Привет, name!")`
- `print('name')`
- `print(name)`

🔮 Шаг 10:

Миссия 9

Лили хочет создать функцию, которая возвращает количество листьев у растения. Какой оператор возвращения значения используется в Python?

Подсказка: В Python для возвращения значения из функции есть специальное ключевое слово.

Ответ:

- **`return`**
- `yield`
- `output`
- `send`

🔮 Шаг 11:

Миссия 10

Сопоставьте части функции с их назначением.

Части функции:

Ответ:

- `def greet(name):`
- `print(f"Привет, {name}!")`
- `return`

Назначение:

1. Объявляет функцию с именем и параметром
2. Выводит сообщение с именем
3. Завершает выполнение функции и возвращает значение

Подсказка: Обратите внимание, что объявление функции и тело — разные части.

- **`def greet(name):`** — Объявляет функцию с именем и параметром
- **`print(f"Привет, {name}!")`** — Выводит сообщение с именем
- **`return`** — Завершает выполнение функции и возвращает значение

🔮 Шаг 12:

Миссия 11

Сопоставьте параметры функции с их описанием.

Ответ:

- Параметры
 - Аргументы
 - Тело функции
 - Возвращаемое значение
1. Переменные, указанные в объявлении функции
 2. Значения, передаваемые при вызове функции
 3. Код, который выполняется внутри функции
 4. Результат работы функции

Подсказка: Параметры и аргументы — разные вещи, обратите внимание на контекст.

- **Параметры** — Переменные, указанные в объявлении функции
- **Аргументы** — Значения, передаваемые при вызове функции
- **Тело функции** — Код, который выполняется внутри функции
- **Возвращаемое значение** — Результат работы функции

 Шаг 13:

Миссия 12

Команда Grow a Garden создает функцию, которая должна принимать два параметра: название растения и его высоту. Сопоставьте параметры с их типами.

Ответ:

- name
 - height
1. Строка (название растения)
 2. Число с плавающей точкой (высота в метрах)

Подсказка: Подумайте, какой тип данных подходит для названия, а какой — для числового значения.

- **name** — Строка (название растения)
- **height** — Число с плавающей точкой (высота в метрах)

 Шаг 14:

Миссия 13

Расставьте в правильном порядке шаги создания функции, которая принимает параметр и выводит приветствие растению.

1. Объявить функцию с параметром
2. Вызвать функцию с конкретным аргументом
3. Написать тело функции с print
4. Запустить программу и проверить результат

Подсказка: Сначала нужно создать функцию, потом использовать её.

Ответ:

- Объявить функцию с параметром
- Написать тело функции с print
- Вызвать функцию с конкретным аргументом
- Запустить программу и проверить результат

Шаг 15:

Миссия 14

Расположите этапы работы с функцией, которая возвращает высоту растения:

1. Определить функцию с параметром height
2. Выполнить вычисления внутри функции
3. Вернуть значение с помощью return
4. Вызвать функцию с конкретным числом

Подсказка: Помните, что функция сначала определяется, затем вызывается.

Ответ:

- Определить функцию с параметром height
- Выполнить вычисления внутри функции
- Вернуть значение с помощью return
- Вызвать функцию с конкретным числом

Шаг 16:

Миссия 15

Садовник Тимофей пишет функцию для подсчёта общего количества листьев на двух растениях. Расположите шаги создания такой функции в правильном порядке:

1. Объявить функцию с двумя параметрами
2. В теле функции сложить параметры
3. Вернуть сумму с помощью return
4. Вызвать функцию с двумя числами
5. Вывести результат

Подсказка: Функция принимает данные, обрабатывает их, возвращает и результат выводится отдельно.

Ответ:

- Объявить функцию с двумя параметрами
- В теле функции сложить параметры
- Вернуть сумму с помощью return
- Вызвать функцию с двумя числами
- Вывести результат

Урок 8 - Симуляция урожая — мини-проект работа со списками и словарями

Шаг 02:

Миссия 1

Алиса помогает в саду собрать урожай. У неё есть список с названиями овощей: ['морковь', 'помидор', 'огурец']. Как получить первый овощ из списка?

Подсказка: Индексация в Python начинается с нуля.

Ответ:

- `vegetables[0]`
- `vegetables[1]`
- `vegetables[-1]`

🔗 Шаг 03:

Миссия 2

В саду у Боба есть словарь с количеством собранных фруктов: {'яблоки': 5, 'груши': 3, 'вишня': 7}. Как получить количество груш?

Подсказка: Для доступа к значению словаря используйте ключ в квадратных скобках.

Ответ:

- `fruits['груши']`
- `fruits[1]`
- `fruits.get(3)`

🔗 Шаг 04:

Миссия 3

Елена хочет добавить новый фрукт 'персик' с количеством 4 в свой словарь урожая. Как это сделать?

Подсказка: Для добавления элемента словаря присвойте значение по новому ключу.

Ответ:

- `fruits['персик'] = 4`
- `fruits.add('персик', 4)`
- `fruits.append({'персик':4})`

🔗 Шаг 05:

Миссия 4

Сопоставьте операции с их результатом для списка `crops = ['яблоко', 'морковь', 'помидор']`:

Операции:

Ответ:

- 1. `crops.append('огурец')`
- 2. `crops[1]`
- 3. `len(crops)`
- 4. `crops.pop()`

Результаты:

1. 'морковь'
2. 4
3. 'огурец'
4. ['яблоко', 'морковь', 'помидор', 'огурец']

Подсказка: Метод `append` добавляет элемент в конец списка, `pop` удаляет последний элемент, `len` — длину списка, а индекс — возвращает элемент по позиции.

- `crops.append('огурец')` — ['яблоко', 'морковь', 'помидор', 'огурец']
- `crops[1]` — 'морковь'
- `len(crops)` — 3
- `crops.pop()` — 'огурец'

🔗 Шаг 06:

Миссия 5

Помогите Лизе сопоставить команды для словаря `harvest = {'картофель': 10, 'лук': 5, 'чеснок': 7}` и их действий:

Команды:

Ответ:

- 1. `harvest['лук'] = 8`
- 2. `del harvest['чеснок']`
- 3. `harvest.get('морковь', 0)`
- 4. `'картофель' in harvest`

Действия:

1. Удаление элемента 'чеснок'
2. Получение количества моркови или 0, если нет
3. Проверка, есть ли 'картофель' в словаре
4. Обновление количества лука

Подсказка: Обновление значения — присваивание по ключу, удаление — `del`, проверка наличия — оператор `in`, а метод `get` возвращает значение с запасным значением.

- **`harvest['лук'] = 8`** — Обновление количества лука
- **`del harvest['чеснок']`** — Удаление элемента 'чеснок'
- **`harvest.get('морковь', 0)`** — Получение количества моркови или 0, если нет
- **`'картофель' in harvest`** — Проверка, есть ли 'картофель' в словаре

Шаг 07:

Миссия 6

В саду у Макса список урожаев выглядит так: `[{'помидор': 3}, {'огурец': 5}, {'морковь': 2}]`. Сопоставьте каждый словарь с описанием, что он содержит:

Словари:

Ответ:

- 1. `{'помидор': 3}`
- 2. `{'огурец': 5}`
- 3. `{'морковь': 2}`

Описание:

1. Количество помидоров в первой партии
2. Количество огурцов в средней партии
3. Количество моркови в последней партии

Подсказка: Каждый словарь представляет урожай по овощам с количеством в отдельной партии.

- **`{'помидор': 3}`** — Количество помидоров в первой партии
- **`{'огурец': 5}`** — Количество огурцов в средней партии
- **`{'морковь': 2}`** — Количество моркови в последней партии

Шаг 08:

Миссия 7

Сопоставьте действия садовода с соответствующими методами списков и словарей:

Ответ:

- Добавить новый овощ в список урожая
- Удалить овощ из словаря урожая
- Проверить, есть ли огурец в списке
- Получить количество помидоров из словаря

Подсказка: Метод удаления в списках и словарях отличается, а проверка наличия элемента — оператор `in`.

- **Добавить новый овощ в список урожая** — `append()`
- **Удалить овощ из словаря урожая** — `pop()`
- **Проверить, есть ли огурец в списке** — `in`
- **Получить количество помидоров из словаря** — `get()`

🔗 Шаг 09:

Миссия 8

Сопоставьте типы данных с примерами хранения урожая:

Ответ:

- Список
- Словарь
- Кортеж

Подсказка: Словарь хранит пары ключ-значение, список — упорядоченный набор элементов, а кортеж — неизменяемый список.

- **Список** — `[10, 5, 7]`
- **Словарь** — `{ 'томат': 10, 'морковь': 5 }`
- **Кортеж** — `(3, 6, 2)`

🔗 Шаг 10:

Миссия 9

Сопоставьте действия с правильным описанием, как изменяется урожай в программе:

Ответ:

- Обновление количества овощей в словаре
- Добавление нового элемента в список урожая
- Получение количества элемента по ключу
- Удаление овоща из списка по индексу

Подсказка: В словаре для обновления используется присваивание по ключу, для получения — метод `get()`.

- **Обновление количества овощей в словаре** — Присвоение нового значения по ключу
- **Добавление нового элемента в список урожая** — `append()`
- **Получение количества элемента по ключу** — `get()`
- **Удаление овоща из списка по индексу** — `pop()`

🔗 Шаг 11:

Миссия 10

Расположите последовательность действий для подсчёта общего урожая из словаря с количеством овощей:

1. Перебрать все значения в словаре
2. Создать переменную для хранения суммы и установить 0
3. Добавить каждое значение к сумме
4. Вернуть итоговую сумму

Подсказка: Сначала создаём счётчик, потом перебираем и суммируем, в конце — возвращаем результат.

Ответ:

- Создать переменную для хранения суммы и установить 0
- Перебрать все значения в словаре
- Добавить каждое значение к сумме
- Вернуть итоговую сумму

Шаг 12:

Миссия 11

Расположите этапы создания функции, которая выводит список овощей из словаря урожая:

1. Получить ключи словаря
2. Создать функцию с параметром словаря
3. Вывести ключи в цикле
4. Вызвать функцию с нужным словарём

Подсказка: Сначала объявляем функцию, потом работаем с ключами, затем выводим и вызываем.

Ответ:

- Создать функцию с параметром словаря
- Получить ключи словаря
- Вывести ключи в цикле
- Вызвать функцию с нужным словарём

Шаг 13:

Миссия 12

Расположите шаги по созданию списка из названий овощей, которые собрал Павел:

1. Инициализировать пустой список
2. Перебрать словарь с урожаем
3. Добавить каждый ключ в список
4. Вернуть готовый список

Подсказка: Создаём пустой список, затем заполняем его ключами из словаря и возвращаем.

Ответ:

- Инициализировать пустой список
- Перебрать словарь с урожаем
- Добавить каждый ключ в список
- Вернуть готовый список

Шаг 14:

Миссия 13

Расположите правильный порядок действий для подсчёта среднего урожая из списка чисел:

1. Найти сумму всех элементов
2. Подсчитать количество элементов
3. Вычислить среднее как сумму, делённую на количество
4. Вернуть результат

Подсказка: Сначала суммируем и считаем длину, потом делим, и возвращаем значение.

Ответ:

- Найти сумму всех элементов
- Подсчитать количество элементов
- Вычислить среднее как сумму, делённую на количество
- Вернуть результат

Модуль 5 - Модуль 5 «Умный сад»

Урок 1 - Дневник садовода — работа с файлами (open, read, write)

Шаг 01:

Миссия 1

Семена выросли! Алиса из команды Grow a Garden хочет записать в дневник садовода название нового растения. Какой режим открытия файла нужен, чтобы ****дописать**** текст в уже существующий файл, не стерев прежние записи?

Подсказка: Если вы хотите добавить данные в файл, а не создать его заново или стереть содержимое.

Ответ:

- "a" (append)
- "w" (write)
- "r" (read)
- "x" (exclusive creation)

Шаг 02:

Миссия 2

Ботаник Борис пишет дневник: он писал все заметки, но теперь хочет ****прочитать**** сохранённый файл и вывести всё, что там есть. Какой режим открытия файла нужен?

Подсказка: Режим, который позволяет только читать, не трогая содержимое файла.

Ответ:

- "r" (read)
- "w"
- "a"
- "rb" (read binary)

Шаг 03:

Миссия 3

Герои заметили, что при открытии файла с режимом "w" дневник садовода очищается, даже если в нём уже был текст. Почему так происходит?

Подсказка: Режим "w" стирает всё содержимое файла, если он существует.

Ответ:

- **Потому что "w" перезаписывает файл и удаляет старое содержимое**
- Потому что файл открыт только для чтения
- Потому что файл открыт в бинарном режиме
- Потому что файл не найден

Шаг 04:

Миссия 4

Задача: Сопоставьте ситуацию с подходящим методом работы с файлом, который применяют герои Grow a Garden.

Ситуации:

Ответ:

- Записать новый дневник, начиная с чистого листа
- Добавить заметку о поливе вечером
- Прочитать, что было записано вчера
- Создать файл только если он не существует

Методы открытия файлов:

1. "w"
2. "a"
3. "r"
4. "x"

Подсказка:

Подумайте, что делает каждый режим: перезаписывает, добавляет, читает, создает только при отсутствии.

- **Записать новый дневник, начиная с чистого листа — "w"**
- **Добавить заметку о поливе вечером — "a"**
- **Прочитать, что было записано вчера — "r"**
- **Создать файл только если он не существует — "x"**

Шаг 05:

Миссия 5

Задача: Сопоставьте команды Python с их действием, которые использует команда Grow a Garden при работе с файловым дневником.

Команды:

Ответ:

- file.read()
- file.readline()
- file.readlines()
- file.write()

Действия:

1. Читает весь файл как строку

2. Читает одну строку (до символа перевода строки)
3. Читает все строки в список строк
4. Записывает строку в файл

Подсказка:

Подумайте: какое поведение нужно, когда у вас один за другим строки, список строк или весь текст целиком.

- **file.read()** — Читает весь файл как строку
- **file.readline()** — Читает одну строку (до символа перевода строки)
- **file.readlines()** — Читает все строки в список строк
- **file.write()** — Записывает строку в файл

🔮 Шаг 06:

Миссия 6

Задача: Сопоставьте тип ошибки с её причиной при работе с файлами, с которыми сталкивались герои:

Типы ошибок:

Ответ:

- FileNotFoundError
- PermissionError
- IsADirectoryError
- ValueError

Причины:

1. Попытка открыть файл, которого нет
2. Попытка записать в файл, когда нет прав на запись
3. **Попытка читать содержимое, думая что файл** — это файл, а на самом деле — папка
4. Неверный режим или аргумент функции открытия файла

Подсказка:

Каждая ошибка описывает конкретную ситуацию: нет файла, нет доступа, папка вместо файла, неправильный параметр.

- **FileNotFoundError** — Попытка открыть файл, которого нет
- **PermissionError** — Попытка записать в файл, когда нет прав на запись
- **IsADirectoryError** — Попытка читать содержимое, думая что файл — это файл, а на самом деле — папка
- **ValueError** — Неверный режим или аргумент функции открытия файла

🔮 Шаг 07:

Миссия 7

Задача: Расположите этапы, которые проходят герои при записи погодного отчёта в дневник садовода — от начала до конца.

1. Открыть файл в режиме, допускающем запись или добавление
2. Проверить, существует ли файл, или создать его, если нет
3. Записать строку с погодой и температурой
4. Закрыть файл вручную или с помощью with

Подсказка:

Подумайте: сначала обеспечение файла, затем открыть, записать, потом закрыть.

Ответ:

- Проверить, существует ли файл, или создать его, если нет
- Открыть файл в режиме, допускающем запись или добавление
- Записать строку с погодой и температурой
- Закрыть файл вручную или с помощью with

 Шаг 08:

Миссия 8

Задача: Герои хотят сделать резервную копию дневника перед большим обновлением. Расставьте шаги в правильном порядке:

1. Открыть исходный файл на чтение
2. Открыть файл копии на запись или создание
3. Прочитать весь текст из исходного файла
4. Записать прочитанный текст в файл копии
5. Закрыть оба файла

Подсказка:

Резервное копирование: чтение → запись → закрытие.

Ответ:

- Открыть исходный файл на чтение
- Открыть файл копии на запись или создание
- Прочитать весь текст из исходного файла
- Записать прочитанный текст в файл копии
- Закрыть оба файла

 Шаг 09:

Миссия 9

Задача: Степан из команды Grow a Garden хочет обработать дневник: сначала получить все строки, потом выбрать только заметки за последние 7 дней, и в конце — сохранить их в новый файл. Расположите операции в правильном порядке:

1. Открыть дневник оригинал на чтение
2. Читать все строки
3. **Отфильтровать строки по дате** — последние 7 дней
4. Открыть новый файл для записи (или перезаписи)
5. Записать отфильтрованные строки в новый файл
6. Закрыть оба файла

Подсказка:

Логика: сначала получать данные, потом фильтровать, потом сохранять.

Ответ:

- Открыть дневник оригинал на чтение
- Читать все строки
- **Отфильтровать строки по дате** — последние 7 дней
- Открыть новый файл для записи (или перезаписи)
- Записать отфильтрованные строки в новый файл
- Закрыть оба файла

Шаг 10:

Миссия 10

Задача: Когда садовод Виктория записывает дневник, ей нужно безопасно обновлять файл: сначала создать резервную копию, затем обновить заметки, и если что-то пошло не так — вернуть резервную копию. Расставьте шаги в правильной последовательности:

1. Открыть оригинальный дневник на чтение
2. Считать все данные из оригинала
3. Записать данные в резервный файл
4. Открыть оригинальный дневник на запись или перезапись
5. Записать обновлённые заметки в оригинальный файл
6. **Если ошибка** — восстановить из резервной копии
7. **Если всё прошло хорошо** — удалить резервную копию (или сохранить для истории)

Подсказка:

При работе с файлами важно предусмотреть возврат к безопасному состоянию.

Ответ:

- Открыть оригинальный дневник на чтение
- Считать все данных из оригинала
- Записать данные в резервный файл
- Открыть оригинальный дневник на запись или перезапись
- Записать обновлённые заметки в оригинальный файл
- **Если ошибка** — восстановить из резервной копии
- **Если всё прошло хорошо** — удалить резервную копию (или сохранить для истории)

Шаг 11:

Миссия 11

Алиса из команды «Grow a Garden» хочет сохранить своё первое наблюдение в дневнике сада. Она только что собрала урожай и хочет записать: «Сегодня цветы распустились». Как правильно открыть файл «diary.txt» для записи (создать или перезаписать)?

Подсказка: Используйте режим, который позволит создать новый файл, если его нет, и стереть старое содержимое, если есть.

Ответ:

- `open('diary.txt', 'w')`
- `open('diary.txt', 'r')`
- `open('diary.txt', 'a')`
- `open('diary.txt', 'r+')`

Шаг 12:

Миссия 12

Боб хочет дописать новый прогноз погоды в конец файла «weather.log», не теряя старых записей. Какой режим открыть файла ему нужен?

Подсказка: Добавление в конец файла — это режим, где ничего не перезаписывается.

Ответ:

- `open('weather.log', 'a')`
- `open('weather.log', 'w')`
- `open('weather.log', 'r')`

- `open('weather.log', 'r+')`

🔗 Шаг 13:

Миссия 13

Клара хочет прочитать всё содержимое дневника сада «garden.txt». Какой режим она должна использовать?

Подсказка: Чтение — не менять файл.

Ответ:

- `open('garden.txt', 'r')`
- `open('garden.txt', 'w')`
- `open('garden.txt', 'a')`
- `open('garden.txt', 'x')`

🔗 Шаг 14:

Миссия 14

Даниил готовит отчёт: он хочет открыть файл для чтения и записи, без стирания предыдущего содержимого. Какая команда подойдёт?

Подсказка: Нужно сохранить старое и при этом иметь возможность дописать и прочитать.

Ответ:

- `open('report.txt', 'r+')`
- `open('report.txt', 'w+')`
- `open('report.txt', 'a')`
- `open('report.txt', 'r')`

🔗 Шаг 15:

Миссия 15

Сопоставьте режимы открытия файлов с их описаниями:

Режимы:

Ответ:

- 'r'
- 'w'
- 'a'
- 'r+'

Описание:

1. Читать и писать, не удаляя содержимое в начале, курсор в начале файла
2. Открыть для записи, стереть старое содержимое, если есть
3. Добавление текста в конец, не трогая начало
4. Только чтение, без изменений

Подсказка: Почитайте про стандартные режимы в документации Python.

- **'r'** — Только чтение, без изменений
- **'w'** — Открыть для записи, стереть старое содержимое, если есть
- **'a'** — Добавление текста в конец, не трогая начало
- **'r+'** — Читать и писать, не удаляя содержимое в начале, курсор в начале файла

Шаг 16:

Миссия 16

Команда «Grow a Garden» ведёт журнал полива в файле «watering.log». Сопоставьте, что будет, если открыть файл с одним из режимов и выполнить операцию write без чтения.

Режимы:

Ответ:

- 'w'
- 'a'
- 'r'
- 'r+'

Что произойдёт:

1. Записать станет возможно, старое содержимое исчезнет.
2. Запись в конец, старое содержимое останется.
3. Ошибка, потому что нельзя писать в режиме, только читать.
4. Можно писать и читать, не уничтожая файл.

Подсказка: Подумайте, какой режим разрешает write и что происходит с данными.

- 'w' — Записать станет возможно, старое содержимое исчезнет.
- 'a' — Запись в конец, старое содержимое останется.
- 'r' — Ошибка, потому что нельзя писать в режиме, только читать.
- 'r+' — Можно писать и читать, не уничтожая файл.

Урок 2 - Архив посевов — сохранение данных в файл

Шаг 01:

Миссия 1

Лёня из команды Grow a Garden хочет сохранить список посевов в файл: у него массив строк с названиями растений. Какой метод нужно использовать, чтобы записать этот список в файл, перезаписав всё предыдущее содержимое?

Подсказка: Режим должен очищать файл, если он уже существует.

Ответ:

- Открыть с режимом "w" и использовать `file.write()/writelines()`
- Открыть с режимом "r"
- Открыть с режимом "a"
- Открыть с режимом "x"

Шаг 02:

Миссия 2

Мария хочет дополнить архив посевов — добавить новую запись, не теряя старых. Какой режим открытия файла ей подойдёт?

Подсказка: Новый текст должен идти после существующего содержимого.

Ответ:

- Режим "a" (append)
- Режим "w"
- Режим "r"

- Режим "x"

Шаг 03:

Миссия 3

Когда Архив посевов ещё не существует, Иван хочет создать его, записать первые данные, но если файл уже есть — не затирать старое содержимое и просто дописать. Какой режим подойдут для этого сценария?

Подсказка: Подумай, как сохранить и создавать при необходимости.

Ответ:

- **Использовать "a" (append)** — создаст, если нет, и допишет, если есть
- Использовать "w"
- Использовать "x"
- Использовать "r"

Шаг 04:

Миссия 4

Задача: Сопоставьте, какую функцию или метод используют герои для различных операций с архивом посевов.

Операции:

Ответ:

- Запись всей коллекции растений за год в файл как список строк
- Добавление одной записи о новом посеве в конце
- Чтение всех записей архива
- Чтение только первой записи файла

Методы/функции:

1. `file.write()`
2. `file.writelines()`
3. `file.read()`
4. `file.readline()`

Подсказка: Каждая операция требует либо список строк, либо одну строку, либо весь файл.

- **Запись всей коллекции растений за год в файл как список строк** — `file.writelines()`
- **Добавление одной записи о новом посеве в конце** — `file.write()`
- **Чтение всех записей архива** — `file.read()`
- **Чтение только первой записи файла** — `file.readline()`

Шаг 05:

Миссия 5

Задача: Команда Grow a Garden хранит архив в текстовом файле. Нужно сопоставить режимы открытия файла с их эффектами в архиве.

Режимы:

Ответ:

- "w"
- "a"

- "x"
- "r"

Эффекты:

1. Создать новый файл и выбросить ошибку, если уже есть
2. Открыть файл только для чтения, ошибка если нет
3. Очищает содержимое, если файл существует, или создаёт, если нет
4. Добавляет в конец, создаёт, если нет

Подсказка: Подумайте, что происходит в каждом режиме: читать, перезаписать, дозаписать, создать-исключительно.

- "w" — Очищает содержимое, если файл существует, или создаёт, если нет
- "a" — Добавляет в конец, создаёт, если нет
- "x" — Создать новый файл и выбросить ошибку, если уже есть
- "r" — Открыть файл только для чтения, ошибка если нет

Шаг 06:

Миссия 6

Задача: Расположите шаги создания архива посевов, начиная с пустого состояния, когда архив ещё не существует:

1. Создать новый файл архива
2. Открыть файл в подходящем режиме для записи
3. Записать список посевов в файл
4. Закрыть файл (или использовать with)

Подсказка: Если файл не существует — мы его создаём, потом записываем, потом закрываем.

Ответ:

- Создать новый файл архива
- Открыть файл в подходящем режиме для записи
- Записать список посевов в файл
- Закрыть файл (или использовать with)

Шаг 07:

Миссия 7

Задача: Команда хочет обновить архив: сначала сохранить резервную копию, затем добавить новые записи, и в конце закрыть все файлы. Расположите шаги в правильном порядке:

1. Прочитать содержимое текущего архива (старый файл)
2. Сохранить резервную копию архива (с названием backup или временный файл)
3. Открыть архив в режиме добавления (append) для новых записей
4. Записать новые записи посевов
5. Закрыть файлы (и оригинал, и резервную копию, если нужно ладить безопасно)

Подсказка: Резерв — это про страховать. Новые записи — только после сохранения старого состояния.

Ответ:

- Прочитать содержимое текущего архива (старый файл)
- Сохранить резервную копию архива (с названием backup или временный файл)

- Открыть архив в режиме добавления (append) для новых записей
- Записать новые записи посевов
- Заккрыть файлы (и оригинал, и резервную копию, если нужно ладить безопасно)

Шаг 08:

Миссия 8

Задача: Сады растут! Героям нужно скоро подсчитать, сколько разных растений посажено за сезон, обновить информацию в архиве и сохранить только уникальные записи. Расставьте шаги:

1. Открыть архив на чтение
2. Прочитать всё содержимое и получить список строк
3. Выделить только уникальные названия растений
4. Открыть архив в режиме записи (перезаписи)
5. Записать уникальные названия в архив, разделив по строкам
6. Заккрыть файл

Подсказка: Сначала собрать, затем фильтровать, затем сохранять.

Ответ:

- Открыть архив на чтение
- Прочитать всё содержимое и получить список строк
- Выделить только уникальные названия растений
- Открыть архив в режиме записи (перезаписи)
- Записать уникальные названия в архив, разделив по строкам
- Заккрыть файл

Шаг 09:

Миссия 9

Задача: Самое сложное: команда хочет создавать архив посевов ежедневно, но если в течение недели ошибок не было, старые записи удаляются, и архив очищается. Расставьте последовательность шагов для этой задачи:

1. **Определить, что сегодня** — первый день новой недели (например, понедельник)
2. **Если да** — сделать резервную копию архива до очистки
3. Открыть архив в режиме записи, чтобы очистить старое содержимое
4. Добавить записи сегодняшнего дня
5. **Если ошибки при записи** — восстановить из резервной копии
6. Заккрыть файл

Подсказка: Действия должны защищать архив от потерь, особенно при автоматической очистке.

Ответ:

- **Определить, что сегодня** — первый день новой недели (например, понедельник)
- **Если да** — сделать резервную копию архива до очистки
- Открыть архив в режиме записи, чтобы очистить старое содержимое
- Добавить записи сегодняшнего дня
- **Если ошибки при записи** — восстановить из резервной копии
- Заккрыть файл

Шаг 10:

Миссия 10

Алиса только что собрала первый урожай в своем саду и хочет сохранить заметку «Сегодня цветы распустились» в файл `diary.txt`. Как правильно открыть файл, чтобы записать эту заметку, создав новый файл или перезаписав старый?

Подсказка: Нужно открыть файл так, чтобы старое содержимое стерлось, если файл существует, или создать новый, если его нет.

Ответ:

- `open('diary.txt', 'w')`
- `open('diary.txt', 'r')`
- `open('diary.txt', 'a')`
- `open('diary.txt', 'x')`

Шаг 11:

Миссия 11

Боб ведёт журнал погоды в файле `weather.log` и хочет добавить новый прогноз в конец файла, не удаляя старые записи. Какой режим открытия файла ему нужен?

Подсказка: Этот режим позволяет дописывать в конец файла без изменения существующего содержимого.

Ответ:

- `open('weather.log', 'a')`
- `open('weather.log', 'w')`
- `open('weather.log', 'r')`
- `open('weather.log', 'r+')`

Шаг 12:

Миссия 12

Клара хочет прочитать все записи из файла дневника сада `garden.txt`. Какой режим открытия файла ей нужен?

Подсказка: Открыть файл только для чтения, не изменяя его содержимое.

Ответ:

- `open('garden.txt', 'r')`
- `open('garden.txt', 'w')`
- `open('garden.txt', 'a')`
- `open('garden.txt', 'x')`

Шаг 13:

Миссия 13

Даниил хочет открыть файл `report.txt` для чтения и записи, не удаляя старые данные. Какой режим нужно использовать?

Подсказка: Файл должен оставаться с прежним содержимым, но к нему можно дописывать и читать.

Ответ:

- `open('report.txt', 'r+')`
- `open('report.txt', 'w+')`

- `open('report.txt', 'a')`
- `open('report.txt', 'r')`

🔗 Шаг 14:

Миссия 14

Сопоставьте режимы открытия файлов с их описаниями:

Режимы:

Ответ:

- `'r'`
- `'w'`
- `'a'`
- `'r+'`

Описание:

1. Открыть для чтения, без изменения файла
2. Открыть для записи, стереть старое содержимое
3. Добавить запись в конец файла
4. Читать и писать без удаления содержимого

Подсказка: Каждый режим по-разному влияет на файл и курсор.

- `'r'` — Открыть для чтения, без изменения файла
- `'w'` — Открыть для записи, стереть старое содержимое
- `'a'` — Добавить запись в конец файла
- `'r+'` — Читать и писать без удаления содержимого

🔗 Шаг 15:

Миссия 15

Команда «Grow a Garden» ведёт журнал полива `watering.log`. Сопоставьте режимы открытия с тем, что произойдет при записи с помощью метода `write()` без предварительного чтения.

Режимы:

Ответ:

- `'w'`
- `'a'`
- `'r'`
- `'r+'`

Результат записи:

1. Стирается старое содержимое и записывается новое
2. Добавляется запись в конец, старое содержимое сохраняется
3. Ошибка, запись невозможна
4. Можно читать и писать, содержимое сохраняется

Подсказка: Учтите, что разные режимы разрешают или запрещают запись и чтение.

- `'w'` — Стирается старое содержимое и записывается новое
- `'a'` — Добавляется запись в конец, старое содержимое сохраняется
- `'r'` — Ошибка, запись невозможна
- `'r+'` — Можно читать и писать, содержимое сохраняется

Шаг 16:

Миссия 16

Сопоставьте методы чтения файла с их описаниями:

Методы:

Ответ:

- `read()`
- `readline()`
- `readlines()`
- `for line in file`

Описание:

1. Читает весь файл как одну строку
2. Читает одну строку за раз
3. Возвращает список всех строк файла
4. Проходит по строкам файла по очереди

Подсказка: Выбирайте методы в зависимости от размера файла и задачи чтения.

- **`read()`** — Читает весь файл как одну строку
- **`readline()`** — Читает одну строку за раз
- **`readlines()`** — Возвращает список всех строк файла
- **`for line in file`** — Проходит по строкам файла по очереди

Урок 3 - Садовые рецепты — подключение модулей (`import`, `random`)

Шаг 01:

Миссия 1

Ксюша из команды Grow a Garden хочет использовать модуль Python, чтобы случайно выбрать семя из списка. Что она должна написать в начале своего скрипта, чтобы подключить модуль, работающий с генерацией случайных чисел и выбором?

Подсказка: Это стандартный модуль, не нужно устанавливать через pip.

Ответ:

- **`import random`**
- `import seed`
- `from random import file`
- `import randint`

Шаг 02:

Миссия 2

Когда модуль `random` уже подключён, Ваня хочет выбрать случайный элемент из списка растений `['роза', 'тюльпан', 'ромашка']`. Какой метод `random` позволит сделать это?

Подсказка: Метод, который работает с последовательностями (списками, строками).

Ответ:

- **`random.choice(my_list)`**
- `random.randint(my_list)`
- `random.random()`
- `random.shuffle(my_list)`

Шаг 03:

Миссия 3

Марта хочет, чтобы выбор растения был воспроизводимым — одна и та же последовательность случайных выборов, когда она перезапускает скрипт. Что она должна использовать?

Подсказка: Подумай, как задать начальное состояние генератора случайных чисел.

Ответ:

- **random.seed(число)**
- random.choice()
- from random import seed
- import seed

Шаг 04:

Миссия 4

Задача: Команда Grow a Garden применяет разные функции из модуля random. Сопоставьте функцию с тем, что она делает.

Функции:

Ответ:

- random.randint(a, b)
- random.random()
- random.choice(seq)
- random.shuffle(list)

Что делает:

1. Возвращает число с плавающей точкой между 0 и 1
2. Перемешивает элементы списка на месте
3. Выбирает случайный элемент из последовательности
4. Выдает случайное целое число в диапазоне [a, b]

Подсказка: Подумайте, что возвращают целые числа, что список, что дробное число, что просто выбор из списка.

- **random.randint(a, b)** — Выдает случайное целое число в диапазоне [a, b]
- **random.random()** — Возвращает число с плавающей точкой между 0 и 1
- **random.choice(seq)** — Выбирает случайный элемент из последовательности
- **random.shuffle(list)** — Перемешивает элементы списка на месте

Шаг 05:

Миссия 5

Задача: Олег создал модуль рецептов, и хочет импортировать из него только одну функцию — допустим, функцию mix_seed. Какой импорт подойдет?

Варианты импорта:

Ответ:

- import module_recipes
- from module_recipes import mix_seed
- import mix_seed
- from random import mix_seed

Подсказка: Подумай, в каком месте находится функция `mix_seed` — в твоём модуле, не в `random`.

- **Олег хочет импортировать весь модуль** — `import module_recipes`
- **Олег хочет импортировать только функцию `mix_seed`** — `from module_recipes import mix_seed`
- **Неправильно: `mix_seed` не является именем модуля** — `import mix_seed`
- **Неправильно: смешение `random` и собственный модуль** — `from random import mix_seed`

🔮 Шаг 06:

Миссия 6

Задача: Команда `Grow a Garden` хочет сгенерировать несколько случайных чисел для роста разных растений: например, рост розы, тюльпана, ромашки — они хотят сгенерировать целое число от 1 до 10 для каждого. Сопоставьте, какой вызов `random` подходит под каждую ситуацию:

Ситуации:

Ответ:

- **Случайный рост розы** — целое число от 1 до 10 включительно
- Случайное число между 0 и 1 с плавающей точкой
- Выбрать случайную позицию в саду из списка позиций ['лево', 'центр', 'право']
- Перемешать весь список растений

Вызовы:

1. `random.randint(1, 10)`
2. `random.random()`
3. `random.choice(['лево', 'центр', 'право'])`
4. `random.shuffle(plants_list)`

Подсказка: Обрати внимание, что `randint` — целые, `random` — дробные, `choice` — выбор из списка, `shuffle` — изменение порядка.

- **Случайный рост розы** — целое число от 1 до 10 включительно — `random.randint(1, 10)`
- **Случайное число между 0 и 1 с плавающей точкой** — `random.random()`
- **Выбрать случайную позицию в саду из списка позиций ['лево', 'центр', 'право']** — `random.choice(['лево', 'центр', 'право'])`
- **Перемешать весь список растений** — `random.shuffle(plants_list)`

🔮 Шаг 07:

Миссия 7

Задача: Расположите шаги, которые делает герой Гриша, чтобы использовать модуль `random` для создания рецепта: сначала импорт, потом он хочет выбрать случайное семя, потом перемешать список вариантов семян, и в конце вывести результат на экран.

1. `import random`
2. `random.shuffle(list_of_seeds)`
3. `print(результат)`
4. `random.choice(list_of_seeds)`

Подсказка: Подумайте, что должно произойти до того, как вы перемешаете, до выбора, и когда показать результат.

Ответ:

- `import random`
- `random.shuffle(list_of_seeds)`
- `random.choice(list_of_seeds)`
- `print(результат)`

 Шаг 08:**Миссия 8**

Задача: Юля хочет генерировать 3 случайных числа для трёх растений, используя `randint`, затем найти среднее этой тройки, и вывести сообщение с результатом рецепта. Расположите шаги:

1. `import random`
2. `random.randint(...)` для первого растения
3. `random.randint(...)` для второго растения
4. `random.randint(...)` для третьего растения
5. Посчитать среднее значение трёх чисел
6. `print("Средний рост растений: ", среднее)`

Подсказка: Сначала импорт, потом генерация трёх чисел, потом вычисление, потом вывод.

Ответ:

- `import random`
- `random.randint(...)` для первого растения
- `random.randint(...)` для второго растения
- `random.randint(...)` для третьего растения
- Посчитать среднее значение трёх чисел
- `print("Средний рост растений: ", среднее)`

 Шаг 09:**Миссия 9**

Задача: Сергей хочет создать рецепт садового микса: у него список `herbs = ['базилик', 'тимьян', 'мята', 'розмарин']`. Он хочет сначала импортировать `random`, затем выбрать случайную подгруппу из двух трав (без повторений), затем перемешать выбранную подгруппу, и напечатать её. Расставьте шаги:

1. `import random`
2. `random.sample(herbs, 2)`
3. `random.shuffle(выбранная_подгруппа)`
4. `print(результат)`

Подсказка: `sample` берёт несколько уникальных элементов, `shuffle` — меняет порядок внутри, `print` — в конце.

Ответ:

- `import random`
- `random.sample(herbs, 2)`
- `random.shuffle(выбранная_подгруппа)`
- `print(результат)`

🔮 Шаг 10:

Миссия 10

Задача: Самая сложная миссия: команда хочет создать несколько рецептов сразу — для роз, тюльпанов и ромашек. Для каждого они хотят: импорт `random`, установить семя генератора для воспроизводимости, сгенерировать рост как целое число от 5 до 15, выбрать случайный цвет лепестков из списка ['красный', 'жёлтый', 'белый', 'розовый'], затем показать все рецепты. Расположите шаги в правильном порядке.

1. `import random`
2. `random.seed(42)`
3. для каждого растения: generate рост через `random.randint(5,15)`
4. выбрать цвет через `random.choice(...)` для каждого растения
5. собрать информацию о рецептах в список/словарь
6. `print` всех рецептов

Подсказка: Подумай: сначала настройка, потом для каждого растения, потом сбор, потом вывод.

Ответ:

- `import random`
- `random.seed(42)`
- для каждого растения: generate рост через `random.randint(5,15)`
- выбрать цвет через `random.choice(...)` для каждого растения
- собрать информацию о рецептах в список/словарь
- `print` всех рецептов

🔮 Шаг 11:

Миссия 11

Аня, садовница из команды `Grow a Garden`, хочет использовать случайные числа для выбора семян. Какой модуль Python нужно импортировать для этого?

Подсказка: Модуль, который генерирует случайные значения.

Ответ:

- **`random`**
- `math`
- `sys`
- `os`

🔮 Шаг 12:

Миссия 12

Код Тимура из `Grow a Garden` пытается использовать функцию `randint`, но Python выдаёт ошибку. Что нужно добавить в начало скрипта?

Подсказка: Помните, что функции из модуля не работают без импорта.

Ответ:

- **`import random`**
- `import randint`
- `from math import randint`
- `import sys`

Шаг 13:

Миссия 13

Лея хочет использовать функцию `choice` из модуля `random`, чтобы случайно выбрать тип растения. Как правильно импортировать только эту функцию?

Подсказка: Используйте импорт определённой функции.

Ответ:

- **from random import choice**
- `import choice from random`
- `import random.choice`
- `from random import randint`

Шаг 14:

Миссия 14

Сопоставьте действия с их описанием при работе с модулем `random`.

Действия:

Ответ:

- `import random`
- `from random import randint`
- `random.randint(1,10)`
- `choice(['яблоко', 'груша', 'слива'])`

Описание:

1. Импорт всей библиотеки `random`
2. Импорт только функции `randint` из модуля `random`
3. Генерация случайного числа от 1 до 10 включительно
4. Выбор случайного элемента из списка фруктов

Подсказка: Импорт позволяет использовать функции модуля в коде.

- **import random** — Импорт всей библиотеки `random`
- **from random import randint** — Импорт только функции `randint` из модуля `random`
- **random.randint(1,10)** — Генерация случайного числа от 1 до 10 включительно
- **choice(['яблоко', 'груша', 'слива'])** — Выбор случайного элемента из списка фруктов

Шаг 15:

Миссия 15

Сопоставьте, какие части кода связаны с импортом и применением функций модуля `random`.

Ответ:

- `from random import choice`
- `random.randint(1, 5)`
- `import random`
- `choice(['солнце', 'дождь', 'ветер'])`

1. Полный импорт модуля `random`
2. Использование функции `randint` из модуля `random`
3. Использование функции `choice` без имени модуля
4. Импорт только функции `choice` из `random`

Подсказка: Обратите внимание, как меняется способ вызова функции в зависимости от импорта.

- **from random import choice** — Импорт только функции choice из random
- **random.randint(1, 5)** — Использование функции randint из модуля random
- **import random** — Полный импорт модуля random
- **choice(['солнце', 'дождь', 'ветер'])** — Использование функции choice без имени модуля

🔮 Шаг 16:

Миссия 16

Расположите действия по использованию модуля random в правильном порядке, чтобы выбрать случайное растение для сада.

1. Вызвать функцию выбора случайного элемента из списка
2. Создать список растений
3. Импортировать модуль random
4. Вывести выбранный элемент

Подсказка: Подумайте, сначала что нужно для работы с функцией, потом данные, потом вызов и вывод.

Ответ:

- Импортировать модуль random
- Создать список растений
- Вызвать функцию выбора случайного элемента из списка
- Вывести выбранный элемент

📖 Урок 4 - Прогноз погоды — использование random для симуляции

🔮 Шаг 01:

Миссия 1

В саду Grow a Garden герои решили смоделировать погоду с помощью модуля random. Какой способ правильный для импортирования модуля random в Python?

Подсказка: Чтобы использовать функции из модуля, нужно его сначала импортировать.

Ответ:

- **import random**
- include random
- using random
- from random import all

🔮 Шаг 02:

Миссия 2

Герои хотят сгенерировать случайное число от 1 до 10 для прогноза погоды. Какой код будет правильным?

Подсказка: В модуле random есть функция для выбора случайного целого числа в диапазоне.

Ответ:

- **random.randint(1, 10)**
- random.random(1, 10)
- random.choice(1, 10)
- random.range(1, 10)

🔮 Шаг 03:

Миссия 3

Для случайного выбора одного из нескольких погодных условий герои хотят использовать `random`. Какая функция лучше всего подойдет?

Подсказка: Функция должна выбрать случайный элемент из списка.

Ответ:

- **`random.choice(list_of_weathers)`**
- `random.randint(list_of_weathers)`
- `random.shuffle(list_of_weathers)`
- `random.sample(list_of_weathers)`

🔮 Шаг 04:

Миссия 4

Сопоставьте функции модуля `random` с их назначением.

Функции:

Ответ:

- `random.randint`
- `random.choice`
- `random.shuffle`
- `random.random`

Назначения:

1. Возвращает случайное число с плавающей точкой от 0 до 1
2. Выбирает случайный элемент из последовательности
3. Перемешивает элементы списка случайным образом
4. Генерирует случайное целое число в заданном диапазоне

Подсказка: Подумайте, какие задачи решают эти функции в контексте погоды.

- **`random.randint`** — Генерирует случайное целое число в заданном диапазоне
- **`random.choice`** — Выбирает случайный элемент из последовательности
- **`random.shuffle`** — Перемешивает элементы списка случайным образом
- **`random.random`** — Возвращает случайное число с плавающей точкой от 0 до 1

🔮 Шаг 05:

Миссия 5

Сопоставьте типы данных, которые часто используются с функциями модуля `random`, и примеры данных.

Типы данных:

Ответ:

- Список
- Диапазон чисел
- Число с плавающей точкой

Примеры данных:

1. ["солнечно", "дождь", "ветер"]
2. 1, 2, 3, ..., 10

3. 0.0, 0.5, 0.9

Подсказка: Посмотрите, какие данные подходят к разным способам генерации случайностей.

- **Список** — ["солнечно", "дождь", "ветер"]
- **Диапазон чисел** — 1, 2, 3, ..., 10
- **Число с плавающей точкой** — 0.0, 0.5, 0.9

🔮 Шаг 06:

Миссия 6

Сопоставьте действия команды Grow a Garden с нужными функциями random для прогноза погоды.

Действия:

Ответ:

- Выбрать случайный тип осадков
- Определить случайную температуру в диапазоне 15-25 градусов
- Перемешать порядок дней недели для прогноза
- Получить случайное число с плавающей точкой для влажности

Функции:

1. random.choice()
2. random.randint()
3. random.shuffle()
4. random.random()

Подсказка: Подумайте, какие функции лучше всего подходят под каждое действие.

- **Выбрать случайный тип осадков** — random.choice()
- **Определить случайную температуру в диапазоне 15-25 градусов** — random.randint()
- **Перемешать порядок дней недели для прогноза** — random.shuffle()
- **Получить случайное число с плавающей точкой для влажности** — random.random()

🔮 Шаг 07:

Миссия 7

Расположите шаги создания функции для случайного прогноза погоды в правильном порядке:

1. Импортировать модуль random
2. Создать список с вариантами погоды
3. Написать функцию, которая возвращает случайный элемент из списка
4. Вызвать функцию и вывести результат

Подсказка: Подумайте, с чего начинается работа с модулями, и как устроена функция.

Ответ:

- Импортировать модуль random
- Создать список с вариантами погоды
- Написать функцию, которая возвращает случайный элемент из списка
- Вызвать функцию и вывести результат

🔮 Шаг 08:

Миссия 8

Маша хочет узнать, будет ли завтра солнечно, чтобы полить свой сад. Какой модуль Python поможет ей сделать случайный прогноз погоды?

Подсказка: Этот модуль часто используют для генерации случайных чисел и выбора случайных вариантов.

Ответ:

- **random**
- math
- datetime
- os

Шаг 09:

Миссия 9

Сергей хочет сгенерировать случайное число от 1 до 3, чтобы выбрать тип погоды: 1 — солнечно, 2 — дождь, 3 — ветер. Какую функцию из модуля random ему использовать?

Подсказка: Функция выбирает случайное целое число в заданном диапазоне.

Ответ:

- **randint**
- choice
- random

Шаг 10:

Миссия 10

Анна хочет случайно выбрать один из трёх видов погоды: "солнечно", "дождь" или "ветер". Какой метод random лучше использовать?

Подсказка: Эта функция выбирает случайный элемент из списка или кортежа.

Ответ:

- **choice**
- randint
- shuffle

Шаг 11:

Миссия 11

Алиса программирует прогноз погоды для сада. Помогите сопоставить функцию random с её назначением:

Ответ:

- randint
- choice
- shuffle
- random

Подсказка: Обратите внимание на тип результата: число, элемент из списка, перемешанный список или число с плавающей точкой.

- **randint** — Генерирует случайное целое число в заданном диапазоне.
- **choice** — Выбирает случайный элемент из последовательности.
- **shuffle** — Перемешивает элементы списка случайным образом.
- **random** — Возвращает случайное число с плавающей точкой от 0 до 1.

🔮 Шаг 12:

Миссия 12

В саду Тимофея прогноз погоды зависит от нескольких факторов. Сопоставь погодные условия с их описаниями:

Ответ:

- Солнечно
- Дождь
- Ветер
- Облачно

Подсказка: Подумай, как каждое условие влияет на растения в саду.

- **Солнечно** — Яркое солнце, тепло и свет для роста растений.
- **Дождь** — Вода с неба, которая помогает растениям пить.
- **Ветер** — Дует воздух, может охлаждать или ломать растения.
- **Облачно** — Много облаков на небе, мало света.

🔮 Шаг 13:

Миссия 13

Паша создает программу, которая выводит прогноз на неделю. Сопоставь день недели с возможной погодой:

Ответ:

- Понедельник
- Среда
- Пятница
- Воскресенье

Подсказка: Прогнозы генерируются случайно, но нужно логично распределить погоду по дням.

- **Понедельник** — Солнечно
- **Среда** — Дождь
- **Пятница** — Ветер
- **Воскресенье** — Облачно

🔮 Шаг 14:

Миссия 14

Валя пишет программу, которая случайно выбирает погодные условия для каждого дня недели. Расположи правильный порядок действий для реализации этой задачи:

Подсказка: Подумай, как сначала подготовить данные, затем случайно выбрать погоду и показать результат.

Ответ:

- Создать список возможных погодных условий.
- Использовать `random.choice` для выбора погоды для каждого дня.
- Создать список дней недели.
- Вывести прогноз погоды для каждого дня.

🔮 Шаг 15:

Миссия 15

Дима хочет смоделировать смену погоды за три дня. Расставь шаги программы в правильном порядке:

Подсказка: Сначала подготовь данные, затем выбери случайные значения, после — покажи пользователю.

Ответ:

- Определить список возможных погодных условий.
- Для каждого дня вызвать `random.choice` для выбора погоды.
- Сохранить выбранные условия в список.
- Вывести прогноз на три дня.

Шаг 16:

Миссия 16

Лена создает расширенную версию прогноза, учитывающую вероятность дождя. Расположи действия по этапам создания программы:

Подсказка: Подумай, как сначала задать вероятности, потом сгенерировать случайное событие, и в конце показать результат.

Ответ:

- Задать вероятности для каждого типа погоды.
- Сгенерировать случайное число для выбора события.
- Сопоставить число с погодным событием.
- Вывести прогноз с учётом вероятности.

Урок 5 - Первый класс — основы ООП (создание объектов)

Шаг 01:

Миссия 1

Аня выращивает розы в своём саду. Она хочет создать в Python объект "Роза" с характеристиками. Что из перечисленного лучше всего подходит для создания объекта в ООП?

Подсказка: Подумайте, как в Python создаются объекты — что нужно определить?

Ответ:

- **Класс, который описывает свойства и методы розы**
- Переменную с числом роз
- Функцию для полива роз
- Список с названиями цветов

Шаг 02:

Миссия 2

Макс создал класс "Цветок" для своего сада. Как правильно создать объект класса "Цветок" в Python?

Подсказка: Вспомните, как вызывается конструктор класса.

Ответ:

- **`flower = Цветок()`**
- `flower = Цветок`
- `flower = Цветок.create()`

🧙 Шаг 03:

Миссия 3

Вася добавил в класс "Растение" атрибут "цвет". Что такое этот "цвет" в контексте объекта?

Подсказка: Это характеристика, которую имеет каждый объект этого класса.

Ответ:

- **Атрибут объекта, описывающий свойство растения**
- Метод для покраски растения
- Функция для изменения цвета
- Переменная вне класса

🧙 Шаг 04:

Миссия 4

Аня создала класс "Грядка" и объекты этого класса для разных растений. Сопоставьте термины с правильными пояснениями.

Термины:

Ответ:

- Класс
- Объект
- Атрибут
- Метод

Определения:

1. Шаблон или чертёж для создания объектов.
2. Конкретный экземпляр класса.
3. Переменная, которая хранит данные объекта.
4. Функция, связанная с объектом.

Подсказка: Класс — это как рецепт, объект — готовое блюдо.

- **Класс** — Шаблон или чертёж для создания объектов.
- **Объект** — Конкретный экземпляр класса.
- **Атрибут** — Переменная, которая хранит данные объекта.
- **Метод** — Функция, связанная с объектом.

🧙 Шаг 05:

Миссия 5

Макс создал класс "Сад" с методами "полить" и "удобрить". Сопоставьте, что делают эти методы.

Методы:

Ответ:

- полить
- удобрить
- собрать_урожай

Действия:

1. Добавить воду растениям
2. Улучшить почву для роста

3. Собрать плоды с растений

Подсказка: Методы обычно описывают действия с объектом.

- **полить** — Добавить воду растениям
- **удобрить** — Улучшить почву для роста
- **собрать_урожай** — Собрать плоды с растений

Шаг 06:

Миссия 6

Макс создал класс "Растение" и написал код для создания объекта. Расположите шаги правильного создания объекта в порядке.

1. Определить класс с атрибутами и методами
2. Создать объект класса
3. Использовать объект (вызывать методы, обращаться к атрибутам)
4. Запустить программу

Подсказка: Класс должен быть определён до создания объектов.

Ответ:

- Определить класс с атрибутами и методами
- Создать объект класса
- Использовать объект (вызывать методы, обращаться к атрибутам)
- Запустить программу

Шаг 07:

Миссия 7

Аня хочет описать процесс полива растения в методе. Расположите действия в правильном порядке:

1. Увеличить количество воды в объекте
2. Проверить текущий уровень воды
3. Вывести сообщение о поливе
4. Вызвать метод полива из программы

Подсказка: Метод сначала проверяет состояние, потом изменяет его, затем сообщает пользователю.

Ответ:

- Вызвать метод полива из программы
- Проверить текущий уровень воды
- Увеличить количество воды в объекте
- Вывести сообщение о поливе

Шаг 08:

Миссия 8

Макс создал класс "Садовник" с методами для ухода за садом. Расположите этапы работы с объектом "Садовник" в правильном порядке:

1. Создать класс с описанием методов
2. Создать объект "садовник"
3. Вызвать метод полива

4. Проверить состояние растений

Подсказка: Сначала описываем класс, затем создаём объект и используем методы.

Ответ:

- Создать класс с описанием методов
- Создать объект "садовник"
- Вызвать метод полива
- Проверить состояние растений

Шаг 09:

Миссия 9

Аня пишет программу для создания объектов "Цветок" с разными параметрами. Расположите этапы создания объекта с параметрами в правильном порядке:

1. Определить метод `__init__` с параметрами
2. Создать объект, передав значения параметров
3. Определить класс
4. Использовать объект в программе

Подсказка: Метод `__init__` создаёт начальные параметры объекта.

Ответ:

- Определить класс
- Определить метод `__init__` с параметрами
- Создать объект, передав значения параметров
- Использовать объект в программе

Шаг 10:

Миссия 10

Анна решила создать первый объект — цветок в своём саду. Как в Python называется шаблон для создания объектов?

Подсказка: Это слово связано с планом или чертежом, по которому создаются экземпляры.

Ответ:

- **Класс**
- Функция
- Модуль
- Переменная

Шаг 11:

Миссия 11

Максим создал класс "Растение". Какой метод Python отвечает за создание нового объекта этого класса?

Подсказка: Этот метод начинается и заканчивается двойным подчёркиванием и часто называют конструктором.

Ответ:

- `__init__`
- `__str__`
- `__del__`

- `__call__`

🔗 Шаг 12:

Миссия 12

В саду Лены у каждого цветка есть имя и цвет. Как правильно определить эти характеристики в классе?

Подсказка: Это переменные, связанные с объектом, а не с самим классом.

Ответ:

- Создать атрибуты объекта в методе `__init__`
- Создать глобальные переменные вне класса
- Создать локальные переменные внутри функций вне класса
- Использовать только методы без атрибутов

🔗 Шаг 13:

Миссия 13

Сопоставьте термины из ООП с их правильным объяснением.

Подсказка: Вспомните, что отвечает за шаблон объекта, что за его конкретный пример, а что — за функции класса.

Термины:

Ответ:

- Класс
- Объект
- Метод
- Атрибут

Определения:

1. Конкретный экземпляр класса с уникальными свойствами.
2. Функция, связанная с классом или объектом.
3. Переменная, хранящая данные объекта.
4. Шаблон для создания объектов.

- **Класс** — Шаблон для создания объектов.
- **Объект** — Конкретный экземпляр класса с уникальными свойствами.
- **Метод** — Функция, связанная с классом или объектом.
- **Атрибут** — Переменная, хранящая данные объекта.

🔗 Шаг 14:

Миссия 14

В саду героя Пети есть разные растения. Сопоставьте типы данных с примерами атрибутов растений.

Подсказка: Подумайте, какие данные хранятся в числах, какие — в строках, а какие — в логическом формате.

Типы данных:

Ответ:

- `str`
- `int`
- `bool`

Примеры атрибутов:

1. Возраст растения в днях
 2. Название цветка
 3. Наличие плодов (да/нет)
- **str** — Название цветка
 - **int** — Возраст растения в днях
 - **bool** — Наличие плодов (да/нет)

Шаг 15:

Миссия 15

Команда садоводов создала класс "Питомник" с методами ухода за растениями. Сопоставьте методы с их описаниями.

Подсказка: Вспомните, что делают методы: полив, подкормка, сбор урожая.

Методы:

Ответ:

- `water()`
- `fertilize()`
- `harvest()`

Описание:

1. Сбор созревших плодов
 2. Добавление питательных веществ
 3. Полив растений
- **`water()`** — Полив растений
 - **`fertilize()`** — Добавление питательных веществ
 - **`harvest()`** — Сбор созревших плодов

Шаг 16:

Миссия 16

Расставьте в правильном порядке этапы создания объекта в Python.

Подсказка: Сначала нужно определить шаблон, затем создать объект, инициализировать его свойства и использовать объект.

1. Вызов конструктора класса
2. Определение класса
3. Использование объекта
4. Инициализация атрибутов в `__init__`

Ответ:

- Определение класса
- Вызов конструктора класса
- Инициализация атрибутов в `__init__`
- Использование объекта

Урок 6 - Садовод-бот — простейший класс Plant

Шаг 01:

Миссия 1

В саду Grow a Garden бот-садовод решил создать первый класс Plant. Какое из описаний лучше всего подходит для класса Plant?

Подсказка: Класс — это шаблон для создания похожих объектов с общими свойствами.

Ответ:

- **Шаблон для создания растений с определёнными свойствами и поведением**
- Готовый объект растения с конкретным состоянием
- Функция для обработки данных о растениях
- Текстовое описание сада

Шаг 02:

Миссия 2

Бот-садовод создаёт экземпляр класса Plant с именем "Роза". Как правильно создать объект растения в Python?

Подсказка: Для создания объекта нужно вызвать класс, используя круглые скобки.

Ответ:

- **rose = Plant("Роза")**
- Plant = "Роза"
- rose = "Роза"
- Plant.rose()

Шаг 03:

Миссия 3

Роза растёт и бот хочет сохранить её цвет. Какое свойство стоит добавить в класс Plant?

Подсказка: Цвет — это характеристика растения, которую удобно хранить в атрибуте объекта.

Ответ:

- **self.color = color**
- def color():
- color = "red"
- print(color)

Шаг 04:

Миссия 4

Помоги бот-садоводу понять, как связаны основные понятия ООП с растениями в саду Grow a Garden.

Понятия:

Ответ:

- Класс
- Объект
- Атрибут
- Метод

Определения:

1. Конкретное растение в саду с именем и характеристиками
2. Шаблон для создания растений с общими свойствами
3. Действие, которое растение может выполнять, например, расти
4. Характеристика растения, например, высота или цвет

Подсказка: Класс — это шаблон, объект — экземпляр этого шаблона, атрибуты описывают состояние, методы — поведение.

- **Класс** — Шаблон для создания растений с общими свойствами
- **Объект** — Конкретное растение в саду с именем и характеристиками
- **Атрибут** — Характеристика растения, например, высота или цвет
- **Метод** — Действие, которое растение может выполнять, например, расти

Шаг 05:

Миссия 5

Бот-садовод создаёт методы для класса Plant. Сопоставь метод с его описанием:

Ответ:

- grow()
- water()
- photosynthesis()
- harvest()

1. Позволяет растению увеличивать размер
2. Обеспечивает растение водой для жизни
3. Процесс превращения света в энергию
4. Сбор плодов с растения

Подсказка: Подумай, что делает каждое действие в реальном саду.

- **grow()** — Позволяет растению увеличивать размер
- **water()** — Обеспечивает растение водой для жизни
- **photosynthesis()** — Процесс превращения света в энергию
- **harvest()** — Сбор плодов с растения

Шаг 06:

Миссия 6

Чтобы создать метод grow, бот пишет:

```
def grow(self):  
    self.height += 1
```

Что значит self в этом методе?

Подсказка: self — это ссылка на текущий объект класса.

Ответ:

- **Текущий объект растения, на котором вызывается метод**
- Имя класса Plant
- Временная переменная для роста
- Функция увеличения размера

🔧 Шаг 07:

Миссия 7

Бот хочет, чтобы у каждого растения был свой уникальный цвет. Как правильно передать цвет при создании объекта?

Подсказка: Значения параметров нужно указывать в конструкторе класса.

Ответ:

- **def __init__(self, name, color):**
- **def __init__(self):**
- **def color(self):**
- **def Plant(color):**

🔧 Шаг 08:

Миссия 8

Садовод-бот Максим создал класс Plant, чтобы управлять растениями. Какое свойство лучше всего хранит название растения?

Подсказка: Имя — это обычно текстовая информация.

Ответ:

- **name** — строка с названием растения
- **age** — число лет растения
- **is_watering** — булево значение о поливе
- **height** — высота растения в сантиметрах

🔧 Шаг 09:

Миссия 9

Садовод-бот Аня хочет добавить свойство, которое показывает, полили ли растение сегодня. Какой тип данных лучше использовать?

Подсказка: Для ответа "да" или "нет" подходит специальный тип.

Ответ:

- **Булево (True/False)**
- Строка
- Число (int)
- Список

🔧 Шаг 10:

Миссия 10

Максим записал в класс Plant свойство "age" — возраст растения в днях. Что лучше использовать для этого свойства?

Подсказка: Возраст — целое количество дней.

Ответ:

- **Целое число (int)**
- Строка
- Список
- Булево

🔮 Шаг 11:

Миссия 11

Садовод-бот Аня хочет, чтобы при создании нового растения в классе Plant сразу задавалось имя и возраст. Какой метод нужно определить?

Подсказка: Инициализация объекта происходит в специальном методе.

Ответ:

- `__init__`
- `create`
- `start`
- `setup`

🔮 Шаг 12:

Миссия 12

Команда Grow a Garden хочет проверить свои знания о свойствах класса Plant. Сопоставьте свойства с их описанием.

Свойства:

Ответ:

- `name`
- `age`
- `is_watering`
- `height`

Описание:

1. Возраст растения в днях
2. Название растения
3. Булево значение: полили сегодня или нет
4. Высота растения в сантиметрах

Подсказка: Вспомните, какие данные хранятся в каждом свойстве.

- **name** — Название растения
- **age** — Возраст растения в днях
- **is_watering** — Булево значение: полили сегодня или нет
- **height** — Высота растения в сантиметрах

🔮 Шаг 13:

Миссия 13

Садовод-бот Максим изучает методы класса Plant. Сопоставьте методы с их действиями.

Методы:

Ответ:

- `water()`
- `grow()`
- `show_info()`
- `__init__()`

Описание:

1. Поливает растение, изменяя состояние

2. Инициализирует объект класса
3. Выводит информацию о растении
4. Увеличивает возраст и высоту растения

Подсказка: Подумайте, что делает каждый метод на основе его названия.

- **water()** — Поливает растение, изменяя состояние
- **grow()** — Увеличивает возраст и высоту растения
- **show_info()** — Выводит информацию о растении
- **__init__()** — Инициализирует объект класса

Шаг 14:

Миссия 14

Аня учит садовод-бота как правильно называть методы в классе Plant. Сопоставьте правила с примерами.

Правила:

Ответ:

- Методы должны быть глаголами
- Имена должны быть понятными
- Использовать маленькие буквы и подчеркивания
- Избегать длинных названий

Примеры:

1. water
2. show_plant_info
3. grow
4. this_method_is_way_too_long

Подсказка: Подумайте, какие названия более удобны и коротки.

- **Методы должны быть глаголами** — water
- **Имена должны быть понятными** — show_plant_info
- **Использовать маленькие буквы и подчеркивания** — grow
- **Избегать длинных названий** — this_method_is_way_too_long

Шаг 15:

Миссия 15

Команда Grow a Garden хочет написать метод grow(), который обновляет возраст и высоту растения. Расположите шаги написания метода в правильном порядке:

1. Добавить к возрасту +1 день
2. Увеличить высоту на заданное количество сантиметров
3. Определить метод grow(self, cm)
4. Сохранить изменения в свойствах объекта

Подсказка: Сначала нужно объявить метод, потом изменить свойства.

Ответ:

- Определить метод grow(self, cm)
- Добавить к возрасту +1 день
- Увеличить высоту на заданное количество сантиметров

- Сохранить изменения в свойствах объекта

Шаг 16:

Миссия 16

Аня пишет метод `water()`, который поливает растение. Расположите шаги в правильном порядке:

1. Установить `is_watering` в `True`
2. Определить метод `water(self)`
3. Вывести сообщение "Растение полито"
4. Вызвать метод `water` для нужного растения

Подсказка: Метод нужно сначала объявить, затем выполнять действия.

Ответ:

- Определить метод `water(self)`
- Установить `is_watering` в `True`
- Вывести сообщение "Растение полито"
- Вызвать метод `water` для нужного растения

Урок 7 - Журнал роста — класс с методами (`grow`, `show_status`)

Шаг 01:

Миссия 1

Садовод-бот Мия создала класс `Plant` для учета роста растений. Какой метод должен отвечать за увеличение роста растения?

Подсказка: Подумай, что значит "расти" для растения.

Ответ:

- **`grow()`**
- `show_status()`
- `plant()`
- `water()`

Шаг 02:

Миссия 2

Когда садовод-бот вызывает метод `show_status()`, что он должен показать?

Подсказка: Представь, что бот рассказывает о растении.

Ответ:

- **Текущий рост и состояние растения**
- Инструкции по уходу за растением
- Список всех растений в саду
- Время полива

Шаг 03:

Миссия 3

Мия заметила, что метод `grow()` должен принимать число — насколько растение выросло. Какой тип данных лучше всего подойдет для этого параметра?

Подсказка: Растения растут на определенное количество сантиметров или единиц.

Ответ:

- **int** или **float**
- **str**
- **bool**
- **list**

Шаг 04:

Миссия 4

Сопоставьте методы класса Plant с их назначением.

Ответ:

- **grow()**
- **show_status()**
- **__init__()**
- **water()**

Назначения:

1. Инициализация нового объекта растения
2. Увеличение роста растения
3. Показ текущего состояния растения
4. Полив растения (увлажнение почвы)

Подсказка: Посмотри на названия методов и подумай, что они делают.

- **grow()** — Увеличение роста растения
- **show_status()** — Показ текущего состояния растения
- **__init__()** — Инициализация нового объекта растения
- **water()** — Полив растения (увлажнение почвы)

Шаг 05:

Миссия 5

Связь методов класса Plant с ожидаемыми эффектами:

Ответ:

- **grow()**
- **show_status()**
- **fertilize()**

Эффекты:

1. Увеличение роста растения
2. Отображение роста и состояния растения
3. Улучшение здоровья и роста растения с помощью удобрений

Подсказка: Подумай, какой метод отвечает за что, учитывая названия и цели.

- **grow()** — Увеличение роста растения
- **show_status()** — Отображение роста и состояния растения
- **fertilize()** — Улучшение здоровья и роста растения с помощью удобрений

Шаг 06:

Миссия 6

Сопоставьте состояния растения с их описанием:

Ответ:

- Seedling
- Growing
- Blooming
- Wilting

Описание:

1. Молодое растение, только что проросшее из семени
2. Растение активно растет и набирает высоту
3. Растение цветет, показывая красоту
4. Растение теряет силу, начинает увядать

Подсказка: Это стадии роста и состояния растения.

- **Seedling** — Молодое растение, только что проросшее из семени
- **Growing** — Растение активно растет и набирает высоту
- **Blooming** — Растение цветет, показывая красоту
- **Wilting** — Растение теряет силу, начинает увядать

Шаг 07:

Миссия 7

Расположите правильный порядок действий, когда растение растет:

1. Увеличить рост с помощью метода `grow()`
2. Показать текущий рост через `show_status()`
3. Инициализировать новое растение
4. Позаботиться о растении (полив, удобрение)

Подсказка: Подумай, с чего начинается работа с растением, и что происходит дальше.

Ответ:

- Инициализировать новое растение
- Позаботиться о растении (полив, удобрение)
- Увеличить рост с помощью метода `grow()`
- Показать текущий рост через `show_status()`

Шаг 08:

Миссия 8

Расположите этапы работы метода `show_status()` в правильном порядке:

1. Сбор данных о росте и состоянии растения
2. Формирование текстового описания состояния
3. Вывод информации на экран
4. Обработка возможных ошибок (например, если данных нет)

Подсказка: Метод должен аккуратно представить информацию пользователю.

Ответ:

- Сбор данных о росте и состоянии растения
- Обработка возможных ошибок (например, если данных нет)
- Формирование текстового описания состояния
- Вывод информации на экран

 Шаг 09:

Миссия 9

Расставьте в правильном порядке этапы роста растения в саду Grow a Garden:

1. Появление ростка (seedling)
2. Активный рост (growing)
3. Цветение (blooming)
4. Увядание (wilting)
5. Уход за растением (полив, удобрение)

Подсказка: Садовод должен понимать последовательность жизни растения.

Ответ:

- Появление ростка (seedling)
- Уход за растением (полив, удобрение)
- Активный рост (growing)
- Цветение (blooming)
- Увядание (wilting)

 Шаг 10:

Миссия 10

Расположите последовательность вызовов методов для правильного учёта роста растения:

1. Создать объект растения
2. Вызвать `grow()` с количеством роста
3. Вызвать `show_status()` для проверки состояния
4. Повторять рост и проверку по циклу

Подсказка: Логика роста — это повторяющийся цикл ухода за растением.

Ответ:

- Создать объект растения
- Вызвать `grow()` с количеством роста
- Вызвать `show_status()` для проверки состояния
- Повторять рост и проверку по циклу

 Шаг 11:

Миссия 11

Анна выращивает в своём саду новые цветы. Она хочет записывать рост каждого цветка. Какой инструмент Python поможет ей хранить данные и управлять ростом растений?

Подсказка: Подумайте, что удобно использовать, если у каждого цветка есть состояние и действия.

Ответ:

- Класс с методами `grow` и `show_status`

- Обычный список без функций
- Простой текстовый файл
- Использовать только переменные

🔮 Шаг 12:

Миссия 12

Максим добавил метод `grow` в класс растения. Что должен делать этот метод?

Подсказка: `grow` отвечает за изменение роста, а что конкретно меняется?

Ответ:

- **Увеличивать текущий рост растения**
- Показывать статус растения
- Удалять растение из сада

🔮 Шаг 13:

Миссия 13

Валерия хочет, чтобы метод `show_status` показывал рост цветка. Какой из вариантов вывода наиболее понятен пользователю?

Подсказка: Вывод должен быть информативным и коротким.

Ответ:

- **"Цветок вырос до 15 см"**
- "show_status вызван"
- "Рост: неизвестен"

🔮 Шаг 14:

Миссия 14

Сопоставьте методы класса растения с их назначением:

Ответ:

- `grow`
- `show_status`
- `__init__`

Назначения:

1. Инициализация объекта с начальными данными
2. Показывает текущий рост растения
3. Увеличивает рост растения на заданное значение

Подсказка: Метод `__init__` запускается при создании объекта.

- **`grow`** — Увеличивает рост растения на заданное значение
- **`show_status`** — Показывает текущий рост растения
- **`__init__`** — Инициализация объекта с начальными данными

🔮 Шаг 15:

Миссия 15

Герои `Grow` и `Garden` используют класс `Plant`. Сопоставьте свойства с их описаниями:

Ответ:

- height
- color
- species

Описание:

1. Высота растения в сантиметрах
2. Вид растения (например, роза, тюльпан)
3. Цвет цветка или листьев

Подсказка: Свойство height связано с числом, остальные — с описаниями.

- **height** — Высота растения в сантиметрах
- **color** — Цвет цветка или листьев
- **species** — Вид растения (например, роза, тюльпан)

 Шаг 16:

Миссия 16

В саду у героя Алексея несколько растений. Как лучше всего организовать объекты растений для удобства управления их ростом?

Подсказка: Нужно хранить все растения вместе, чтобы быстро проходить по ним и вызывать методы.

Ответ:

- **Список объектов класса Plant**
- Отдельные переменные для каждого растения
- Обычный текстовый файл

 Урок 8 - Цветущий сад — итоговый мини-проект класс растений + файл хране

 Шаг 01:

Миссия 1

Анна выращивает в своём саду новые цветы. Она хочет записывать рост каждого цветка. Какой инструмент Python поможет ей хранить данные и управлять ростом растений?

Подсказка: Подумайте, что удобно использовать, если у каждого цветка есть состояние и действия.

Ответ:

- **Класс с методами grow и show_status**
- Обычный список без функций
- Простой текстовый файл
- Использовать только переменные

 Шаг 02:

Миссия 2

Максим добавил метод grow в класс растения. Что должен делать этот метод?

Подсказка: grow отвечает за изменение роста, а что конкретно меняется?

Ответ:

- **Увеличивать текущий рост растения**
- Показывать статус растения
- Удалять растение из сада

🔮 **Шаг 03:**

Миссия 3

Валерия хочет, чтобы метод `show_status` показывал рост цветка. Какой из вариантов вывода наиболее понятен пользователю?

Подсказка: Вывод должен быть информативным и коротким.

Ответ:

- **"Цветок вырос до 15 см"**
- "show_status вызван"
- "Рост: неизвестен"

🔮 **Шаг 04:**

Миссия 4

Сопоставьте методы класса растения с их назначением:

Ответ:

- `grow`
- `show_status`
- `__init__`

Назначения:

1. Инициализация объекта с начальными данными
2. Показывает текущий рост растения
3. Увеличивает рост растения на заданное значение

Подсказка: Метод `__init__` запускается при создании объекта.

- **`grow`** — Увеличивает рост растения на заданное значение
- **`show_status`** — Показывает текущий рост растения
- **`__init__`** — Инициализация объекта с начальными данными

🔮 **Шаг 05:**

Миссия 5

Герои `Grow a Garden` используют класс `Plant`. Сопоставьте свойства с их описаниями:

Ответ:

- `height`
- `color`
- `species`

Описание:

1. Высота растения в сантиметрах
2. Вид растения (например, роза, тюльпан)
3. Цвет цветка или листьев

Подсказка: Свойство `height` связано с числом, остальные — с описаниями.

- **height** — Высота растения в сантиметрах
- **color** — Цвет цветка или листьев
- **species** — Вид растения (например, роза, тюльпан)

🔮 Шаг 06:

Миссия 6

В саду у героя Алексея несколько растений. Как лучше всего организовать объекты растений для удобства управления их ростом?

Подсказка: Нужно хранить все растения вместе, чтобы быстро проходить по ним и вызывать методы.

Ответ:

- **Список объектов класса Plant**
- Отдельные переменные для каждого растения
- Обычный текстовый файл

🔮 Шаг 07:

Миссия 7

Сопоставьте этапы работы метода `grow` с их описаниями:

Ответ:

- Получить текущее значение роста
- Добавить заданное значение к росту
- Обновить значение роста
- Вернуть обновлённый рост

Подсказка: Представьте, как метод изменяет внутреннее состояние объекта.

- **Получить текущее значение роста** — Начинается с чтения текущего роста растения
- **Добавить заданное значение к росту** — К текущему росту прибавляется число сантиметров
- **Обновить значение роста** — Записать новое значение обратно в объект
- **Вернуть обновлённый рост** — Метод возвращает новое значение роста

🔮 Шаг 08:

Миссия 8

Расположите правильный порядок вызова методов для учета роста растения за несколько дней:

1. Создать объект класса `Plant` с начальными параметрами
2. Вызвать метод `grow` с увеличением роста
3. Вывести текущий статус растения
4. Повторить рост на следующий день

Подсказка: Сначала объект создается, потом растёт, потом показываем статус.

Ответ:

- Создать объект класса `Plant` с начальными параметрами
- Вызвать метод `grow` с увеличением роста
- Вывести текущий статус растения
- Повторить рост на следующий день

Шаг 09:

Миссия 9

Расположите действия для создания класса Plant с методами grow и show_status в правильном порядке:

1. Определить класс Plant
2. Создать метод __init__, чтобы задать начальный рост
3. Написать метод grow, который увеличивает рост
4. Написать метод show_status, который выводит рост

Подсказка: Сначала нужно определить класс, потом его методы по очереди.

Ответ:

- Определить класс Plant
- Создать метод __init__, чтобы задать начальный рост
- Написать метод grow, который увеличивает рост
- Написать метод show_status, который выводит рост

Шаг 10:

Миссия 10

Расположите этапы тестирования метода grow:

1. Создать объект Plant с начальным ростом
2. Вызвать метод grow с увеличением на 5 см
3. Проверить, что рост увеличился на 5 см
4. Вывести сообщение об успешном тесте

Подсказка: Тест начинается с подготовки объекта, затем вызова метода, проверки результата и сообщения.

Ответ:

- Создать объект Plant с начальным ростом
- Вызвать метод grow с увеличением на 5 см
- Проверить, что рост увеличился на 5 см
- Вывести сообщение об успешном тесте

Шаг 11:

Миссия 11

Аня хочет хранить данные о каждом цветке в саду, чтобы легко отслеживать их рост. Что в Python лучше использовать для такого удобного хранения и управления?

Подсказка: Подумайте, что удобно использовать, если у каждого цветка есть свои характеристики и действия.

Ответ:

- **Класс с методами для работы с растением**
- Простой список чисел
- Текстовый файл с описанием
- Отдельные переменные для каждого растения

Шаг 12:

Миссия 12

Максим добавил метод `grow` в класс `Plant`. Что должен делать этот метод?

Подсказка: `grow` отвечает за рост — что конкретно изменяется?

Ответ:

- **Увеличивать текущую высоту растения**
- Показывать статус растения
- Удалять растение из списка

🔮 Шаг 13:

Миссия 13

Валя хочет, чтобы метод `show_status` выводил рост цветка. Какой вариант вывода наиболее понятен пользователю?

Подсказка: Вывод должен быть информативным и коротким.

Ответ:

- **"Цветок вырос до 15 см"**
- "show_status вызван"
- "Рост неизвестен"

🔮 Шаг 14:

Миссия 14

Сопоставьте методы класса `Plant` с их назначением:

Ответ:

- `grow`
- `show_status`
- `__init__`

Назначения:

1. Инициализация объекта с начальными параметрами
2. Показывает текущий рост растения
3. Увеличивает рост растения на заданное значение

Подсказка: Метод `__init__` вызывается автоматически при создании объекта.

- **`grow`** — Увеличивает рост растения на заданное значение
- **`show_status`** — Показывает текущий рост растения
- **`__init__`** — Инициализация объекта с начальными параметрами

🔮 Шаг 15:

Миссия 15

Сопоставьте свойства класса `Plant` с их описаниями:

Ответ:

- `height`
- `color`
- `species`

Описание:

1. Высота растения в сантиметрах

2. Цвет цветка или листьев
3. Вид растения (например, роза, тюльпан)

Подсказка: height — число, остальные — слова.

- **height** — Высота растения в сантиметрах
- **color** — Цвет цветка или листьев
- **species** — Вид растения (например, роза, тюльпан)

Шаг 16:

Миссия 16

У Алексея в саду несколько растений. Как удобнее всего хранить объекты растений для управления их ростом?

Подсказка: Нужно иметь быстрый доступ к каждому растению и легко вызывать методы.

Ответ:

- **Список объектов класса Plant**
- Отдельные переменные для каждого растения
- Хранить растения в текстовом файле